THE KUBERNETES HANDBOOK

From Zero to Production

A complete beginner-to-advanced guide, with a hands-on DevOps capstone



Containers • Architecture • Workloads • Networking • Storage • Security

Observability • Helm • Operators • GitOps • CI/CD • Production

Edition 2026

Preface

Who this handbook is for

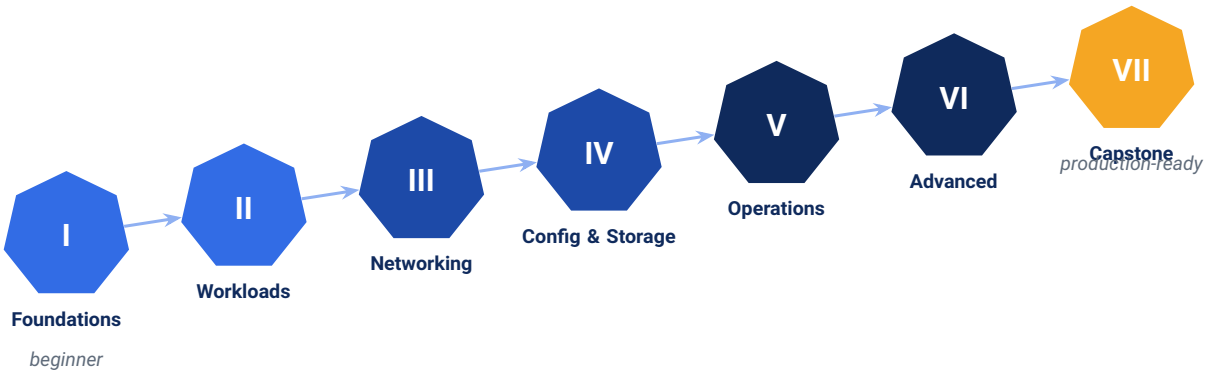
This handbook is written for someone who has *never* touched Kubernetes. You do not need to know what a container is, what "orchestration" means, or why everyone keeps talking about YAML. If you can open a terminal, run a command, and read a little code, you have everything you need.

By the last page you will be able to:

- explain, in plain words, what Kubernetes is and why it exists;
- describe every major component of a cluster and what it does;
- write and apply manifests for Pods, Deployments, Services, Gateways, ConfigMaps, Secrets, volumes, Jobs, StatefulSets and more;
- secure, scale, observe, and troubleshoot real workloads;
- package applications with Helm and Kustomize, extend Kubernetes with CRDs and Operators, and deliver changes with CI/CD and GitOps;
- take a real multi-service application from source code all the way to a monitored, autoscaled, GitOps-managed deployment.

How the book is organised

The book climbs a staircase. Each part assumes only what came before it.



Part I – Foundations. Containers, the problem Kubernetes solves, cluster architecture, installing a local cluster, and the `kubectl`/YAML workflow.

Part II – Running Workloads. Pods, labels and namespaces, Deployments and rollouts, StatefulSets, DaemonSets, Jobs and CronJobs.

Part III – Networking. Services, DNS, Ingress, the Gateway API, and NetworkPolicies.

Part IV – Configuration & Storage. ConfigMaps, Secrets, Volumes, PersistentVolumes and StorageClasses.

Part V – Operating Kubernetes. Requests and limits, scheduling, autoscaling, security, logging, metrics and tracing.

Part VI – Advanced Kubernetes. Helm, Kustomize, CRDs, Operators, GitOps, CI/CD, and running clusters in production.

Part VII – Capstone. We build **TaskFlow**, a small but realistic three-tier application, and do *all* of the DevOps work on Kubernetes: images, manifests, Kustomize overlays, a Helm chart, a CI pipeline, GitOps with Argo CD, autoscaling, network policies, monitoring and alerts.

How to read the boxes

Throughout the book, coloured boxes highlight different kinds of information:

> **Key Idea**
> A concept you must remember. If you only read the Key Idea boxes, you will still understand the shape of Kubernetes.

> **Real-World Analogy**
> An everyday comparison that makes an abstract idea concrete.

> **Try It Yourself**
> A short hands-on exercise. Do them; Kubernetes is learned with your fingers.

> **Watch Out**
> A common mistake, a trap, or something that can hurt you in production.

> **Pro Tip**
> A habit or shortcut used by experienced engineers.

> **Under the Hood**
> Optional detail about how something works internally. Skip on first reading.

Conventions

Commands you type are shown in terminal boxes. Files are shown with their file name in the title bar:

```
terminal
# Lines starting with # are comments; you do not type them.
kubectl get pods
```

```
hello.yaml
apiVersion: v1
kind: Pod
metadata:
  name: hello
```

Kubernetes moves quickly. Everything here targets Kubernetes **v1.30 and newer**. Where a tool version appears (for example a Helm chart version), check the project's site for the latest release before you copy it.

Trademarks

Kubernetes and the Kubernetes logo are trademarks of The Linux Foundation. The Kubernetes resource icons come from the official `kubernetes/community` repository. Docker, Helm, Prometheus, Argo, etcd, containerd, Istio and other names and logos belong to their respective owners and are used here only to identify those projects.

Contents

Preface i

I Foundations 1

1 Before Kubernetes: Applications and Containers 2

1.1 The problem: “it works on my machine” . . . 2
1.2 Virtual machines: the first answer . . . 2
1.3 Containers: a lighter answer . . . 3
1.4 The container vocabulary . . . 4
1.5 Your first Dockerfile . . . 4
1.6 Good image habits . . . 5
1.7 Why containers are not enough . . . 6

2 What Is Kubernetes? 7

2.1 A one-sentence definition . . . 7
2.2 A short history . . . 7
2.3 What Kubernetes gives you . . . 8
2.4 The big idea: declarative desired state . . . 8
2.5 The reconciliation loop . . . 8
2.6 What Kubernetes is not . . . 9
2.7 Where Kubernetes runs . . . 9

3 Kubernetes Architecture 11

3.1 The cluster at a glance . . . 11
3.2 Control-plane components . . . 12
3.3 Node components . . . 13
3.4 Add-ons you will meet . . . 13
3.5 Walkthrough: what happens when you create a Deployment . . . 14
3.6 Kubernetes objects: spec and status . . . 14

4 Setting Up Your Kubernetes Lab **16**

4.1 Choosing a local cluster . . . 16
4.2 Install the tools . . . 16
4.3 Create a multi-node cluster . . . 17
4.4 kubeconfig and contexts . . . 18
4.5 Helpful extras . . . 19

5 Speaking Kubernetes: kubectl and YAML **20**

5.1 YAML in ten minutes . . . 20
5.2 Anatomy of a manifest . . . 21
5.3 Discovering the API yourself . . . 21
5.4 Imperative vs. declarative . . . 22
5.5 The everyday command set . . . 22

II Running Workloads **24**

6 Pods: The Smallest Unit **25**

6.1 What is a Pod? . . . 25
6.2 Your first real Pod . . . 26
6.3 Command, args, and environment . . . 26
6.4 The Pod lifecycle . . . 27
6.5 Multi-container Pods . . . 28
6.6 Health checks: probes . . . 29
6.7 Graceful shutdown . . . 30
6.8 Static Pods and Pod templates . . . 30

7 Organising: Labels, Selectors, Annotations, Namespaces **32**

7.1 Labels: sticky notes on objects . . . 32
7.2 Selectors: queries over labels . . . 32
7.3 Annotations: notes for tools and humans . . . 33
7.4 Namespaces: rooms in the house . . . 34

8 ReplicaSets and Deployments **36**

8.1 The ownership chain . . . 36
8.2 A complete Deployment . . . 37

8.3 Rolling updates . . . 37
8.4 Managing rollouts . . . 38
8.5 Release strategies . . . 39

9 StatefulSets, DaemonSets, Jobs, and CronJobs 40
9.1 Choosing a workload type . . . 40
9.2 StatefulSets . . . 40
9.3 DaemonSets . . . 42
9.4 Jobs . . . 42
9.5 CronJobs . . . 43

III Networking 45

10 Services and Cluster DNS 46
10.1 The networking model . . . 46
10.2 What a Service is . . . 46
10.3 Service types . . . 48
10.4 Cluster DNS . . . 49
10.5 Useful Service options . . . 49
10.6 Port-forward vs. Service . . . 50

11 Ingress, Gateway API, and Network Policies 51
11.1 The problem: many services, one front door . . . 51
11.2 Ingress . . . 52
11.3 Gateway API . . . 52
11.4 TLS certificates with cert-manager . . . 54
11.5 Network Policies: a firewall for Pods . . . 54
11.6 CNI: the network plugin . . . 56

IV Configuration & Storage 57

12 ConfigMaps and Secrets 58
12.1 Config belongs outside the image . . . 58
12.2 ConfigMaps . . . 58
12.3 Secrets . . . 60

12.4 Production secret management 61

13 Volumes and Persistent Storage **63**

13.1 The problem 63
13.2 Ephemeral volumes 63
13.3 Persistent storage: three objects 64
13.4 Access modes 65
13.5 Lifecycle and reclaim policy 65
13.6 Expansion and snapshots 66
13.7 CSI: the storage plugin model 66

V Operating Kubernetes **68**

14 Resources and Scheduling **69**

14.1 Requests and limits 69
14.2 Quality of Service classes 70
14.3 Namespace guard rails 70
14.4 Controlling where Pods run 71
14.5 Voluntary disruptions 73

15 Autoscaling **75**

15.1 Three ways to scale 75
15.2 Prerequisite: metrics-server 75
15.3 Horizontal Pod Autoscaler 76
15.4 Vertical Pod Autoscaler 77
15.5 Scaling the cluster itself 78
15.6 Event-driven autoscaling with KEDA 78

16 Security **79**

16.1 Defence in depth: the 4 Cs 79
16.2 Who are you? Authentication 79
16.3 What may you do? RBAC 80
16.4 Hardening Pods: securityContext 81
16.5 Pod Security Standards 82
16.6 Policy as code 82

16.7 Supply-chain security . . . 83
16.8 Runtime security and auditing . . . 83
16.9 Hardening checklist . . . 84

17 Observability: Logs, Metrics, Traces 85
17.1 Why observability? . . . 85
17.2 Logs . . . 85
17.3 Metrics with Prometheus . . . 86
17.4 What to measure: golden signals . . . 87
17.5 Alerts . . . 88
17.6 Traces with OpenTelemetry . . . 88
17.7 Kubernetes events . . . 89

VI Advanced Kubernetes 90

18 Packaging: Helm and Kustomize 91
18.1 The problem . . . 91
18.2 Kustomize . . . 91
18.3 Helm . . . 93
18.4 Which one should I use? . . . 95

19 Extending Kubernetes: CRDs, Controllers, Operators 96
19.1 Kubernetes as a platform for platforms . . . 96
19.2 CustomResourceDefinitions . . . 96
19.3 Writing a controller . . . 97
19.4 Operators . . . 98
19.5 Admission webhooks . . . 99
19.6 Other extension points . . . 99

20 CI/CD and GitOps 101
20.1 From commit to cluster . . . 101
20.2 Push vs. pull . . . 101
20.3 A CI pipeline with GitHub Actions . . . 102
20.4 GitOps with Argo CD . . . 102
20.5 Environment promotion . . . 104

20.6 Progressive delivery . . . 104

21 Kubernetes in Production 106

21.1 Managed or self-managed? . . . 106
21.2 High availability . . . 106
21.3 Upgrades . . . 107
21.4 Backup and disaster recovery . . . 107
21.5 Multi-cluster . . . 108
21.6 Service mesh . . . 108
21.7 Cost management . . . 108
21.8 Go-live checklist . . . 109

VII Capstone Project 110

22 Capstone, Part 1: The TaskFlow Codebase 111

22.1 What we are building . . . 111
22.2 Repository layout . . . 112
22.3 The API . . . 113
22.4 The web frontend . . . 117
22.5 Container images . . . 119
22.6 Run the API locally first . . . 120

23 Capstone, Part 2: DevOps for TaskFlow on Kubernetes 122

23.1 The road map . . . 122
23.2 Step 1 — Cluster and platform add-ons . . . 122
23.3 Step 2 — Build images and load them into kind . . . 123
23.4 Step 3 — The base manifests . . . 124
23.5 Step 4 — Environment overlays . . . 133
23.6 Step 5 — Deploy and verify . . . 136
23.7 Step 6 — Day-2 drills . . . 138
23.8 Step 7 — Monitoring and alerting . . . 140
23.9 Step 8 — Continuous Integration . . . 142
23.10 Step 9 — GitOps with Argo CD . . . 144
23.11 Optional — Packaging TaskFlow as a Helm chart . . . 146
23.12 What you have built . . . 148

23.13 Where to go next . . . 148

A kubectl Cheat Sheet 150

A.1 Context and configuration . . . 150
A.2 Viewing resources . . . 150
A.3 Creating and changing . . . 150
A.4 Rollouts . . . 151
A.5 Debugging . . . 151
A.6 Nodes . . . 151
A.7 JSONPath and output tricks . . . 152
A.8 Short names . . . 152
A.9 The resource icon set . . . 152

B Troubleshooting Playbook 153

B.1 The universal first five commands . . . 153
B.2 Decision tree: my app is not reachable . . . 154
B.3 Pod status reference . . . 155
B.4 Networking checks . . . 155
B.5 Node and cluster checks . . . 156
B.6 Habits that prevent incidents . . . 156

C Glossary and Learning Path 157

C.1 Glossary . . . 157
C.2 Your learning path . . . 160
C.3 Certifications . . . 160
C.4 Where to keep learning . . . 160



PART I

Foundations

Containers, the big idea, the architecture, and your first cluster

CHAPTER 1

1 Before Kubernetes: Applications and Containers



In this chapter you will learn

- why “it works on my machine” is the oldest problem in software;
- what a container is, and how it differs from a virtual machine;
- the vocabulary: image, container, registry, tag, layer;
- how to write a Dockerfile and build, run, and push an image.

Kubernetes is a system for running *containers*. Before we can understand Kubernetes, we need to understand what it is managing. This chapter is your crash course.

1.1 The problem: “it works on my machine”

Imagine you have written a small web API in Python. On your laptop it runs perfectly. You copy it to a server and it crashes: the server has a different Python version, a missing system library, and a different configuration file. A teammate tries it on a Mac and gets a third set of errors.

Every application depends on much more than its own source code:

- a language runtime (Python 3.12, Node 22, Java 21, . . .);
- libraries and their exact versions;
- operating-system packages (OpenSSL, libpq, fonts, timezone data);
- environment variables and configuration files;
- a particular directory layout and file permissions.

Historically, teams solved this with long installation documents and careful manual server setup. That is slow, fragile, and impossible to repeat exactly.

1.2 Virtual machines: the first answer

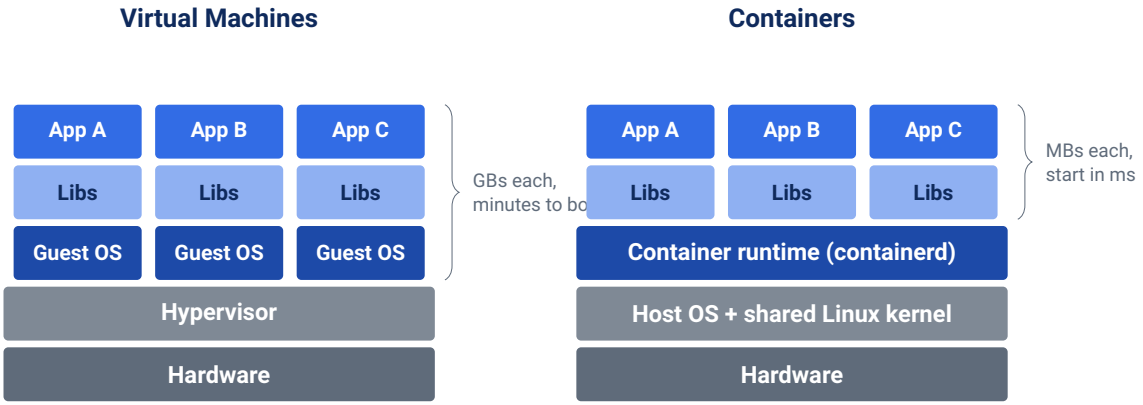
A **virtual machine** (VM) emulates a whole computer. You install a full operating system inside it, then your application. A VM image can be copied anywhere and behaves the same. The cost: every VM carries a complete operating system kernel, takes gigabytes of disk, and needs a minute or more to boot.

1.3 Containers: a lighter answer

A **container** packages an application *together with everything it needs to run* — runtime, libraries, OS packages, config — but it does *not* include its own kernel. All containers on a machine share the host's Linux kernel. The kernel uses two features to keep containers apart:

- **namespaces** give each container its own private view of processes, network interfaces, mount points, and hostnames;
- **cgroups** (control groups) limit how much CPU, memory, and I/O a container may use.

So a container is really just an ordinary Linux process that has been given blinkers (namespaces) and a budget (cgroups).



Real-World Analogy

A virtual machine is a *detached house*: its own foundations, plumbing and roof. A container is an *apartment* in a building: it has its own locked front door and furniture, but shares the foundations and plumbing (the kernel) with its neighbours. Apartments are cheaper and quicker to build.

1.4 The container vocabulary

Term	Meaning
Image	A read-only, versioned package of an application and its filesystem. Think of it as a *class* or a *recipe*.
Container	A running instance of an image. Think *object* or *the cooked meal*. Many containers can run from one image.
Layer	Images are built in stacked, cached layers. Changing one line of your Dockerfile only rebuilds the layers after it.
Registry	A server that stores images: Docker Hub, GitHub Container Registry (`ghcr.io`), Amazon ECR, Google Artifact Registry, Harbor…
Tag	A human label for an image version, e.g. `nginx:1.27`.
Digest	An immutable content hash, e.g. `nginx@sha256:4c0f...`. A tag can be moved; a digest cannot.
Dockerfile	The text recipe used to build an image.
OCI	The Open Container Initiative: the open standard for image and runtime formats. Docker, Podman, containerd, and Kubernetes all speak OCI.

1.5 Your first Dockerfile

Let us containerise a tiny Python web server. Create a folder with two files.

app.py

```python
from http.server import BaseHTTPRequestHandler, HTTPServer
import os, socket

class Hello(BaseHTTPRequestHandler):
    def do_GET(self):
        msg = f"Hello from {socket.gethostname()} (version {os.getenv('VERSION', '1')})\n"
        self.send_response(200)
        self.end_headers()
        self.wfile.write(msg.encode())

HTTPServer(("0.0.0.0", 8080), Hello).serve_forever()
```

Dockerfile

```dockerfile
# 1. Start from an official, small base image
FROM python:3.12-slim

# 2. Work inside /app in the image
WORKDIR /app

# 3. Copy our code into the image
COPY app.py .

# 4. Run as a non-root user (good security habit)
RUN useradd --uid 10001 appuser
```

```
USER 10001

# 5. Document the port and define the start command
EXPOSE 8080
CMD ["python", "app.py"]
```

Now build, run, and test it:

```
>_ terminal
# Build an image called hello with tag 1.0 from the current folder (.)
docker build -t hello:1.0 .

# Run it in the background (-d), mapping host port 8080 to container port 8080
docker run -d --name hello -p 8080:8080 hello:1.0

curl http://localhost:8080
# Hello from 3f2a9c1b7e44 (version 1)

docker ps            # list running containers
docker logs hello    # see its output
docker stop hello && docker rm hello
```

1.5.1 Pushing to a registry

Kubernetes nodes pull images from a registry, so images must be pushed somewhere the cluster can reach.

```
>_ terminal
docker login ghcr.io -u YOUR_GITHUB_USER
docker tag hello:1.0 ghcr.io/YOUR_GITHUB_USER/hello:1.0
docker push ghcr.io/YOUR_GITHUB_USER/hello:1.0
```

1.6 Good image habits

- **Use small, official base images** (`-slim`, `-alpine`, or "distroless"). Less software means fewer vulnerabilities.
- **Pin versions.** `python:3.12-slim` is better than `python:latest`; a digest is better still.
- **Order layers from least to most frequently changed**: install dependencies first, copy source code last, so rebuilds are fast.
- **Use multi-stage builds** to compile in one stage and copy only the result into a small runtime stage.
- **Run as non-root.** Kubernetes can enforce this for you later.
- **One process per container.** Let the platform handle restarts and scaling instead of running a process supervisor inside.
- **Log to stdout/stderr**, never to files inside the container.

1.7 Why containers are not enough

Containers solve packaging. But once you run real systems, new questions appear:

- You have 40 containers across 10 servers. Which server should a new one go on?
- A server dies at 3 a.m. Who restarts its containers somewhere else?
- Traffic triples on Black Friday. Who starts more copies, and who removes them afterwards?
- How do containers find each other when their IP addresses keep changing?
- How do you roll out version 2 without downtime, and roll back if it is broken?
- Where do passwords and configuration live?

Answering these questions automatically is called **container orchestration**. And the de-facto standard orchestrator is Kubernetes.

Chapter Recap

- Containers package an app with its dependencies but share the host kernel.
- An *image* is the recipe; a *container* is a running instance.
- Images are built from a Dockerfile and stored in a registry.
- Running many containers reliably across many machines needs an orchestrator — that is Kubernetes.

2 CHAPTER 2 What Is Kubernetes?

In this chapter you will learn

- what Kubernetes is, where it came from, and why it won;
- the single most important idea: *declarative desired state*;
- the control loop that makes Kubernetes "self-healing";
- what Kubernetes does *not* do for you.

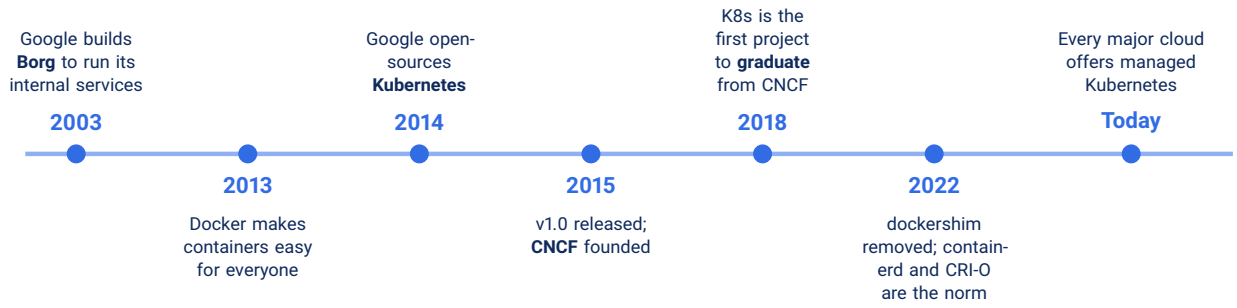
2.1 A one-sentence definition

Key Idea

Kubernetes is an open-source platform that runs and manages containerised applications across a group of machines, continuously working to keep the system in the state you described.

The name comes from Greek *kubernētēs*, meaning "helmsman" or "pilot" — the person who steers a ship. That is why the logo is a ship's wheel with seven spokes. You will often see it written **K8s**: a "K", then 8 letters, then an "s".

2.2 A short history



Kubernetes is now maintained by thousands of contributors under the CLOUD NATIVE COMPUTING FOUNDATION Cloud Native Computing Foundation. A new minor version ships roughly every four months, and each is supported for about a year.

2.3 What Kubernetes gives you

Capability	What it means for you
Scheduling	You say “run 5 copies”; Kubernetes picks the machines with enough free capacity.
Self-healing	Crashed containers are restarted; Pods on a dead machine are recreated elsewhere.
Service discovery & load balancing	Every app gets a stable name and virtual IP; traffic is spread across healthy copies.
Rolling updates & rollbacks	New versions replace old ones gradually, with automatic rollback on demand.
Autoscaling	Add or remove copies (or whole machines) based on load.
Config & secrets	Inject configuration and credentials without rebuilding images.
Storage orchestration	Attach cloud disks, NFS, or local volumes to the apps that need them.
Extensibility	Teach Kubernetes new resource types and automate anything with Operators.

2.4 The big idea: declarative desired state

There are two ways to tell a system what to do.

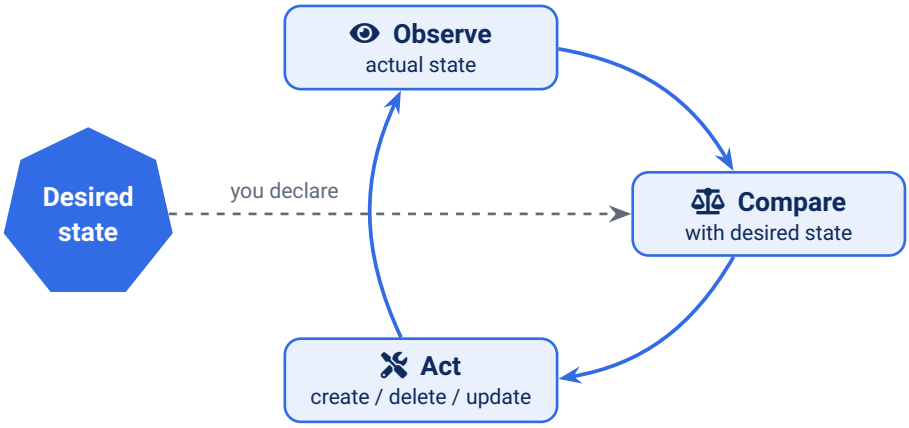
- **Imperative**: give step-by-step commands. “Start a container on server 3. Now start another on server 5. Now open port 80.”
- **Declarative**: describe the end result. “I want 3 copies of `web:2.1` running, reachable on port 80.”

Kubernetes is declarative. You write down the **desired state** in a file (called a **manifest**, usually YAML) and hand it to the cluster. Kubernetes stores it, compares it with the **actual state** of the world, and takes whatever actions are needed to make reality match your wish.

> **Real-World Analogy**
>
> A thermostat. You do not tell the heater “run for 12 minutes”. You set the desired temperature to 21 °C. The thermostat measures the room, compares, and switches the heater on or off — forever. If someone opens a window, it reacts automatically. Kubernetes is a thermostat for your software.

2.5 The reconciliation loop

Inside Kubernetes, many small programs called **controllers** each run the same endless loop:



Example: you declare "3 replicas". One machine crashes, taking a copy with it. The controller observes 2, compares with 3, and acts by creating one more. No human involved. This is what people mean by *self-healing*.

> **Key Idea**
>
> In Kubernetes you almost never "do" things directly. You *change the desired state*, and controllers do the work. Learn to think in terms of "what should exist", not "what should I run".

2.6 What Kubernetes is not

- It does **not** build your source code or images (CI tools do).
- It is **not** a Platform-as-a-Service like Heroku out of the box; you assemble the platform from building blocks.
- It does **not** provide databases, message queues or logging by itself, although it can run all of them.
- It is **not** always the right tool. A single small app may be happier on one VM or a serverless platform. Kubernetes shines when you have many services, teams, or environments.

2.7 Where Kubernetes runs

- **Local, for learning**: kind, minikube, k3d, Docker Desktop, Rancher Desktop.
- **Managed cloud**: Amazon EKS, Google GKE, Azure AKS, DigitalOcean DOKS, Linode LKE, and others. The provider runs the control plane for you.
- **Self-managed**: `kubeadm`, k3s, RKE2, Talos, OpenShift, on your own VMs or bare metal.

The same manifests work everywhere. That portability is a huge part of why Kubernetes won.

Chapter Recap

- Kubernetes (K8s) orchestrates containers across many machines.
- You declare desired state; controllers reconcile actual state toward it.
- That loop gives you self-healing, scaling, and safe rollouts.
- Kubernetes is a foundation for building platforms, not a finished PaaS.

3

CHAPTER 3

Kubernetes Architecture



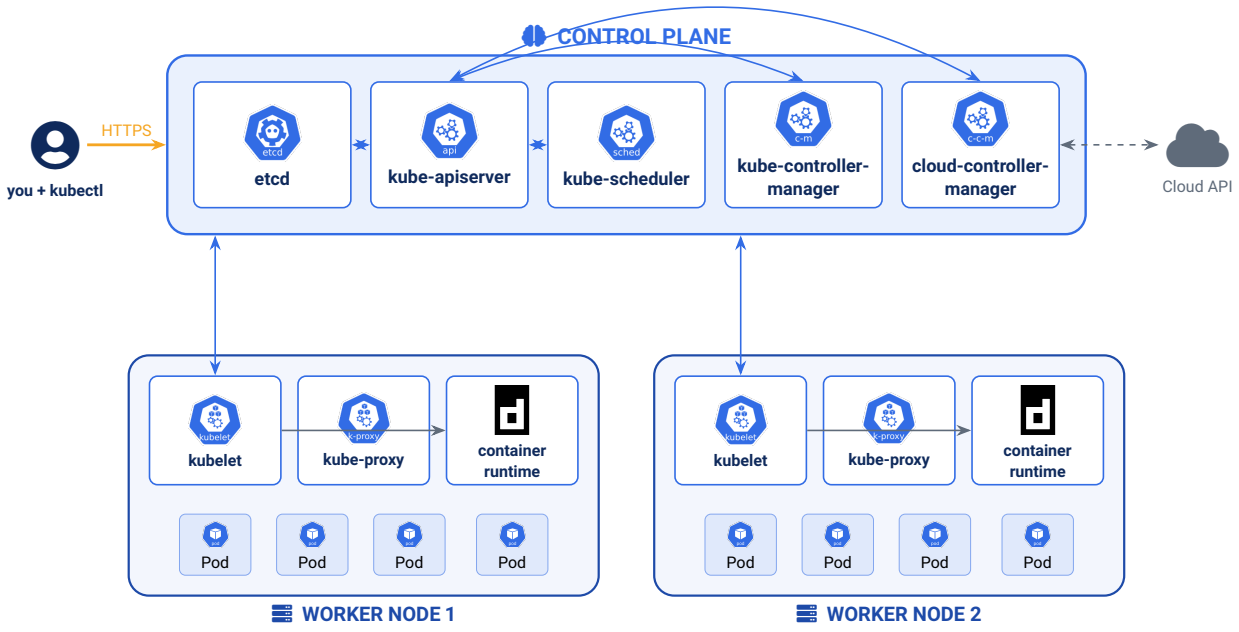
In this chapter you will learn

- what a cluster, a node, and the control plane are;
- the job of every control-plane component: API server, etcd, scheduler, controller manager, cloud controller manager;
- the job of every node component: kubelet, kube-proxy, container runtime;
- what actually happens, step by step, when you create a Deployment.

3.1 The cluster at a glance

A Kubernetes **cluster** is a set of machines (physical or virtual) called **nodes**, split into two roles:

- the **control plane** — the "brain" that stores desired state and makes decisions;
- **worker nodes** — the "muscle" that actually runs your containers.



> **Key Idea**
>
> **Everything talks to the API server, and only the API server talks to etcd.** The scheduler, controllers, kubelets, `kubectl`, and your CI system are all just clients of one HTTP API.

3.2 Control-plane components

3.2.1 kube-apiserver — the front door

The API server exposes the Kubernetes REST API. Every create, read, update, or delete — from you, from a controller, or from a node — goes through it. For each request it:

1. **authenticates** the caller (who are you? certificate, token, OIDC);
2. **authorises** the action (are you allowed? usually RBAC);
3. runs **admission control** (should this be modified or rejected? e.g. inject defaults, enforce policies);
4. **validates** the object against its schema;
5. **persists** it to etcd and notifies anyone *watching*.

The API server is stateless, so you can run several copies behind a load balancer.

3.2.2 etcd — the memory

etcd is a distributed, strongly consistent key–value store. It holds the *entire* state of the cluster: every Pod, Service, Secret, and config. It uses the Raft consensus algorithm, so production clusters run 3 or 5 etcd members and tolerate the loss of 1 or 2.

> **Watch Out**
>
> If you lose etcd and have no backup, you lose your cluster's definition. Back it up regularly (Chapter 21). Managed services do this for you.

3.2.3 kube-scheduler — the matchmaker

When a new Pod has no node assigned, the scheduler picks one. It works in two phases:

- **Filtering**: remove nodes that cannot run the Pod (not enough CPU/memory, wrong labels, taints, missing volumes, port conflicts).
- **Scoring**: rank the remaining nodes (spread replicas apart, prefer nodes with the image cached, balance utilisation) and pick the best.

The scheduler only *decides*. It writes the chosen node name into the Pod object; it does not start anything.

3.2.4 kube-controller-manager — the autopilots

A single binary running dozens of controllers, each owning one kind of object: the Deployment controller, ReplicaSet controller, Node controller (notices dead nodes), Job controller, EndpointSlice controller, ServiceAccount controller, and many more. Each runs the reconcile loop from Chapter 2.

3.2.5 cloud-controller-manager — the cloud bridge

Only present on cloud clusters. It talks to the cloud provider's API to create load balancers for Services, attach routes, and learn when cloud VMs are deleted.

3.3 Node components

3.3.1 kubelet — the node agent

Runs on every node. It watches the API server for Pods assigned to its node, asks the container runtime to start their containers, mounts volumes, runs health probes, and reports status back. If a container dies, the kubelet restarts it.

3.3.2 kube-proxy — the traffic cop

Implements Services on each node by programming packet-forwarding rules (iptables, IPVS, or nftables) so that traffic to a Service's virtual IP lands on a healthy Pod. Some network plugins (like Cilium) replace kube-proxy with eBPF.

3.3.3 Container runtime

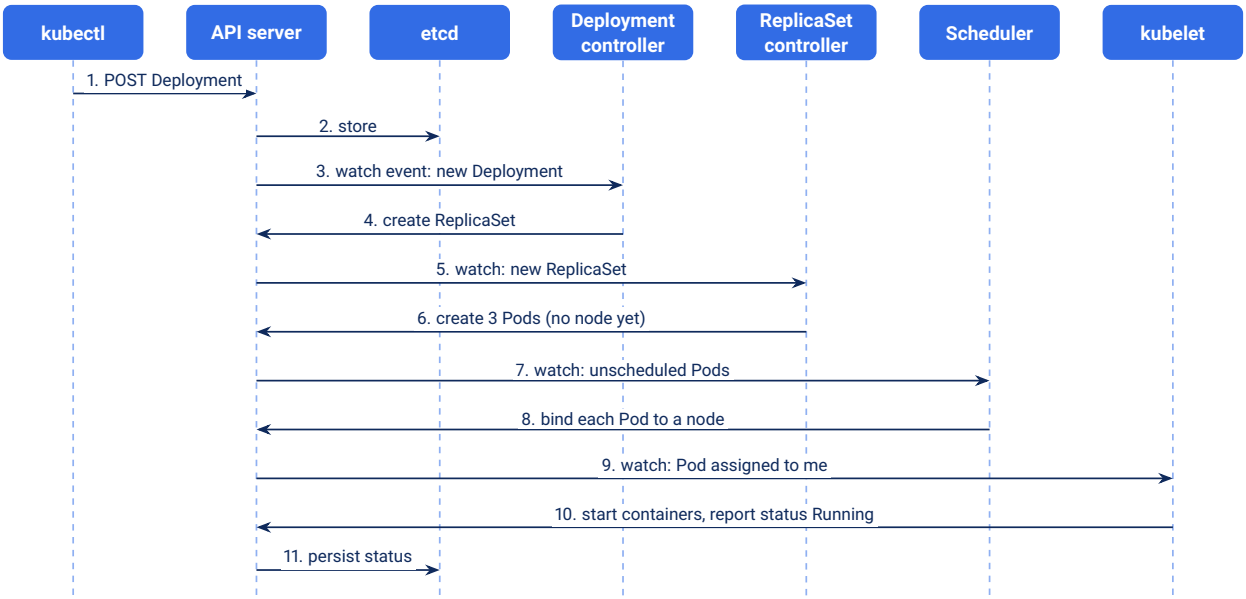
The software that actually pulls images and runs containers — **containerd** or **CRI-O**. The kubelet talks to it through the **Container Runtime Interface** (CRI). Images built with Docker run fine, because they are standard OCI images.

3.4 Add-ons you will meet

Add-on	Purpose
CoreDNS	Cluster DNS: lets Pods reach `my-service.my-namespace` by name.
CNI plugin	Pod networking. Examples: Calico, Cilium, Flannel, kindnet.
CSI drivers	Storage plugins for cloud disks, NFS, Ceph, etc.
metrics-server	Live CPU/memory figures for `kubectl top` and autoscaling.
Ingress / Gateway controller	HTTP(S) routing from outside the cluster.

3.5 Walkthrough: what happens when you create a Deployment

You run `kubectl apply -f web.yaml`, asking for a Deployment with 3 replicas.



Notice that no component orders another around. Each one simply *watches* the API for objects it cares about and reacts. This loose coupling is what makes Kubernetes so robust and extensible.

> **Under the Hood**
>
> Controllers do not poll. They open long-lived *watch* connections to the API server and keep a local cache (an "informer"). Every object carries a `resourceVersion`; updates use optimistic concurrency, so two controllers can never silently overwrite each other's changes.

3.6 Kubernetes objects: spec and status

Almost every object you create has two halves:

- `spec` — written by *you*: the desired state;
- `status` — written by *Kubernetes*: the observed state.

Controllers spend their lives trying to make `status` match `spec`.

> **Chapter Recap**
>
> - A cluster = control plane (API server, etcd, scheduler, controller managers) + worker nodes (kubelet, kube-proxy, runtime).
> - The API server is the only door; etcd is the only memory.
> - Components cooperate by watching the API, not by calling each other.

- Objects have a `spec` (desired) and a `status` (actual).



CHAPTER 4

Setting Up Your Kubernetes Lab



In this chapter you will learn

- which local cluster tool to choose (kind, minikube, k3d, Docker Desktop);
- how to install `kubectl`, Docker, kind, and Helm on Windows, macOS, and Linux;
- how to create a multi-node cluster and verify it works;
- what a kubeconfig, a context, and a namespace default are.

4.1 Choosing a local cluster

You do not need a cloud account to learn Kubernetes. All of these run a real cluster on your laptop:

Tool	How it works	Best for
kind	Each node is a Docker container. Multi-node in seconds.	CI pipelines, multi-node experiments. **Used in this book.**
minikube	A VM or container with many built-in add-ons.	Beginners who like `minikube addons enable ...`.
k3d	Runs the lightweight k3s distribution in Docker.	Very fast, low memory. Built-in load balancer.
Docker Desktop	One-click single-node cluster.	Simplest possible start on Windows/macOS.

Pro Tip

Give Docker at least **4 CPUs and 8 GB RAM** for the later chapters (especially monitoring). On Windows, use Docker Desktop with the WSL 2 backend.

4.2 Install the tools

You need four command-line tools: **Docker** (to run containers and kind), **kubectl** (to talk to clusters), **kind** (to create local clusters), and **Helm** (the package manager, used from Chapter 18).

4.2.1 Windows (PowerShell)

PowerShell

```
# Install Docker Desktop from https://www.docker.com/products/docker-desktop/
winget install -e --id Kubernetes.kubectl
winget install -e --id Kubernetes.kind
winget install -e --id Helm.Helm
```

4.2.2 macOS (Homebrew)

terminal

```
brew install --cask docker
brew install kubectl kind helm
```

4.2.3 Linux

terminal

```
# kubectl (latest stable)
curl -LO "https://dl.k8s.io/release/$(curl -Ls https://dl.k8s.io/release/stable.txt)/bin/linux/
    ↪ amd64/kubectl"
sudo install -m 0755 kubectl /usr/local/bin/kubectl

# kind (check https://kind.sigs.k8s.io for the latest version)
curl -Lo ./kind https://kind.sigs.k8s.io/dl/latest/kind-linux-amd64
sudo install -m 0755 kind /usr/local/bin/kind

# Helm
curl -fsSL https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
```

Check everything:

terminal

```
docker version
kubectl version --client
kind version
helm version
```

4.3 Create a multi-node cluster

A single-node cluster hides interesting behaviour (scheduling, spreading, node failure). Let us create one control-plane node and two workers.

kind-cluster.yaml

```
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
name: lab
nodes:
```

```
  - role: control-plane
  - role: worker
  - role: worker
```

terminal

```
kind create cluster --config kind-cluster.yaml
kubectl cluster-info --context kind-lab
kubectl get nodes -o wide
```

You should see something like:

output

```
NAME                STATUS   ROLES           AGE   VERSION
lab-control-plane   Ready    control-plane   60s   v1.33.1
lab-worker          Ready    <none>          40s   v1.33.1
lab-worker2         Ready    <none>          40s   v1.33.1
```

Look inside: the control-plane components themselves run as Pods!

terminal

```
kubectl get pods -n kube-system
# coredns-..., etcd-lab-control-plane, kube-apiserver-lab-control-plane,
# kube-controller-manager-..., kube-scheduler-..., kube-proxy-..., kindnet-...
```

4.4 kubeconfig and contexts

How does `kubectl` know which cluster to talk to? It reads a **kubeconfig** file, by default `~/.kube/config`. That file holds three lists:

- **clusters** — API server addresses and CA certificates;
- **users** — credentials (certificates, tokens, or plugins);
- **contexts** — a named pairing of *cluster + user + default namespace*.

terminal

```
kubectl config get-contexts          # list contexts (* = current)
kubectl config current-context       # e.g. kind-lab
kubectl config use-context kind-lab  # switch cluster
kubectl config set-context --current --namespace=dev   # change default namespace
```

Watch Out

The most expensive mistake in Kubernetes is running a command against the *wrong cluster*. Always check your context before destructive commands. Tools like `kubectx`/`kubens` and a shell prompt that shows the current context are worth installing on day one.

4.5 Helpful extras

- **Shell completion**: `kubectl completion bash|zsh|powershell`.
- **An alias**: `alias k=kubectl` (PowerShell: `Set-Alias k kubectl`).
- **k9s**: a terminal UI to browse and manage the cluster.
- **stern**: tail logs from many Pods at once.
- **VS Code / Cursor Kubernetes extension**: YAML schema validation and auto-completion.

>_ Try It Yourself

Create the `lab` cluster, then delete one worker's container with `docker stop lab-worker2`. Run `kubectl get nodes -w` and watch the node turn `NotReady` after about 40 seconds. Start it again with `docker start lab-worker2`. You just simulated a machine failure.

Cleaning up

>_ terminal

```
kind delete cluster --name lab
```

Chapter Recap

- kind runs a full multi-node cluster inside Docker containers.
- `kubectl` reads kubeconfig; a *context* = cluster + user + namespace.
- Even control-plane components run as Pods in `kube-system`.



CHAPTER 5

Speaking Kubernetes: kubectl and YAML



In this chapter you will learn

- just enough YAML to read and write any manifest;
- the four fields every Kubernetes object has;
- imperative commands vs. declarative `kubectl apply`;
- the twenty `kubectl` commands you will use every day;
- how to discover fields yourself with `kubectl explain`.

5.1 YAML in ten minutes

YAML is a human-friendly way to write structured data. Kubernetes accepts JSON too, but everyone writes YAML. There are only a few rules.

yaml-basics.yaml

```yaml
# A comment
name: web                  # key: value (a "map" entry)
replicas: 3                # numbers are not quoted
enabled: true              # booleans
version: "1.0"             # quote strings that look like numbers

labels:                    # a nested map (indent with SPACES, usually 2)
  app: web
  tier: frontend

ports:                     # a list: each item starts with "- "
  - 80
  - 443

containers:                # a list of maps
  - name: nginx
    image: nginx:1.27
  - name: sidecar
    image: busybox:1.36

script: |                  # multi-line string, newlines kept
  echo hello
  echo world
---
# "---" separates multiple documents in one file
```

```
kind: Second
```

> **Watch Out**
>
> **Never use tabs** in YAML, and be careful with indentation: it defines structure. A key indented by one extra space moves into a different object, and the error message will be confusing. Use an editor with a YAML extension.

5.2 Anatomy of a manifest

Every Kubernetes object has the same skeleton:

pod.yaml

```yaml
apiVersion: v1            # 1. which API group/version defines this kind
kind: Pod                 # 2. what type of object
metadata:                 # 3. identity: name, namespace, labels, annotations
  name: hello
  namespace: default
  labels:
    app: hello
spec:                     # 4. desired state (different for every kind)
  containers:
    - name: hello
      image: nginx:1.27
      ports:
        - containerPort: 80
```

Field	Explanation
`apiVersion`	Core objects use `v1`. Others are `group/version`, e.g. `apps/v1`, `batch/v1`, `networking.k8s.io/v1`.
`kind`	The object type: Pod, Deployment, Service, ConfigMap…
`metadata`	`name` must be unique per kind per namespace. Labels and annotations live here.
`spec`	What you want. Kubernetes adds a `status` section later.

5.3 Discovering the API yourself

You never need to memorise fields. Ask the cluster:

terminal

```
kubectl api-resources                    # every kind, its short name, and API group
kubectl explain pod                      # docs for a kind
kubectl explain pod.spec.containers      # drill into fields
kubectl explain deployment.spec.strategy --recursive
```

Short names save typing: `po` (pods), `deploy`, `svc`, `ns`, `cm` (configmaps), `pvc`, `sts`, `ds`, `ing`, `hpa`, `sa`.

5.4 Imperative vs. declarative

```terminal
# Imperative: tell Kubernetes what to DO (fine for quick experiments)
kubectl run hello --image=nginx:1.27
kubectl create deployment web --image=nginx:1.27 --replicas=3
kubectl scale deployment web --replicas=5
kubectl delete deployment web

# Declarative: tell Kubernetes what should EXIST (use this for real work)
kubectl apply -f pod.yaml          # create or update to match the file
kubectl apply -f ./manifests/      # a whole folder
kubectl diff  -f pod.yaml          # preview what would change
kubectl delete -f pod.yaml
```

Key Idea

Files in Git are the source of truth; `kubectl apply` makes the cluster match them. You can re-run `apply` a hundred times and nothing changes unless the file changes. This property (idempotence) is what makes GitOps possible later.

Pro Tip

Let `kubectl` write YAML for you, then edit it:

```terminal
kubectl create deployment web --image=nginx:1.27 --dry-run=client -o yaml > web.yaml
```

5.5 The everyday command set

```terminal
# --- Looking around ------------------------------------------------------
kubectl get pods                          # list in the current namespace
kubectl get pods -A                       # all namespaces
kubectl get pods -o wide                  # + node and IP
kubectl get pod hello -o yaml             # full object, including status
kubectl get deploy,svc,pods -l app=web    # several kinds, filtered by label
kubectl get pods -w                       # watch for changes
kubectl describe pod hello                # human summary + EVENTS (read these!)
kubectl get events --sort-by=.lastTimestamp

# --- Debugging -----------------------------------------------------------
kubectl logs hello                        # container stdout/stderr
kubectl logs hello -c sidecar -f          # a specific container, follow
kubectl logs deploy/web --previous        # logs of the crashed previous run
kubectl exec -it hello -- sh              # open a shell inside the container
kubectl port-forward pod/hello 8080:80    # localhost:8080 -> pod port 80
kubectl debug -it hello --image=busybox:1.36 --target=hello   # ephemeral debug container
kubectl top pods                          # CPU/memory (needs metrics-server)
```

```
# --- Changing things ----------------------------------------------------
kubectl apply -f file.yaml
kubectl edit deployment web              # edit live object in your $EDITOR
kubectl set image deployment/web nginx=nginx:1.28
kubectl label pod hello env=dev
kubectl annotate pod hello owner=team-a
kubectl delete pod hello
```

5.5.1 Output formats

terminal

```
kubectl get pods -o name
kubectl get pods -o jsonpath='{.items[*].metadata.name}'
kubectl get pods -o custom-columns=NAME:.metadata.name,NODE:.spec.nodeName
kubectl get pod hello -o json | jq '.status.podIP'
```

Try It Yourself

1. Save `pod.yaml` above and run `kubectl apply -f pod.yaml`.
2. Run `kubectl get pod hello -o yaml` and find the `status` section Kubernetes added. What is `status.podIP`?
3. Port-forward with `kubectl port-forward pod/hello 8080:80` and open `http://localhost:8080`.
4. Change the image to `nginx:1.28` in the file and apply again. What happens? (Hint: some Pod fields are immutable — read the error.)
5. Clean up with `kubectl delete -f pod.yaml`.

Chapter Recap

- Manifests = `apiVersion` + `kind` + `metadata` + `spec`.
- Use `kubectl explain` and `api-resources` instead of memorising.
- Prefer declarative `apply` with files in Git.
- `describe` and `get events` are your first debugging tools.



PART II

Running Workloads

Pods, labels, Deployments, and every other way to run code



CHAPTER 6

Pods: The Smallest Unit



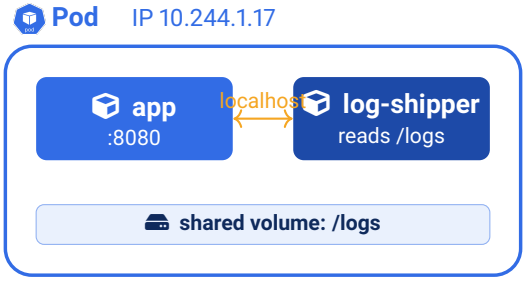
In this chapter you will learn

- what a Pod is and why Kubernetes does not run bare containers;
- the Pod lifecycle, phases, and restart policies;
- commands, arguments, and environment variables;
- multi-container Pods: init containers and sidecars;
- health checks: liveness, readiness, and startup probes;
- graceful shutdown.

6.1 What is a Pod?

A **Pod** is a group of one or more containers that are always scheduled together on the same node and share:

- one **network identity** — a single IP address and port space (containers in a Pod reach each other on `localhost`);
- **storage volumes** that any container in the Pod can mount;
- a **lifecycle** — they are created and destroyed together.



Real-World Analogy

A Pod is like a *shared office*. The people (containers) inside share one street address (IP), one mailbox (port space), and filing cabinets (volumes). Most offices have a single occupant; occasionally an assistant moves in.

> **Key Idea**
>
> Pods are **ephemeral and disposable**. They are not repaired; they are replaced. A replaced Pod gets a new name and a new IP. That is why you will almost never create Pods directly — you will use a Deployment (Chapter 8) and a Service (Chapter 10).

6.2 Your first real Pod

hello-pod.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: hello
  labels:
    app: hello
spec:
  containers:
    - name: web
      image: nginx:1.27
      ports:
        - containerPort: 80
          name: http
      env:
        - name: GREETING
          value: "Hello, Kubernetes"
      resources:              # covered in depth in Chapter 14
        requests:
          cpu: 50m
          memory: 64Mi
        limits:
          memory: 128Mi
```

terminal

```
kubectl apply -f hello-pod.yaml
kubectl get pod hello -o wide
kubectl describe pod hello      # read the Events at the bottom
kubectl exec -it hello -- sh -c 'echo $GREETING'
kubectl delete pod hello
```

6.3 Command, args, and environment

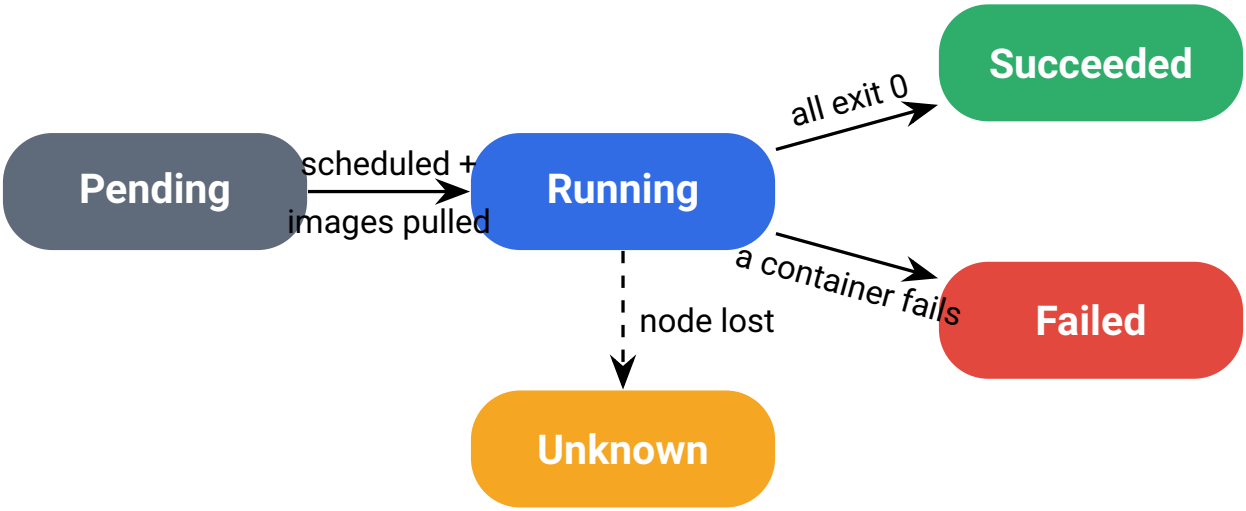
Containers start with the image's `ENTRYPOINT` and `CMD`. In Kubernetes you can override them: `command` replaces `ENTRYPOINT`, `args` replaces `CMD`.

busybox-pod.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
```

```
  name: counter
spec:
  restartPolicy: OnFailure
  containers:
    - name: counter
      image: busybox:1.36
      command: ["sh", "-c"]
      args: ["for i in 1 2 3 4 5; do echo tick $i from $POD_NAME; sleep 1; done"]
      env:
        - name: POD_NAME              # the Downward API: facts about the Pod itself
          valueFrom:
            fieldRef:
              fieldPath: metadata.name
        - name: NODE_NAME
          valueFrom:
            fieldRef:
              fieldPath: spec.nodeName
```

6.4 The Pod lifecycle



Phase	Meaning
Pending	Accepted, but not yet running: waiting to be scheduled or pulling images.
Running	Bound to a node; at least one container is running.
Succeeded	All containers exited successfully and will not restart.
Failed	All containers have stopped and at least one failed.
Unknown	The node stopped reporting.

The `STATUS` column of `kubectl get pods` often shows more detailed reasons. Learn these by heart:

Status	Usual cause
`ContainerCreating`	Pulling image, mounting volumes. Normal for a few seconds.
`ImagePullBackOff`	Wrong image name/tag, private registry without credentials.
`CrashLoopBackOff`	The container keeps exiting. Check `kubectl logs -previous`.
`OOMKilled`	Exceeded its memory limit.
`Pending` (long)	No node has enough CPU/memory, or a PVC is unbound. Check events.
`CreateContainerConfi`	Referenced ConfigMap/Secret/key does not exist.

6.4.1 Restart policy

`spec.restartPolicy` tells the kubelet what to do when a container exits: `Always` (default; for long-running services), `OnFailure` (for batch jobs), or `Never`. Restarts use exponential back-off: 10 s, 20 s, 40 s … up to 5 minutes — hence "CrashLoop*BackOff*".

6.5 Multi-container Pods

6.5.1 Init containers

Init containers run *to completion, in order*, before the main containers start. Use them to wait for a dependency, run migrations, or prepare files.

init-demo.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: init-demo
spec:
  initContainers:
    - name: wait-for-db
      image: busybox:1.36
      command: ["sh", "-c", "until nc -z postgres 5432; do echo waiting; sleep 2; done"]
  containers:
    - name: app
      image: nginx:1.27
```

6.5.2 Sidecar containers

A **sidecar** runs alongside the main container for its whole life: a log shipper, a proxy, a config reloader. Since Kubernetes 1.29 the recommended way is a *native sidecar*: an init container with `restartPolicy: Always`. It starts before the app, keeps running, and is stopped after the app.

sidecar-demo.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
```

```yaml
  name: sidecar-demo
spec:
  volumes:
    - name: logs
      emptyDir: {}
  initContainers:
    - name: log-shipper              # native sidecar
      image: busybox:1.36
      restartPolicy: Always
      command: ["sh", "-c", "touch /logs/app.log; tail -F /logs/app.log"]
      volumeMounts:
        - name: logs
          mountPath: /logs
  containers:
    - name: app
      image: busybox:1.36
      command: ["sh", "-c", "while true; do date >> /logs/app.log; sleep 2; done"]
      volumeMounts:
        - name: logs
          mountPath: /logs
```

terminal

```
kubectl apply -f sidecar-demo.yaml
kubectl logs sidecar-demo -c log-shipper -f
```

6.6 Health checks: probes

The kubelet can check your container's health in three different ways, each with a different consequence:

Probe	Question it asks	If it fails...
startup	"Has the app finished starting?"	Other probes wait. After `failureThreshold`, the container is restarted.
liveness	"Is the app still alive, or stuck?"	The container is **restarted**.
readiness	"Can the app serve traffic right now?"	The Pod is **removed from Service endpoints** (no restart).

Probes can use `httpGet`, `tcpSocket`, `grpc`, or `exec` (run a command; exit code 0 means healthy).

probes.yaml

```yaml
containers:
  - name: api
    image: ghcr.io/example/api:1.4.0
    ports:
      - containerPort: 8000
    startupProbe:
      httpGet: { path: /healthz, port: 8000 }
      periodSeconds: 5
```

```
    failureThreshold: 24       # allow up to 2 minutes to start
  livenessProbe:
    httpGet: { path: /healthz, port: 8000 }
    periodSeconds: 10
    failureThreshold: 3
  readinessProbe:
    httpGet: { path: /readyz, port: 8000 }
    periodSeconds: 5
    failureThreshold: 2
```

Watch Out

Do **not** check your database inside the *liveness* probe. If the database blips, every replica fails liveness and Kubernetes restarts your whole fleet — turning a small outage into a big one. Dependencies belong in *readiness*; liveness should only answer "is this process wedged?".

6.7 Graceful shutdown

When a Pod is deleted (scale-down, rollout, node drain), Kubernetes:

1. marks the Pod `Terminating` and removes it from Service endpoints;
2. runs any `preStop` hook;
3. sends `SIGTERM` to each container's main process;
4. waits up to `terminationGracePeriodSeconds` (default 30 s);
5. sends `SIGKILL` to anything still running.

graceful.yaml

```yaml
spec:
  terminationGracePeriodSeconds: 45
  containers:
    - name: api
      image: ghcr.io/example/api:1.4.0
      lifecycle:
        preStop:
          exec:
            command: ["sh", "-c", "sleep 5"]   # let load balancers notice first
```

Pro Tip

Make your app handle `SIGTERM`: stop accepting new requests, finish in-flight ones, close DB connections, exit. A short `preStop` sleep avoids dropped requests while endpoint removal propagates across the cluster.

6.8 Static Pods and Pod templates

Two more terms you will see:

- **Static Pods** are defined by files on a node (e.g. `/etc/kubernetes/manifests`) and run directly by the kubelet. That is how `kubeadm` runs the control plane itself.
- A **Pod template** is the `template:` section inside Deployments, Jobs, StatefulSets, and so on — a blueprint those controllers stamp out Pods from.

Chapter Recap

- A Pod = one or more containers sharing network, storage, and lifecycle.
- Pods are cattle, not pets: replaced, never repaired.
- Init containers prepare; native sidecars accompany; the app does the work.
- Startup, liveness, and readiness probes answer different questions.
- Handle `SIGTERM` for zero-downtime shutdowns.



CHAPTER 7

Organising: Labels, Selectors, Annotations, Namespaces



In this chapter you will learn

- how labels and selectors glue Kubernetes objects together;
- the recommended label set;
- when to use annotations instead of labels;
- how namespaces divide a cluster, and what they do *not* isolate.

7.1 Labels: sticky notes on objects

A **label** is a key/value pair attached to an object's `metadata`. Labels mean nothing to Kubernetes on their own — *you* decide what they mean — but they are how objects find each other.

labels.yaml

```yaml
metadata:
  name: api-7d9f8c-xk2lp
  labels:
    app.kubernetes.io/name: taskflow-api
    app.kubernetes.io/part-of: taskflow
    app.kubernetes.io/version: "1.4.0"
    app.kubernetes.io/component: backend
    environment: production
    team: payments
```

7.2 Selectors: queries over labels

A **selector** picks objects by their labels. You will see selectors everywhere:

- a **Service** sends traffic to Pods whose labels match its selector;
- a **Deployment/ReplicaSet** owns the Pods matching its selector;
- a **NetworkPolicy** applies to Pods matching its selector;
- `kubectl` filters with `-l`.

terminal

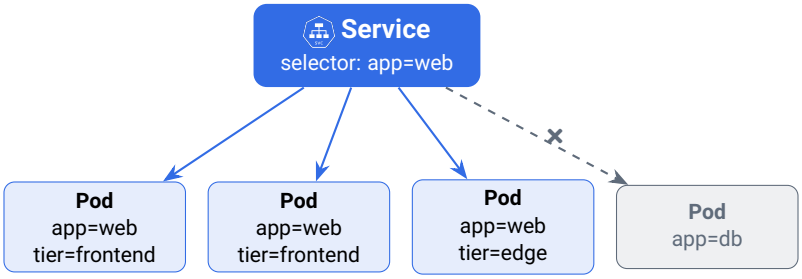
```
kubectl get pods -l app=web                        # equality
kubectl get pods -l 'environment in (staging,production)'   # set-based
kubectl get pods -l 'app=web,tier!=cache'          # AND, not-equal
kubectl get pods --show-labels
kubectl get pods -L app,version                    # labels as columns
```

In YAML, selectors come in two flavours:

selectors.yaml

```yaml
# Simple form (Services)
selector:
  app: web

# Rich form (Deployments, Jobs, NetworkPolicies...)
selector:
  matchLabels:
    app: web
  matchExpressions:
    - key: tier
      operator: In            # In, NotIn, Exists, DoesNotExist
      values: [frontend, edge]
```



Watch Out

A Deployment's `spec.selector` is **immutable** and must match its Pod template labels. Choose stable labels (like `app.kubernetes.io/name`), never something that changes per release like a version number.

7.3 Annotations: notes for tools and humans

Annotations are also key/value metadata, but they are *not* used for selecting. Values can be long (even JSON). Tools read them to change behaviour.

annotations.yaml

```yaml
metadata:
  annotations:
    kubernetes.io/change-cause: "upgrade to 1.4.0 for faster search"
    prometheus.io/scrape: "true"
    owner-contact: "payments-oncall@example.com"
    description: "Public API for task management"
```

	Labels	Annotations
Used by selectors	Yes	No
Value size	Short (63 chars)	Large (up to 256 KB total)
Typical use	Identity, grouping, routing	Tool configuration, docs, audit trail

7.4 Namespaces: rooms in the house

A **namespace** is a virtual partition of a cluster. Names need only be unique *within* a namespace, so team A and team B can both have a Deployment called `api`.

terminal

```
kubectl get namespaces
# default, kube-system, kube-public, kube-node-lease  (+ your own)

kubectl create namespace dev
kubectl apply -f app.yaml -n dev
kubectl get pods -n dev
kubectl get pods --all-namespaces      # or -A
kubectl config set-context --current --namespace=dev
```

namespace.yaml

```
apiVersion: v1
kind: Namespace
metadata:
  name: dev
  labels:
    team: platform
    pod-security.kubernetes.io/enforce: baseline   # see Chapter 16
```

7.4.1 What namespaces give you

- a scope for names;
- a boundary for **RBAC** permissions (Chapter 16);
- a boundary for **ResourceQuotas** and **LimitRanges** (Chapter 14);
- a target for **NetworkPolicies** (Chapter 11);
- a DNS segment: `api.dev.svc.cluster.local`.

Watch Out

By default, namespaces are **not** a network or security wall. A Pod in `dev` can talk to a Pod in `prod` unless you add NetworkPolicies, and Pods on the same node share a kernel. For hard multi-tenancy, use separate clusters or strong policy tooling.

7.4.2 Cluster-scoped objects

Some objects do not live in any namespace: Nodes, Namespaces themselves, PersistentVolumes, StorageClasses, ClusterRoles, CRDs. Check with `kubectl api-resources -namespaced=false`.

7.4.3 A common layout

- one namespace *per application per environment* (`shop-dev`, `shop-staging`) on shared clusters, or
- one namespace *per application* with separate clusters per environment (recommended for production isolation);
- platform tools in their own namespaces: `monitoring`, `argocd`, `cert-manager`, `ingress`.

Chapter Recap

- Labels identify and group; selectors query labels.
- Services, Deployments, and policies all find Pods via selectors.
- Annotations carry non-identifying metadata for tools.
- Namespaces scope names, RBAC, and quotas — but are not a firewall by themselves.

CHAPTER 8

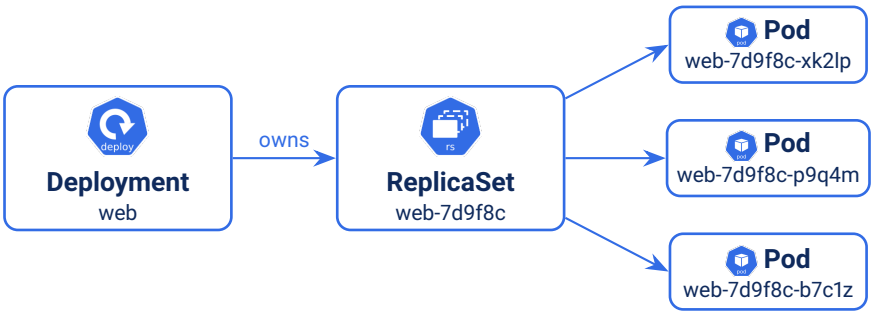
8 ReplicaSets and Deployments



In this chapter you will learn

- how a ReplicaSet keeps N copies of a Pod alive;
- why you use Deployments instead of ReplicaSets directly;
- rolling updates, `maxSurge` and `maxUnavailable`;
- checking, pausing, and rolling back rollouts;
- release strategies: recreate, rolling, blue/green, canary.

8.1 The ownership chain



- A **ReplicaSet** has one job: ensure exactly `replicas` Pods matching its selector exist. Too few? Create. Too many? Delete.
- A **Deployment** manages ReplicaSets to provide *updates*: each new version of the Pod template gets a new ReplicaSet, and the Deployment shifts replicas from old to new.

Key Idea

For stateless applications, the **Deployment** is the object you create. You will rarely write a ReplicaSet yourself, and almost never a bare Pod.

8.2 A complete Deployment

web-deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
  labels:
    app: web
spec:
  replicas: 3
  revisionHistoryLimit: 5          # keep 5 old ReplicaSets for rollback
  selector:
    matchLabels:
      app: web                     # must match template labels
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1                  # at most 1 extra Pod during the update
      maxUnavailable: 0            # never go below 3 ready Pods
  template:                        # the Pod template
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: web
          image: nginx:1.27
          ports:
            - containerPort: 80
          readinessProbe:
            httpGet: { path: /, port: 80 }
            periodSeconds: 5
          resources:
            requests: { cpu: 50m, memory: 64Mi }
            limits:   { memory: 128Mi }
```

terminal

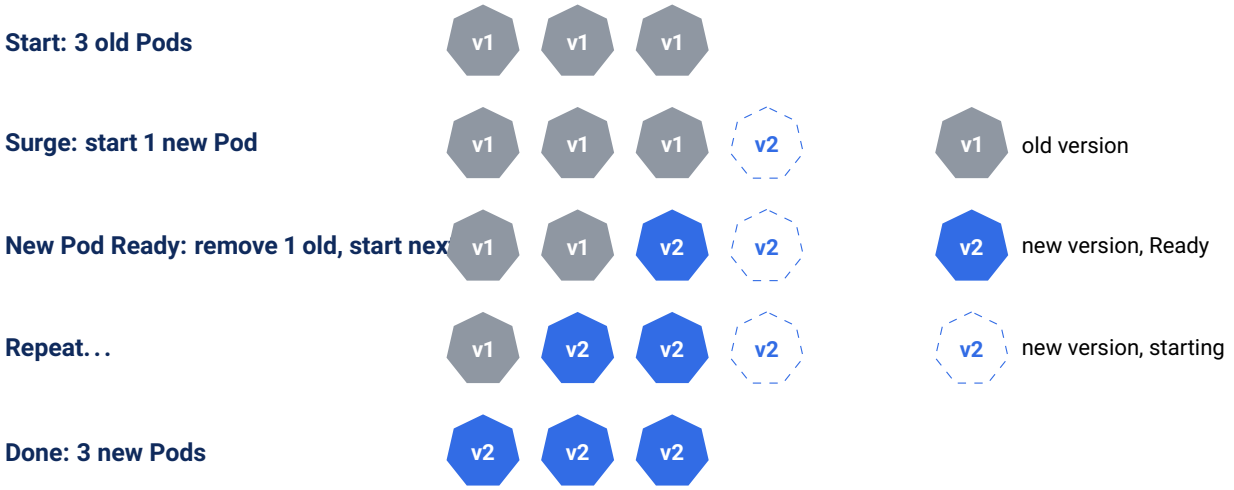
```
kubectl apply -f web-deployment.yaml
kubectl get deploy,rs,pods -l app=web
kubectl scale deployment web --replicas=5
```

Try It Yourself

Delete one of the Pods with `kubectl delete pod <name>` and immediately run `kubectl get pods -w`. Within a second the ReplicaSet creates a replacement with a new random suffix. That is self-healing in action.

8.3 Rolling updates

Change anything in the Pod template — the image, an env var, a label — and the Deployment performs a **rolling update**:



With `maxSurge: 1` and `maxUnavailable: 0`, Kubernetes adds one new Pod, waits until it is *Ready* (readiness probe passes), removes one old Pod, and repeats. Capacity never drops, so users see no downtime.

Setting	Effect
`maxSurge`	How many Pods *above* `replicas` may exist during the update (number or %, default 25%). Higher = faster, more resources.
`maxUnavailable`	How many Pods *below* `replicas` may be unavailable (default 25%). 0 = safest.
`minReadySeconds`	A new Pod must stay Ready this long before it counts. Catches apps that crash shortly after starting.
`progressDeadlineSe`	If no progress for this long (default 600), the rollout is marked failed.

8.4 Managing rollouts

```terminal
# Trigger an update (or edit the YAML and apply it)
kubectl set image deployment/web web=nginx:1.28
kubectl annotate deployment/web kubernetes.io/change-cause="nginx 1.28"

kubectl rollout status deployment/web       # wait and watch progress
kubectl rollout history deployment/web      # list revisions
kubectl rollout history deployment/web --revision=2

kubectl rollout undo deployment/web                # back to previous revision
kubectl rollout undo deployment/web --to-revision=1

kubectl rollout pause deployment/web        # batch several changes...
kubectl rollout resume deployment/web       # ...then roll them out at once
kubectl rollout restart deployment/web      # re-create Pods (e.g. to reload config)
```

Try It Yourself

Set the image to a tag that does not exist: `kubectl set image deployment/web web=nginx:does-not-exist`. Run `kubectl get pods`: the new Pod is stuck in `ImagePullBackOff`, but because `maxUnavailable` is 0 your three old Pods keep serving. Fix it with `kubectl rollout undo deployment/web`.

8.5 Release strategies

Strategy	How	Trade-off
Recreate	`strategy.type: Recreate`: kill all old Pods, then start new.	Downtime; needed when two versions cannot run together.
Rolling	Default. Replace Pods gradually.	Two versions serve at once for a while.
Blue/Green	Run two Deployments (`blue`, `green`); switch the Service selector from one to the other.	Instant switch and rollback; double resources.
Canary	Send a small slice of traffic (say 5%) to the new version, watch metrics, then increase.	Safest for risky changes; needs traffic splitting (Gateway API, service mesh, or Argo Rollouts).

Pro Tip

For automated canaries with metric-based promotion and rollback, look at **Argo Rollouts** or **Flagger**. Both build on the concepts in this chapter and integrate with the Gateway API.

Chapter Recap

- Deployment → ReplicaSet → Pods.
- Changing the Pod template triggers a rolling update via a new ReplicaSet.
- Readiness probes + `maxUnavailable: 0` = zero-downtime releases.
- `kubectl rollout status/history/undo` manage the release.

CHAPTER 9

9 StatefulSets, DaemonSets, Jobs, and CronJobs



In this chapter you will learn

- which workload controller to choose for which job;
- StatefulSets for databases and clustered software;
- DaemonSets for one-Pod-per-node agents;
- Jobs and CronJobs for batch and scheduled work.

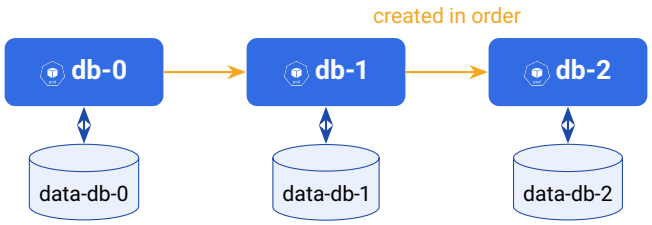
9.1 Choosing a workload type

Controller	Use when…	Examples
Deployment	Pods are interchangeable and stateless.	Web apps, APIs, workers reading a queue.
StatefulSet	Each Pod needs a stable name and its own disk.	PostgreSQL, Kafka, Elasticsearch, ZooKeeper, Redis cluster.
DaemonSet	You need exactly one Pod on every (or some) node.	Log collectors, node monitoring agents, CNI and storage plugins.
Job	Work runs to completion, then stops.	DB migrations, batch processing, report generation.
CronJob	A Job on a schedule.	Nightly backups, cleanup, periodic sync.

9.2 StatefulSets

Deployments treat Pods as cattle: random names, shared storage (if any), any order. Stateful systems need more. A **StatefulSet** guarantees:

- **stable, ordinal names**: `db-0`, `db-1`, `db-2` — a replaced `db-1` is still `db-1`;
- **stable network identity** through a headless Service: `db-0.db.default.svc.cluster.local`;
- **stable storage**: each Pod gets its own PersistentVolumeClaim from `volumeClaimTemplates`, which follows it across rescheduling;
- **ordered** start-up, scaling, and rolling updates (0, 1, 2 . . .).



postgres-statefulset.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: db
spec:
  clusterIP: None              # "headless": DNS returns Pod IPs directly
  selector:
    app: db
  ports:
    - port: 5432
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: db
spec:
  serviceName: db              # the headless Service above
  replicas: 1
  selector:
    matchLabels:
      app: db
  template:
    metadata:
      labels:
        app: db
    spec:
      containers:
        - name: postgres
          image: postgres:16-alpine
          env:
            - name: POSTGRES_PASSWORD
              valueFrom:
                secretKeyRef: { name: db-secret, key: password }
            - name: PGDATA
              value: /var/lib/postgresql/data/pgdata
          ports:
            - containerPort: 5432
          volumeMounts:
            - name: data
              mountPath: /var/lib/postgresql/data
  volumeClaimTemplates:        # one PVC per Pod: data-db-0, data-db-1, ...
    - metadata:
        name: data
      spec:
        accessModes: ["ReadWriteOnce"]
        resources:
          requests:
            storage: 5Gi
```

> **Watch Out**
> A StatefulSet gives you stable building blocks; it does **not** make a database highly available, replicate data, or take backups. For production databases use a managed service or a mature **Operator** (e.g. CloudNativePG, Percona, Strimzi for Kafka) — see Chapter 19.

Deleting a StatefulSet does *not* delete its PVCs by default — your data survives. Control this with `persistentVolumeClaimRetentionPolicy`.

9.3 DaemonSets

A **DaemonSet** runs one copy of a Pod on every node. When a node joins, it gets the Pod; when a node leaves, the Pod is garbage-collected.

node-agent-daemonset.yaml

```yaml
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: node-agent
  namespace: monitoring
spec:
  selector:
    matchLabels:
      app: node-agent
  template:
    metadata:
      labels:
        app: node-agent
    spec:
      tolerations:                        # also run on control-plane nodes
        - key: node-role.kubernetes.io/control-plane
          operator: Exists
          effect: NoSchedule
      containers:
        - name: agent
          image: prom/node-exporter:v1.8.2
          ports:
            - containerPort: 9100
          resources:
            requests: { cpu: 20m, memory: 32Mi }
            limits:   { memory: 64Mi }
```

Use a `nodeSelector` or node affinity in the template to target only some nodes (for example, only GPU nodes).

9.4 Jobs

A **Job** creates Pods that run until they *succeed*. If a Pod fails, the Job retries it (up to `backoffLimit`).

pi-job.yaml

```yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: pi
spec:
  completions: 5             # need 5 successful Pods in total
  parallelism: 2             # run up to 2 at a time
  backoffLimit: 3            # retries before the Job is marked Failed
  activeDeadlineSeconds: 600 # give up after 10 minutes
  ttlSecondsAfterFinished: 3600   # auto-delete 1 hour after finishing
  template:
    spec:
      restartPolicy: Never   # Jobs must use Never or OnFailure
      containers:
        - name: pi
          image: perl:5.40
          command: ["perl", "-Mbignum=bpi", "-wle", "print bpi(500)"]
```

terminal

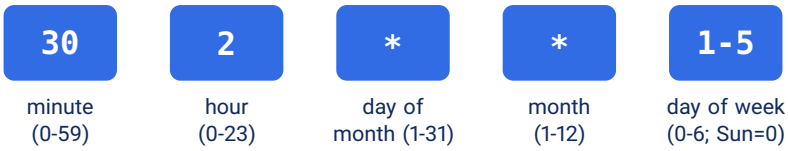
```
kubectl apply -f pi-job.yaml
kubectl get jobs -w
kubectl logs job/pi
```

Pro Tip

`completionMode: Indexed` gives each Pod an index (`JOB_COMPLETION_INDEX` env var), which is perfect for splitting work into shards: Pod 0 processes file 0, Pod 1 file 1, and so on.

9.5 CronJobs

A **CronJob** creates a Job on a schedule written in classic cron syntax.



*"30 2 * * 1-5" = 02:30 every Monday to Friday*

backup-cronjob.yaml

```yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: nightly-backup
spec:
  schedule: "30 2 * * *"           # 02:30 every day
  timeZone: "Asia/Dhaka"           # optional; defaults to the controller's zone (UTC)
  concurrencyPolicy: Forbid         # skip if the previous run is still going
  successfulJobsHistoryLimit: 3
```

```
  failedJobsHistoryLimit: 3
  startingDeadlineSeconds: 300
  jobTemplate:
    spec:
      backoffLimit: 2
      template:
        spec:
          restartPolicy: OnFailure
          containers:
            - name: backup
              image: postgres:16-alpine
              command: ["sh", "-c", "pg_dump -h db -U postgres app > /backup/app-$(date +%F).sql"]
              env:
                - name: PGPASSWORD
                  valueFrom:
                    secretKeyRef: { name: db-secret, key: password }
              volumeMounts:
                - name: backup
                  mountPath: /backup
          volumes:
            - name: backup
              persistentVolumeClaim:
                claimName: backups
```

terminal

```
kubectl get cronjobs
kubectl create job --from=cronjob/nightly-backup manual-run-1   # run it now
```

Chapter Recap

- Deployment: stateless and interchangeable. StatefulSet: identity + own disk.
- DaemonSet: one per node. Job: run to completion. CronJob: Jobs on a schedule.
- Stateful workloads in production usually deserve an Operator.



PART III

Networking

How Pods find each other and how the world finds your app



CHAPTER 10

Services and Cluster DNS



In this chapter you will learn

- the Kubernetes networking model in four rules;
- why Pods need a Service in front of them;
- ClusterIP, NodePort, LoadBalancer, ExternalName, and headless Services;
- `port`, `targetPort`, and `nodePort` — finally explained;
- how DNS names are built and how to debug them.

10.1 The networking model

Kubernetes networking rests on a few simple rules, implemented by your CNI plugin:

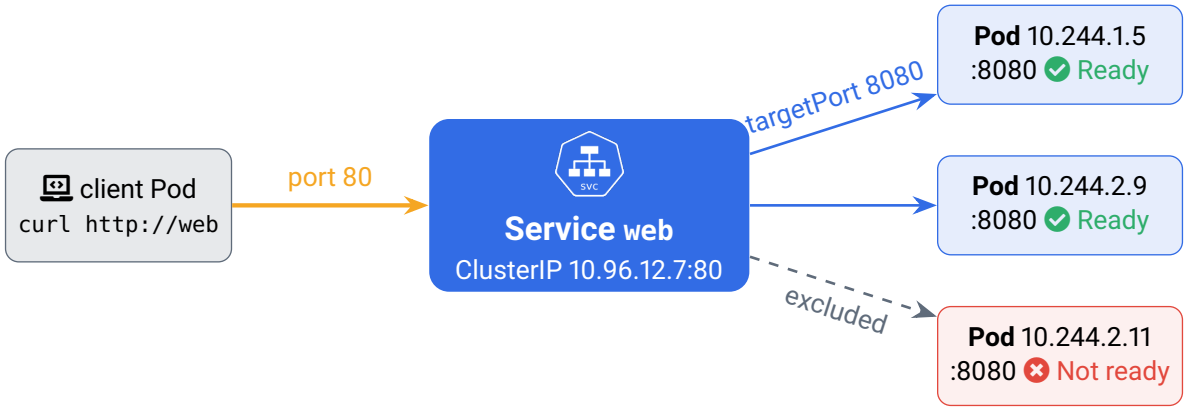
1. Every Pod gets its **own IP address**.
2. Every Pod can reach every other Pod **directly**, across nodes, without NAT.
3. Agents on a node (like the kubelet) can reach all Pods on that node.
4. Containers in the same Pod share the IP and talk via `localhost`.

This flat network is simple, but Pod IPs are *temporary*. Every replacement Pod gets a new IP, and a Deployment may have many Pods. Clients need one stable address that always reaches a healthy Pod. That is a **Service**.

10.2 What a Service is

A Service gives a set of Pods (chosen by a label selector):

- a stable **virtual IP** (the ClusterIP) that never changes;
- a stable **DNS name**, e.g. `web.shop.svc.cluster.local`;
- **load balancing** across all *Ready* Pods behind it.



Behind the scenes, the EndpointSlice controller keeps a list of the IPs of all Ready Pods matching the selector, and kube-proxy on every node programs rules so that packets sent to the ClusterIP are forwarded to one of them.

web-service.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: web
spec:
  type: ClusterIP             # the default
  selector:
    app: web                  # traffic goes to Ready Pods with this label
  ports:
    - name: http
      port: 80                # the port clients use on the Service
      targetPort: 8080        # the port the container listens on (or a port NAME)
      protocol: TCP
```

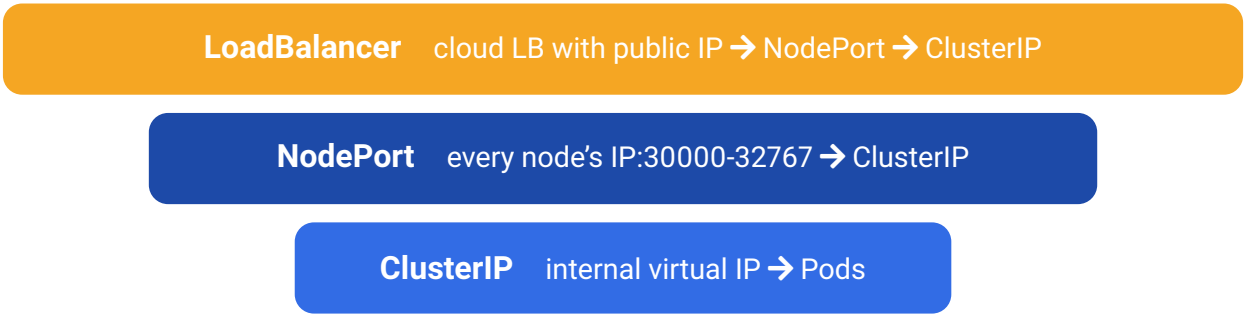
terminal

```
kubectl apply -f web-service.yaml
kubectl get svc web
kubectl get endpointslices -l kubernetes.io/service-name=web
kubectl run tmp --rm -it --image=busybox:1.36 -- wget -qO- http://web
```

Pro Tip

Name your container ports (`name: http`) and use `targetPort: http` in the Service. Then you can change the container port later without touching the Service.

10.3 Service types



each type builds on the one below it

Type	Reachable from	Typical use
`ClusterIP`	Inside the cluster only.	Service-to-service traffic. The default and the most common.
`NodePort`	`<any-node-IP>:<nodePort>` (30000–32767).	Quick demos, bare-metal setups behind your own LB.
`LoadBalancer`	A cloud load balancer with its own IP.	Exposing a single service publicly on a cloud.
`ExternalName`	DNS alias (CNAME) to an outside name.	Pointing `db` at `mydb.rds.amazonaws.com`.
Headless (`clusterIP: None`)	DNS returns each Pod IP.	StatefulSets, client-side load balancing.

nodeport-and-lb.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: web-nodeport
spec:
  type: NodePort
  selector: { app: web }
  ports:
    - port: 80
      targetPort: 8080
      nodePort: 30080         # optional; picked automatically if omitted
---
apiVersion: v1
kind: Service
metadata:
  name: web-public
spec:
  type: LoadBalancer
  externalTrafficPolicy: Local   # keep client source IP; only nodes with Pods receive traffic
  selector: { app: web }
  ports:
    - port: 80
      targetPort: 8080
```

```yaml
---
apiVersion: v1
kind: Service
metadata:
  name: payments-db
spec:
  type: ExternalName
  externalName: payments.abc123.eu-west-1.rds.amazonaws.com
```

> **Watch Out**
>
> One `LoadBalancer` Service = one cloud load balancer = one monthly bill. For many HTTP services, put a single Gateway or Ingress controller behind one load balancer and route by host/path instead (next chapter).

10.4 Cluster DNS

CoreDNS gives every Service a name. The full form is:

`web.shop.svc.cluster.local`

service . namespace . type . cluster domain

- From a Pod in the *same* namespace: just `web`.
- From another namespace: `web.shop` (or the full name).
- Headless Services in a StatefulSet also give each Pod a name: `db-0.db.shop.svc.cluster.local`.
- Named ports get SRV records: `_http._tcp.web.shop.svc.cluster.local`.

terminal

```
kubectl run dns --rm -it --image=busybox:1.36 -- sh
/ # nslookup web
/ # nslookup web.shop.svc.cluster.local
/ # cat /etc/resolv.conf        # search domains that make short names work
```

10.5 Useful Service options

- `sessionAffinity: ClientIP` — send a given client to the same Pod.
- `internalTrafficPolicy: Local` — only route to Pods on the caller's node.
- `trafficDistribution: PreferClose` — prefer topologically close endpoints (same zone).
- Services without a selector + manual EndpointSlices — point a Service at IPs outside the cluster.

10.6 Port-forward vs. Service

`kubectl port-forward svc/web 8080:80` tunnels from your laptop to the cluster through the API server. It is perfect for development and debugging, but it is *not* a way to serve real users.

Chapter Recap

- Pods get IPs, but IPs change; Services provide a stable IP + DNS name.
- Services select *Ready* Pods by label and load-balance across them.
- `port` is what clients call; `targetPort` is where the container listens.
- ClusterIP inside; NodePort/LoadBalancer outside; headless for per-Pod DNS.
- DNS: `service.namespace.svc.cluster.local`.

CHAPTER 11

11 Ingress, Gateway API, and Network Policies

In this chapter you will learn

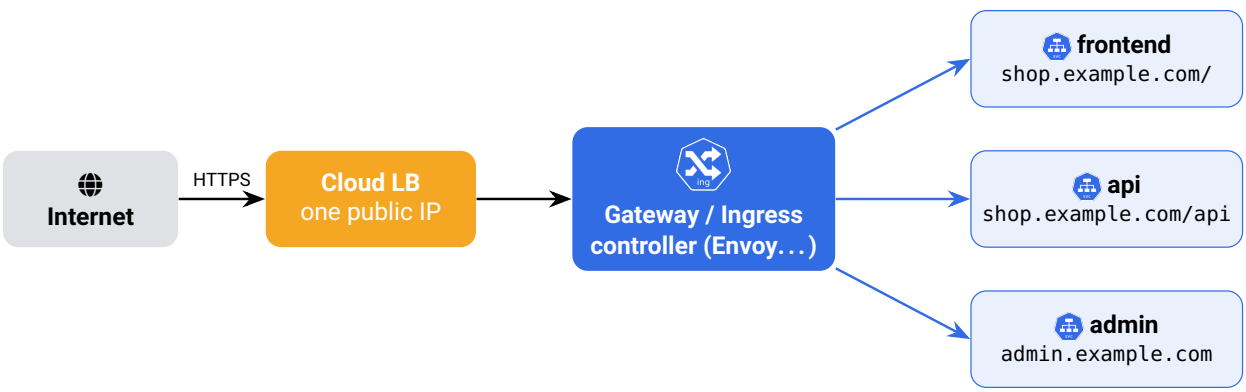
- how HTTP traffic enters a cluster through one entry point;
- the Ingress resource and the newer, more powerful Gateway API;
- TLS certificates with cert-manager;
- NetworkPolicies: a firewall between Pods;
- what a CNI plugin is and how to choose one.

11.1 The problem: many services, one front door

You have a frontend, an API, and an admin panel. You want:

- `shop.example.com` → frontend;
- `shop.example.com/api` → API;
- `admin.example.com` → admin;
- HTTPS everywhere, with one public IP and one load balancer.

This is **Layer-7** (HTTP) routing. Kubernetes models it with two APIs: the original **Ingress** and its successor, the **Gateway API**. Both are only *descriptions*; a **controller** (a proxy such as Envoy, NGINX, HAProxy, or Traefik running in your cluster) reads them and does the actual routing.



11.2 Ingress

shop-ingress.yaml

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: shop
spec:
  ingressClassName: traefik        # which controller should handle this
  tls:
    - hosts: [shop.example.com]
      secretName: shop-tls          # certificate + key stored in a Secret
  rules:
    - host: shop.example.com
      http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
              service: { name: api, port: { number: 80 } }
          - path: /
            pathType: Prefix
            backend:
              service: { name: frontend, port: { number: 80 } }
```

Ingress is stable and widely supported, but it is limited: only host/path routing is standard, and everything else (rewrites, timeouts, canaries, header matching) is done with controller-specific annotations that are not portable.

> **Watch Out**
>
> The popular community **ingress-nginx** controller has been **retired** by the Kubernetes project (best-effort maintenance ended in March 2026). Existing clusters should plan a migration. For new work, prefer the **Gateway API** with a maintained implementation such as Envoy Gateway, Istio, Cilium, NGINX Gateway Fabric, Traefik, or your cloud provider's gateway.

11.3 Gateway API

The **Gateway API** is the modern, role-oriented, expressive replacement for Ingress (GA since v1.0 in 2023). It splits responsibility across three kinds of objects:

Object	Owned by	Describes
`GatewayClass`	Infrastructure provider	Which controller implements gateways (like a StorageClass for traffic).
`Gateway`	Platform / cluster operator	A concrete entry point: listeners, ports, hostnames, TLS.
`HTTPRoute` (+ `GRPCRoute`, `TLSRoute`...)	Application team	How requests are matched and forwarded to Services.

gateway.yaml

```yaml
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: public
  namespace: infra
spec:
  gatewayClassName: eg                # e.g. Envoy Gateway
  listeners:
    - name: https
      protocol: HTTPS
      port: 443
      hostname: "*.example.com"
      tls:
        mode: Terminate
        certificateRefs:
          - name: wildcard-example-com
      allowedRoutes:
        namespaces:
          from: Selector
          selector:
            matchLabels:
              gateway-access: "true"
```

shop-route.yaml

```yaml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: shop
  namespace: shop
spec:
  parentRefs:
    - name: public
      namespace: infra
  hostnames: ["shop.example.com"]
  rules:
    - matches:
        - path: { type: PathPrefix, value: /api }
      backendRefs:
        - name: api-v1
          port: 80
          weight: 90                  # canary: 90% to v1 ...
        - name: api-v2
```

```
        port: 80
        weight: 10                  # ... and 10% to v2
    - matches:
        - headers:
            - name: x-beta-user
              value: "true"
      backendRefs:
        - name: frontend-beta
          port: 80
    - backendRefs:                  # default route
        - name: frontend
          port: 80
```

Weighted backends, header matching, redirects, rewrites, timeouts, and request mirroring are all part of the *standard* API — no annotations needed.

11.4 TLS certificates with cert-manager

cert-manager automates certificates from Let's Encrypt (or your internal CA) and stores them as Secrets that Ingress and Gateways reference. It renews them before expiry.

cluster-issuer.yaml

```yaml
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: letsencrypt
spec:
  acme:
    server: https://acme-v02.api.letsencrypt.org/directory
    email: ops@example.com
    privateKeySecretRef:
      name: letsencrypt-account-key
    solvers:
      - http01:
          gatewayHTTPRoute:
            parentRefs:
              - name: public
                namespace: infra
```

11.5 Network Policies: a firewall for Pods

By default **every Pod can talk to every other Pod**. A **NetworkPolicy** restricts that. Once any policy selects a Pod for a direction (ingress or egress), only traffic explicitly allowed by some policy is permitted in that direction.

> **Key Idea**
>
> NetworkPolicies are **allow-lists** and they are **additive**. Start with "deny all" in a namespace, then add policies that open exactly the paths your application needs.

default-deny.yaml

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny
  namespace: shop
spec:
  podSelector: {}           # every Pod in the namespace
  policyTypes: [Ingress, Egress]
```

allow-api-to-db.yaml

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: db-allow-api
  namespace: shop
spec:
  podSelector:
    matchLabels: { app: db }          # this policy protects the DB Pods
  policyTypes: [Ingress]
  ingress:
    - from:
        - podSelector:
            matchLabels: { app: api }  # only API Pods ...
      ports:
        - port: 5432                    # ... on the Postgres port
          protocol: TCP
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-dns-egress
  namespace: shop
spec:
  podSelector: {}
  policyTypes: [Egress]
  egress:
    - to:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: kube-system
          podSelector:
            matchLabels:
              k8s-app: kube-dns
      ports:
        - { port: 53, protocol: UDP }
        - { port: 53, protocol: TCP }
```

Watch Out

After a default-deny on *egress*, Pods cannot even resolve DNS. Always add the DNS egress rule above. Also note the YAML subtlety: `namespaceSelector` and `podSelector` in the *same* list item mean AND; as *separate* items (each with its own dash) they mean OR.

> **Watch Out**
>
> **NetworkPolicies need a CNI plugin that enforces them.** Calico and Cilium do; Flannel alone does not. If your plugin ignores them, your policies are silently decorative.

11.6 CNI: the network plugin

The **Container Network Interface** (CNI) is the plugin standard that gives Pods their IPs and connects them. Popular choices:

Plugin	Highlights
Cilium	eBPF-based, very fast, NetworkPolicies at L3–L7, can replace kube-proxy, rich observability (Hubble).
Calico	Mature, BGP routing, strong NetworkPolicy support, widely used on-prem.
Flannel	Simple overlay network; no policy enforcement by itself.
Cloud CNIs	AWS VPC CNI, Azure CNI, GKE Dataplane V2 — Pods get VPC-native IPs.

Chapter Recap

- One controller + one load balancer can route all HTTP traffic by host and path.
- Ingress is simple and stable; the Gateway API is the expressive, portable successor.
- cert-manager automates TLS certificates.
- NetworkPolicies are additive allow-lists; start from default-deny and allow DNS.
- Policies only work if your CNI enforces them.



PART IV

Configuration & Storage

Separating config from code and keeping data alive



CHAPTER 12

ConfigMaps and Secrets



In this chapter you will learn

- why configuration must live outside the image;
- creating ConfigMaps and consuming them as env vars or files;
- Secrets: types, consumption, and the truth about base64;
- how configuration changes reach running Pods;
- production secret management: encryption at rest, External Secrets, Sealed Secrets, SOPS.

12.1 Config belongs outside the image

The same image should run in development, staging, and production. Only its configuration should differ: database host, log level, feature flags, API keys. This is principle III of the well-known "Twelve-Factor App". Kubernetes gives you two objects for it:

- **ConfigMap** — non-sensitive settings;
- **Secret** — passwords, tokens, keys, certificates.

12.2 ConfigMaps

app-config.yaml

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  LOG_LEVEL: "info"
  FEATURE_SEARCH: "true"
  DB_HOST: "postgres"
  app.properties: |              # a whole file as one key
    cache.size=256
    cache.ttl=300s
```

Or create one from the command line:

terminal

```
kubectl create configmap app-config \
  --from-literal=LOG_LEVEL=info \
  --from-file=app.properties \
  --dry-run=client -o yaml > app-config.yaml
```

12.2.1 Consuming a ConfigMap

consume-config.yaml

```yaml
spec:
  containers:
    - name: api
      image: ghcr.io/example/api:1.4.0
      # 1) one key -> one env var
      env:
        - name: LOG_LEVEL
          valueFrom:
            configMapKeyRef: { name: app-config, key: LOG_LEVEL }
      # 2) every key -> env vars
      envFrom:
        - configMapRef: { name: app-config }
      # 3) keys -> files in a directory
      volumeMounts:
        - name: config
          mountPath: /etc/app        # /etc/app/app.properties, /etc/app/LOG_LEVEL ...
          readOnly: true
  volumes:
    - name: config
      configMap:
        name: app-config
        items:                       # optional: pick specific keys
          - key: app.properties
            path: app.properties
```

12.2.2 How changes propagate

Consumed as...	After you edit the ConfigMap...
Environment variables	**Nothing changes** until the Pod restarts. Run `kubectl rollout restart deployment/api`.
Mounted volume	Files update automatically within about a minute (not with `subPath` mounts). The app must re-read them.

> **Pro Tip**
>
> A popular pattern is to put a hash of the config into a Pod-template annotation, so that any config change produces a new template and therefore an automatic rolling update. Kustomize's `configMapGenerator` and Helm's `sha256sum` function do exactly this (Chapter 18).

Mark ConfigMaps or Secrets `immutable: true` when they should never change; this protects against accidents and reduces API-server load.

12.3 Secrets

db-secret.yaml

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
type: Opaque
stringData:                    # plain text here; stored base64-encoded
  username: app
  password: "s3cr3t-Pa55w0rd"
```

terminal

```
kubectl create secret generic db-secret \
  --from-literal=username=app --from-literal=password='s3cr3t-Pa55w0rd'

kubectl get secret db-secret -o jsonpath='{.data.password}' | base64 -d
```

Watch Out

base64 is encoding, not encryption. Anyone who can read the Secret object can read the value. Never commit plain Secret manifests to Git. Protect Secrets with RBAC, enable encryption at rest, and use one of the tools in Section 12.4.

12.3.1 Secret types

Type	Use
`Opaque`	Arbitrary key/value data (default).
`kubernetes.io/tls`	A certificate and key (`tls.crt`, `tls.key`) for Ingress/Gateway.
`kubernetes.io/dockerconfigjson`	Credentials for pulling from private registries.
`kubernetes.io/basic-auth`, `ssh-auth`	Structured credentials.

terminal

```
# A registry pull secret, then reference it from a Pod or ServiceAccount
kubectl create secret docker-registry ghcr-pull \
  --docker-server=ghcr.io --docker-username=USER --docker-password=TOKEN

kubectl create secret tls shop-tls --cert=tls.crt --key=tls.key
```

use-secret.yaml

```yaml
spec:
  imagePullSecrets:
    - name: ghcr-pull
```

```
  containers:
    - name: api
      image: ghcr.io/example/private-api:1.4.0
      env:
        - name: DB_PASSWORD
          valueFrom:
            secretKeyRef: { name: db-secret, key: password }
      volumeMounts:
        - name: creds
          mountPath: /var/run/secrets/app
          readOnly: true
  volumes:
    - name: creds
      secret:
        secretName: db-secret
        defaultMode: 0400
```

Pro Tip

Prefer mounting Secrets as *files* over env vars. Environment variables leak easily (crash dumps, `/proc`, child processes, debug endpoints that print the environment).

12.4 Production secret management

Approach	How it works
Encryption at rest	Configure the API server (or your cloud's KMS integration) so Secrets are encrypted inside etcd. Managed clusters usually offer a checkbox.
External Secrets Operator	Secrets live in AWS Secrets Manager, GCP Secret Manager, Azure Key Vault, HashiCorp Vault… An `ExternalSecret` object tells the operator to sync them into Kubernetes Secrets.
Sealed Secrets	Encrypt a Secret with the cluster's public key into a `SealedSecret` that is safe to commit. Only the in-cluster controller can decrypt it.
SOPS (+ age/KMS)	Encrypt values in YAML files in Git; decrypt in your CD pipeline or with a Kustomize/Helm plugin.
Secrets Store CSI Driver	Mount secrets from an external vault directly as files, without creating Kubernetes Secrets.

external-secret.yaml

```
apiVersion: external-secrets.io/v1
kind: ExternalSecret
metadata:
  name: db-secret
spec:
  refreshInterval: 1h
  secretStoreRef:
    name: aws-secrets-manager
    kind: ClusterSecretStore
```

```
  target:
    name: db-secret              # the Kubernetes Secret to create
  data:
    - secretKey: password
      remoteRef:
        key: prod/taskflow/db
        property: password
```

Chapter Recap

- ConfigMaps for settings, Secrets for credentials — never bake either into images.
- Consume as env vars (need restart) or files (update live).
- Secrets are only base64-encoded; protect them with RBAC, encryption at rest, and an external secret manager.



CHAPTER 13

Volumes and Persistent Storage



In this chapter you will learn

- why container filesystems are temporary;
- ephemeral volumes: `emptyDir`, `configMap`, `secret`, `projected`;
- PersistentVolumes, PersistentVolumeClaims, and StorageClasses;
- access modes, reclaim policies, volume expansion, and snapshots;
- the CSI driver model.

13.1 The problem

A container's filesystem lives and dies with the container. Restart the container and every file it wrote is gone. Databases, uploads, and caches need storage with a different lifetime. Kubernetes offers **volumes**: directories mounted into containers whose lifetime is managed separately.

13.2 Ephemeral volumes

These live as long as the *Pod* (surviving container restarts, but not Pod deletion).

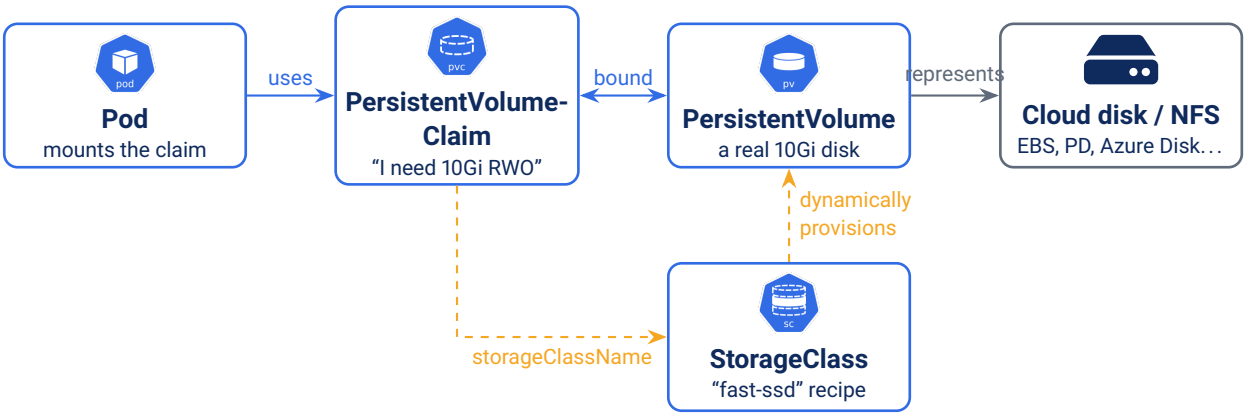
Volume	Use
`emptyDir`	Scratch space shared between containers in a Pod. Set `medium: Memory` for a RAM disk.
`configMap`, `secret`	Expose config or credentials as files.
`projected`	Combine several sources (configMap, secret, downwardAPI, serviceAccountToken) into one directory.
`downwardAPI`	Expose Pod labels, annotations, and resources as files.
Generic ephemeral	A per-Pod PVC that is deleted with the Pod.

Watch Out

`hostPath` mounts a directory from the node. It ties the Pod to one node and can give a

container access to the host — a serious security risk. Avoid it except for system agents that truly need it.

13.3 Persistent storage: three objects



- A **PersistentVolume** (PV) is a piece of storage in the cluster — cluster-scoped, like a node is a piece of compute.
- A **PersistentVolumeClaim** (PVC) is a namespaced *request* for storage: size, access mode, class.
- A **StorageClass** describes a *kind* of storage and which driver creates it, enabling **dynamic provisioning**: when a PVC appears, a matching PV (and real disk) is created automatically.

Real-World Analogy

A StorageClass is a menu item ("large SSD"), a PVC is your order, and the PV is the plate that arrives. You never walk into the kitchen yourself.

storageclass.yaml

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-ssd
  annotations:
    storageclass.kubernetes.io/is-default-class: "false"
provisioner: ebs.csi.aws.com        # the CSI driver (cloud specific)
parameters:
  type: gp3
reclaimPolicy: Delete               # or Retain
allowVolumeExpansion: true
volumeBindingMode: WaitForFirstConsumer   # create the disk in the zone where the Pod lands
```

pvc-and-pod.yaml

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: uploads
spec:
  accessModes: ["ReadWriteOnce"]
  storageClassName: fast-ssd        # omit to use the cluster default class
  resources:
    requests:
      storage: 10Gi
---
apiVersion: v1
kind: Pod
metadata:
  name: uploader
spec:
  containers:
    - name: app
      image: nginx:1.27
      volumeMounts:
        - name: data
          mountPath: /usr/share/nginx/html
  volumes:
    - name: data
      persistentVolumeClaim:
        claimName: uploads
```

terminal

```
kubectl get storageclass           # kind ships "standard" (local-path) as default
kubectl apply -f pvc-and-pod.yaml
kubectl get pvc,pv
kubectl describe pvc uploads       # Pending? read the events
```

13.4 Access modes

Mode	Short	Meaning
`ReadWriteOnce`	RWO	Mounted read-write by one *node*. Block disks (EBS, PD). Most common.
`ReadWriteOncePod`	RWOP	Read-write by exactly one *Pod*. Strongest guarantee.
`ReadOnlyMany`	ROX	Read-only by many nodes.
`ReadWriteMany`	RWX	Read-write by many nodes. Needs a shared filesystem (NFS, EFS, CephFS, Azure Files).

13.5 Lifecycle and reclaim policy

When a PVC is deleted, the bound PV's `reclaimPolicy` decides the fate of the data:

- `Delete` — the PV and underlying disk are deleted (default for dynamic provisioning);
- `Retain` — the PV and disk are kept for manual recovery. Use it for anything precious.

> **Watch Out**
> With `reclaimPolicy: Delete`, `kubectl delete pvc` or deleting a namespace **destroys the data**. For production databases use a `Retain` class and take backups or snapshots.

13.6 Expansion and snapshots

If the StorageClass has `allowVolumeExpansion: true`, grow a volume by editing the PVC's `spec.resources.requ`
Shrinking is not supported.

With a CSI driver that supports it, take point-in-time snapshots:

snapshot.yaml

```yaml
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshot
metadata:
  name: uploads-2026-10-10
spec:
  volumeSnapshotClassName: csi-snapclass
  source:
    persistentVolumeClaimName: uploads
---
# Restore: a new PVC whose dataSource is the snapshot
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: uploads-restored
spec:
  accessModes: ["ReadWriteOnce"]
  resources:
    requests:
      storage: 10Gi
  dataSource:
    name: uploads-2026-10-10
    kind: VolumeSnapshot
    apiGroup: snapshot.storage.k8s.io
```

13.7 CSI: the storage plugin model

The **Container Storage Interface** lets storage vendors ship drivers independently of Kubernetes releases. Every modern backend — AWS EBS/EFS, GCP PD/Filestore, Azure Disk/Files, Ceph (Rook), Longhorn, Portworx, NetApp — has a CSI driver. You install the driver, create a StorageClass that names it, and PVCs do the rest.

Chapter Recap

- Container filesystems are temporary; volumes give data a separate lifetime.
- PVC = request, PV = actual storage, StorageClass = recipe for dynamic provisioning.
- Choose access modes and reclaim policies deliberately; `Delete` really deletes.
- Use snapshots and backups for anything you cannot lose.



PART V

Operating Kubernetes

Resources, scheduling, scaling, security, and observability



CHAPTER 14

Resources and Scheduling



In this chapter you will learn

- CPU and memory *requests* and *limits*, and their units;
- Quality-of-Service classes and what happens under pressure;
- LimitRanges and ResourceQuotas for namespaces;
- controlling placement: nodeSelector, affinity, taints and tolerations, topology spread, priority and preemption;
- PodDisruptionBudgets and node draining.

14.1 Requests and limits

Each container can declare two numbers per resource:

- **request** — what the container is *guaranteed*. The scheduler only places a Pod on a node with enough unrequested capacity.
- **limit** — the *maximum* it may use.

resources.yaml

```yaml
resources:
  requests:
    cpu: 250m          # 250 millicores = 1/4 of a CPU core
    memory: 256Mi      # mebibytes (Mi, Gi) -- not MB
  limits:
    cpu: "1"           # optional, see the Pro Tip below
    memory: 512Mi
```

	CPU (compressible)	Memory (incompressible)
Units	`1` = one core; `500m` = half a core.	`Mi`, `Gi` (powers of 2); `M`, `G` (powers of 10).
Over limit	The container is **throttled** (slowed down).	The container is **OOM-killed** and restarted.

Pro Tip

Always set **memory requests and limits** (often equal). Always set **CPU requests**. Many teams

> *omit CPU limits* for latency-sensitive services, because CPU throttling causes hard-to-diagnose slowness while idle cores sit unused. Measure first with `kubectl top` and your monitoring.

14.2 Quality of Service classes

From requests and limits, Kubernetes assigns each Pod a QoS class. When a node runs out of memory, the kubelet evicts Pods in this order: BestEffort first, then Burstable, and Guaranteed last.

Class	Rule	Eviction priority
Guaranteed	Every container has CPU and memory requests *equal to* limits.	Last to go.
Burstable	At least one request or limit set, but not Guaranteed.	Middle.
BestEffort	No requests or limits at all.	First to go.

14.3 Namespace guard rails

14.3.1 LimitRange: defaults per container

limitrange.yaml

```yaml
apiVersion: v1
kind: LimitRange
metadata:
  name: defaults
  namespace: shop
spec:
  limits:
    - type: Container
      defaultRequest: { cpu: 100m, memory: 128Mi }   # applied if omitted
      default:        { memory: 256Mi }              # default limit
      max:            { cpu: "2", memory: 2Gi }
```

14.3.2 ResourceQuota: a budget per namespace

quota.yaml

```yaml
apiVersion: v1
kind: ResourceQuota
metadata:
  name: shop-budget
  namespace: shop
spec:
  hard:
    requests.cpu: "8"
    requests.memory: 16Gi
    limits.memory: 32Gi
    pods: "60"
```

```
    persistentvolumeclaims: "10"
    services.loadbalancers: "1"
```

terminal

```
kubectl describe quota shop-budget -n shop     # used vs hard
```

14.4 Controlling where Pods run

14.4.1 nodeSelector: the simple way

terminal

```
kubectl label node lab-worker disktype=ssd
```

nodeselector.yaml

```
spec:
  nodeSelector:
    disktype: ssd
```

14.4.2 Node affinity: the expressive way

node-affinity.yaml

```
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:     # hard rule
        nodeSelectorTerms:
          - matchExpressions:
              - key: kubernetes.io/arch
                operator: In
                values: [amd64, arm64]
      preferredDuringSchedulingIgnoredDuringExecution:    # soft preference
        - weight: 80
          preference:
            matchExpressions:
              - key: topology.kubernetes.io/zone
                operator: In
                values: [eu-west-1a]
```

14.4.3 Pod affinity and anti-affinity

Place Pods *relative to other Pods*: keep a cache near its app, or keep replicas of the same app apart.

pod-anti-affinity.yaml

```yaml
spec:
  affinity:
    podAntiAffinity:
      preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 100
          podAffinityTerm:
            topologyKey: kubernetes.io/hostname    # "not on the same node"
            labelSelector:
              matchLabels: { app: api }
```

14.4.4 Topology spread constraints

The modern, more precise way to spread replicas across zones and nodes:

topology-spread.yaml

```yaml
spec:
  topologySpreadConstraints:
    - maxSkew: 1
      topologyKey: topology.kubernetes.io/zone
      whenUnsatisfiable: ScheduleAnyway
      labelSelector:
        matchLabels: { app: api }
    - maxSkew: 1
      topologyKey: kubernetes.io/hostname
      whenUnsatisfiable: DoNotSchedule
      labelSelector:
        matchLabels: { app: api }
```

14.4.5 Taints and tolerations

Affinity *attracts* Pods to nodes. **Taints** *repel* them: a tainted node accepts only Pods that **tolerate** the taint.

> **Real-World Analogy**
>
> A taint is a “Staff only” sign on a door. Most people (Pods) stay out. A toleration is the staff badge. Note that the badge *allows* entry but does not *force* it — combine with affinity if you want the Pod to go there.

terminal

```
kubectl taint nodes gpu-node-1 gpu=true:NoSchedule
kubectl taint nodes gpu-node-1 gpu=true:NoSchedule-     # remove (trailing dash)
```

toleration.yaml

```yaml
spec:
  tolerations:
    - key: gpu
      operator: Equal
      value: "true"
```

```
      effect: NoSchedule
  nodeSelector:
    gpu: "true"
```

Effects: `NoSchedule` (do not place new Pods), `PreferNoSchedule` (try not to), `NoExecute` (also evict running Pods that do not tolerate it).

14.4.6 Priority and preemption

priorityclass.yaml

```yaml
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: business-critical
value: 100000
globalDefault: false
description: "Customer-facing services; may preempt batch work."
---
# in a Pod spec:
# priorityClassName: business-critical
```

If a high-priority Pod cannot be scheduled, the scheduler may *evict* lower-priority Pods to make room.

14.5 Voluntary disruptions

Node upgrades and `kubectl drain` evict Pods on purpose. A **PodDisruptionBudget** (PDB) tells Kubernetes how many Pods of an app must stay up during such voluntary disruptions.

pdb.yaml

```yaml
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: api-pdb
spec:
  minAvailable: 2          # or maxUnavailable: 1
  selector:
    matchLabels: { app: api }
```

terminal

```
kubectl cordon lab-worker                          # no new Pods here
kubectl drain lab-worker --ignore-daemonsets --delete-emptydir-data
kubectl uncordon lab-worker
```

Chapter Recap

- Requests drive scheduling; limits cap usage. Memory over limit = OOMKill.
- QoS classes decide who gets evicted first.

- LimitRanges set defaults; ResourceQuotas cap a namespace.
- nodeSelector/affinity attract, taints repel, topology spread distributes.
- PDBs keep apps available during drains and upgrades.

CHAPTER 15

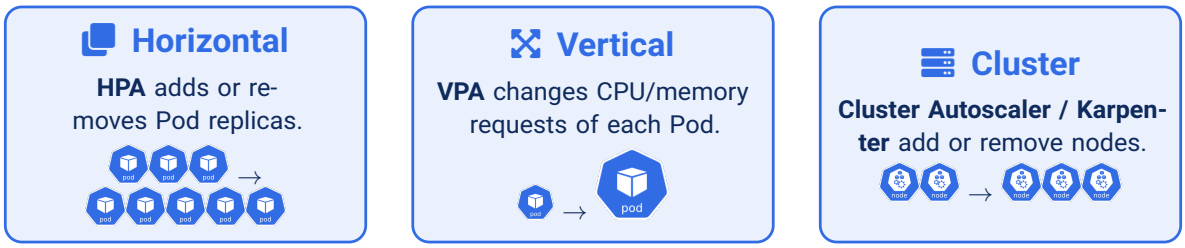
15 Autoscaling



In this chapter you will learn

- the three dimensions of scaling: more Pods, bigger Pods, more nodes;
- the Horizontal Pod Autoscaler and its algorithm;
- the Vertical Pod Autoscaler and in-place resize;
- Cluster Autoscaler and Karpenter;
- event-driven scaling with KEDA.

15.1 Three ways to scale



They work together: the HPA asks for more Pods; if there is no room, Pods go `Pending`; the Cluster Autoscaler sees Pending Pods and adds a node.

15.2 Prerequisite: metrics-server

The HPA needs live CPU/memory figures from **metrics-server**.

terminal

```
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/
    ↪ components.yaml

# kind/local clusters use self-signed kubelet certs; allow them (lab only!)
kubectl patch deployment metrics-server -n kube-system --type=json \
  -p='[{"op":"add","path":"/spec/template/spec/containers/0/args/-","value":"--kubelet-insecure-
    ↪ tls"}]'

kubectl top nodes
kubectl top pods -A
```

15.3 Horizontal Pod Autoscaler

api-hpa.yaml

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: api
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: api
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70        # % of the CPU *request*
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 0
      policies:
        - type: Percent
          value: 100                    # at most double every 30s
          periodSeconds: 30
    scaleDown:
      stabilizationWindowSeconds: 300   # wait 5 min before shrinking
```

15.3.1 The algorithm

Every 15 seconds the HPA computes:

$$\text{desiredReplicas} = \left\lceil \text{currentReplicas} \times \frac{\text{currentMetricValue}}{\text{targetMetricValue}} \right\rceil$$

With 4 replicas at 140 % average CPU and a 70 % target: $\lceil 4 \times 140/70 \rceil = 8$ replicas.

Watch Out

Utilisation is measured against the **request**. A Deployment without CPU requests cannot be autoscaled on CPU. Also, do not set `replicas` in your Deployment YAML when an HPA manages it — every `apply` would reset the count.

Try It Yourself

terminal

```
kubectl create deployment php --image=registry.k8s.io/hpa-example --port=80
kubectl set resources deployment php --requests=cpu=200m
kubectl expose deployment php --port=80
kubectl autoscale deployment php --cpu-percent=50 --min=1 --max=10

# generate load in a second terminal
kubectl run load --rm -it --image=busybox:1.36 -- \
  sh -c "while true; do wget -q -O- http://php; done"

kubectl get hpa php -w      # watch replicas climb, then fall after you stop the load
```

15.3.2 Custom and external metrics

Through the `custom.metrics.k8s.io` and `external.metrics.k8s.io` APIs (provided by adapters like the Prometheus Adapter), the HPA can scale on requests per second, queue length, or any business metric:

hpa-custom-metric.yaml

```yaml
metrics:
  - type: Pods
    pods:
      metric:
        name: http_requests_per_second
      target:
        type: AverageValue
        averageValue: "50"
```

15.4 Vertical Pod Autoscaler

The **VPA** (an add-on) watches real usage and recommends — or applies — better requests. Start with `updateMode: "Off"` to only collect recommendations.

api-vpa.yaml

```yaml
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: api
spec:
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: api
  updatePolicy:
    updateMode: "Off"          # "Off" = recommend only; "Recreate"/"Auto" apply changes
```

terminal

```
kubectl describe vpa api       # see "Target", "Lower Bound", "Upper Bound"
```

Do not let the HPA and VPA both act on CPU/memory for the same workload: they will fight. Recent Kubernetes versions also support *in-place Pod resize*, changing requests without restarting the Pod.

15.5 Scaling the cluster itself

- **Cluster Autoscaler** adds nodes to a node group when Pods are Pending for lack of resources and removes under-used nodes.
- **Karpenter** (AWS, Azure, and others) provisions right-sized nodes on demand directly, often faster and cheaper.

15.6 Event-driven autoscaling with KEDA

KEDA scales workloads on external events — Kafka lag, RabbitMQ queue depth, SQS messages, cron schedules, Prometheus queries — and can scale all the way *to zero* when there is no work.

keda-scaledobject.yaml

```yaml
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: worker
spec:
  scaleTargetRef:
    name: worker               # a Deployment
  minReplicaCount: 0
  maxReplicaCount: 30
  triggers:
    - type: rabbitmq
      metadata:
        queueName: emails
        mode: QueueLength
        value: "20"            # one replica per 20 waiting messages
      authenticationRef:
        name: rabbitmq-auth
```

Chapter Recap

- HPA = more Pods; VPA = bigger Pods; Cluster Autoscaler/Karpenter = more nodes.
- HPA needs metrics-server and CPU/memory *requests*.
- Tune `behavior` to scale up fast and down slowly.
- KEDA scales on events and down to zero.



CHAPTER 16

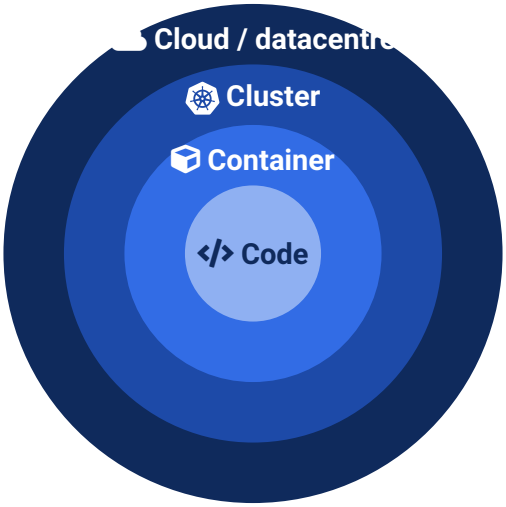
Security



In this chapter you will learn

- the layers of Kubernetes security (the “4 Cs”);
- authentication, authorisation with RBAC, and ServiceAccounts;
- hardening Pods with `securityContext` and Pod Security Standards;
- admission policies with ValidatingAdmissionPolicy, Kyverno, and Gatekeeper;
- supply-chain security: scanning, signing, and minimal images;
- a practical hardening checklist.

16.1 Defence in depth: the 4 Cs



Each layer protects the one inside it. A perfectly configured cluster cannot save vulnerable code, and secure code cannot survive an exposed cloud account. This chapter focuses on the *cluster* and *container* layers.

16.2 Who are you? Authentication

Kubernetes has no “user” objects. Humans are authenticated externally:

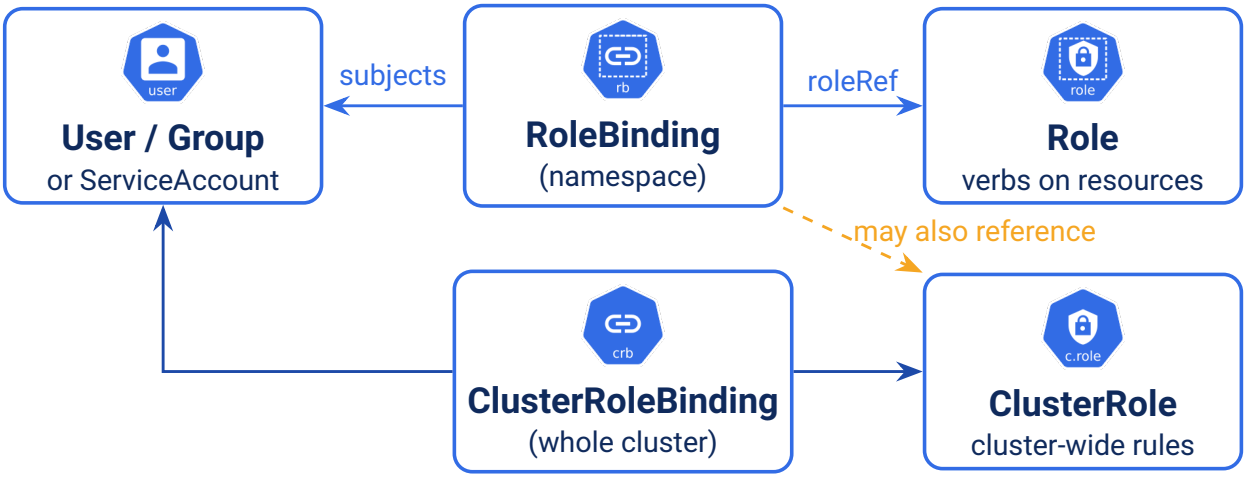
- **client certificates** (common in kubeadm/kind admin configs);

- **OIDC** tokens from an identity provider (Google, Entra ID, Okta, Keycloak);
- **cloud IAM** integrations (EKS access entries, GKE IAM, AKS Entra ID).

Workloads authenticate as **ServiceAccounts** — real namespaced objects. Every Pod runs as one (`default` if you do not choose), and receives a short-lived, automatically rotated token.

16.3 What may you do? RBAC

Role-Based Access Control answers "may *this subject* perform *this verb* on *this resource*?" with four objects:



rbac-ci-deployer.yaml

```yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: ci-deployer
  namespace: shop
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: deployer
  namespace: shop
rules:
  - apiGroups: ["apps"]
    resources: ["deployments"]
    verbs: ["get", "list", "watch", "update", "patch"]
  - apiGroups: [""]                    # "" = the core API group
    resources: ["pods", "pods/log"]
    verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: ci-deployer-binding
  namespace: shop
```

```yaml
subjects:
  - kind: ServiceAccount
    name: ci-deployer
    namespace: shop
roleRef:
  kind: Role
  name: deployer
  apiGroup: rbac.authorization.k8s.io
```

terminal

```
kubectl auth can-i update deployments -n shop --as=system:serviceaccount:shop:ci-deployer   # yes
kubectl auth can-i delete secrets -n shop --as=system:serviceaccount:shop:ci-deployer       # no
kubectl auth can-i --list --as=system:serviceaccount:shop:ci-deployer -n shop
```

Watch Out

Be stingy. Avoid `cluster-admin` bindings, wildcard `"*"` verbs or resources, and granting `get/list` on Secrets (listing Secrets reveals their contents). Remember that the ability to *create Pods* in a namespace effectively grants access to every Secret and ServiceAccount in it.

Built-in ClusterRoles cover most needs: `view`, `edit`, `admin`, and `cluster-admin`. Bind `view` or `edit` per namespace for developers.

Pro Tip

Set `automountServiceAccountToken: false` on Pods that never call the Kubernetes API. Fewer tokens lying around means less to steal.

16.4 Hardening Pods: securityContext

hardened-pod.yaml

```yaml
spec:
  automountServiceAccountToken: false
  securityContext:                        # Pod level
    runAsNonRoot: true
    runAsUser: 10001
    runAsGroup: 10001
    fsGroup: 10001
    seccompProfile:
      type: RuntimeDefault                # block dangerous syscalls
  containers:
    - name: api
      image: ghcr.io/example/api@sha256:3b1f...   # pin by digest
      securityContext:                    # container level
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true
        privileged: false
        capabilities:
          drop: ["ALL"]
```

```
      volumeMounts:
        - name: tmp
          mountPath: /tmp                # writable scratch, since root fs is read-only
  volumes:
    - name: tmp
      emptyDir: {}
```

16.5 Pod Security Standards

Kubernetes defines three standard profiles, enforced per namespace by the built-in **Pod Security Admission** controller (it replaced the removed PodSecurityPolicy):

Profile	Effect
`privileged`	No restrictions. Only for trusted system components.
`baseline`	Blocks known privilege escalations: privileged containers, host namespaces, hostPath, etc.
`restricted`	Baseline plus best practices: non-root, drop all capabilities, seccomp, no privilege escalation.

restricted-namespace.yaml

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: shop
  labels:
    pod-security.kubernetes.io/enforce: restricted   # reject violating Pods
    pod-security.kubernetes.io/warn: restricted      # warn on kubectl apply
    pod-security.kubernetes.io/audit: restricted     # record in audit log
```

16.6 Policy as code

For rules beyond Pod security — “images must come from our registry”, “every Deployment needs an `owner` label”, “no `latest` tags” — use admission policies:

- **ValidatingAdmissionPolicy** — built in (GA since 1.30), rules written in CEL, no extra software.
- **Kyverno** — policies written as Kubernetes YAML; can validate, mutate, generate, and verify image signatures.
- **OPA Gatekeeper** — policies in the Rego language.

no-latest-tag.yaml

```yaml
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingAdmissionPolicy
metadata:
  name: no-latest-tag
```

```yaml
spec:
  failurePolicy: Fail
  matchConstraints:
    resourceRules:
      - apiGroups: ["apps"]
        apiVersions: ["v1"]
        operations: ["CREATE", "UPDATE"]
        resources: ["deployments"]
  validations:
    - expression: >-
        object.spec.template.spec.containers.all(c,
          !c.image.endsWith(':latest') && c.image.contains(':'))
      message: "Images must use an explicit, non-latest tag."
---
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingAdmissionPolicyBinding
metadata:
  name: no-latest-tag
spec:
  policyName: no-latest-tag
  validationActions: ["Deny"]
```

16.7 Supply-chain security

- **Scan images** for known vulnerabilities in CI (Trivy, Grype) and continuously in the registry.
- **Use minimal base images** (distroless, Chainguard, `-slim`).
- **Sign images** with Sigstore **cosign** and verify signatures at admission (Kyverno, Sigstore policy-controller).
- **Generate SBOMs** (software bills of materials) with Syft or BuildKit.
- **Pin by digest** so the image you tested is the image you run.

```terminal
trivy image --severity HIGH,CRITICAL ghcr.io/example/api:1.4.0
cosign sign ghcr.io/example/api@sha256:3b1f...
cosign verify ghcr.io/example/api@sha256:3b1f... \
  --certificate-identity-regexp 'https://github.com/example/.*' \
  --certificate-oidc-issuer https://token.actions.githubusercontent.com
```

16.8 Runtime security and auditing

- **Falco** or **Tetragon** detect suspicious behaviour at runtime (a shell spawned in a container, unexpected file writes).
- **Audit logging** on the API server records who did what and when.
- **kube-bench** checks a cluster against the CIS Kubernetes Benchmark.

16.9 Hardening checklist

- [x] API server not public, or behind IP allow-lists
- [x] SSO/OIDC for humans; no shared admin kubeconfigs
- [x] Least-privilege RBAC, reviewed regularly
- [x] `restricted` Pod Security on app namespaces
- [x] Default-deny NetworkPolicies
- [x] Secrets encrypted at rest; external secret manager
- [x] Images scanned, signed, pinned by digest
- [x] Non-root, read-only root FS, all capabilities dropped
- [x] Resource requests/limits everywhere
- [x] Audit logs shipped off-cluster
- [x] Cluster and nodes kept on supported versions
- [x] etcd backups tested by restoring

Chapter Recap

- Authentication is external; ServiceAccounts identify workloads.
- RBAC = Roles (what) + Bindings (who); grant the minimum.
- Harden Pods with `securityContext`; enforce with Pod Security Admission.
- Use admission policies, image scanning, and signing to secure the supply chain.

CHAPTER 17

17 Observability: Logs, Metrics, Traces

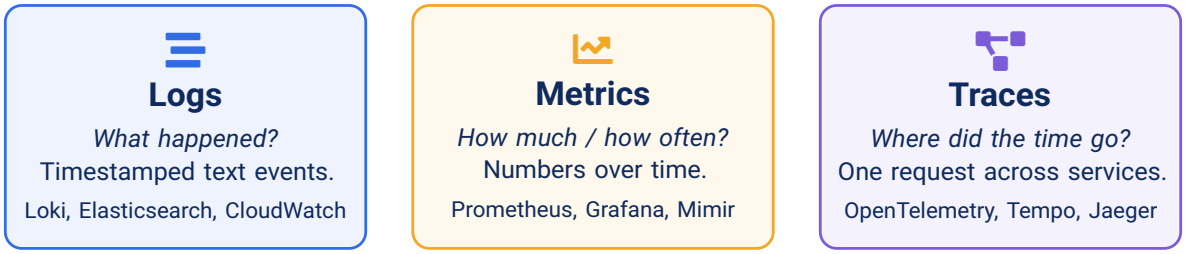


In this chapter you will learn

- the three pillars of observability and how they fit together;
- how container logging works and how to centralise logs;
- metrics with Prometheus and dashboards with Grafana;
- alerts that wake you up only when it matters;
- distributed tracing with OpenTelemetry.

17.1 Why observability?

In a cluster, Pods move, scale, and die constantly. You cannot SSH into "the server" to look around. You need the system to *tell you* what is going on. That ability is **observability**, built from three signals:



17.2 Logs

Containers write to `stdout` and `stderr`. The container runtime saves that output to files on the node, and the kubelet serves them to `kubectl logs`.

terminal

```
kubectl logs deploy/api                    # one Pod of the Deployment
kubectl logs -l app=api --all-containers --prefix --since=15m
kubectl logs api-7d9f8c-xk2lp --previous    # the run before the last crash
kubectl logs -f api-7d9f8c-xk2lp
stern api -n shop                          # tail all matching Pods, colourised
```

> **Watch Out**
> Node log files are rotated and disappear with the node. For anything beyond live debugging, ship logs to a central store: a DaemonSet agent (**Fluent Bit**, **Grafana Alloy**, **Vector**) on each node reads the log files and forwards them to **Loki**, **Elasticsearch/OpenSearch**, or your cloud's logging service.

> **Pro Tip**
> Log in **structured JSON** with fields like `level`, `msg`, `trace_id`, and `user_id`. Structured logs can be filtered and correlated; free text can only be grepped.

17.3 Metrics with Prometheus

Prometheus is the CNCF standard for metrics. It *pulls* ("scrapes") an HTTP endpoint, usually `/metrics`, on every target every 15–30 seconds and stores the numbers as time series.

GET /metrics

```
# HELP http_requests_total Total HTTP requests.
# TYPE http_requests_total counter
http_requests_total{method="GET",handler="/api/tasks",status="2xx"} 10432
http_requests_total{method="POST",handler="/api/tasks",status="5xx"} 3
# HELP http_request_duration_seconds Request latency.
# TYPE http_request_duration_seconds histogram
http_request_duration_seconds_bucket{handler="/api/tasks",le="0.1"} 9870
```

17.3.1 The kube-prometheus-stack

The easiest way to get a complete monitoring setup is the **kube-prometheus-stack** Helm chart. It installs:

- the **Prometheus Operator** and a Prometheus server;
- **Alertmanager** for routing alerts (Slack, PagerDuty, e-mail);
- **Grafana** with ready-made dashboards for nodes, Pods, and workloads;
- **node-exporter** (node metrics) and **kube-state-metrics** (object state: replicas, Pod phases, restarts).

terminal

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
helm install monitoring prometheus-community/kube-prometheus-stack \
  --namespace monitoring --create-namespace

kubectl -n monitoring port-forward svc/monitoring-grafana 3000:80
# open http://localhost:3000  (user: admin; password below)
kubectl -n monitoring get secret monitoring-grafana \
  -o jsonpath='{.data.admin-password}' | base64 -d
```

17.3.2 Telling Prometheus what to scrape

With the operator, you describe scrape targets with a `ServiceMonitor`:

api-servicemonitor.yaml

```yaml
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: api
  namespace: shop
  labels:
    release: monitoring        # the chart's default selector picks this up
spec:
  selector:
    matchLabels:
      app: api                 # the Service to scrape
  endpoints:
    - port: http               # named Service port
      path: /metrics
      interval: 15s
```

17.3.3 PromQL in five queries

PromQL

```
# Requests per second, per handler, over the last 5 minutes
sum by (handler) (rate(http_requests_total[5m]))

# Error ratio (5xx / all)
sum(rate(http_requests_total{status="5xx"}[5m])) / sum(rate(http_requests_total[5m]))

# 95th percentile latency
histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_bucket[5m])))

# Container restarts in the last hour
increase(kube_pod_container_status_restarts_total[1h]) > 0

# Memory usage as % of limit
container_memory_working_set_bytes / on(pod, container) kube_pod_container_resource_limits{
    ↪ resource="memory"}
```

17.4 What to measure: golden signals

Signal	Question
Latency	How long do requests take (p50, p95, p99)?
Traffic	How many requests per second?
Errors	What fraction of requests fail?
Saturation	How full is the system (CPU, memory, queue depth, connections)?

17.5 Alerts

api-alerts.yaml

```yaml
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: api-alerts
  namespace: shop
  labels:
    release: monitoring
spec:
  groups:
    - name: api.rules
      rules:
        - alert: APIHighErrorRate
          expr: |
            sum(rate(http_requests_total{namespace="shop",status="5xx"}[5m]))
              / sum(rate(http_requests_total{namespace="shop"}[5m])) > 0.05
          for: 10m
          labels:
            severity: page
          annotations:
            summary: "API error rate above 5% for 10 minutes"
            runbook_url: "https://wiki.example.com/runbooks/api-errors"
```

Pro Tip

Alert on **symptoms users feel** (error rate, latency, availability — ideally expressed as SLOs), not on every cause ("CPU is 80%"). Every paging alert should be actionable and link to a runbook.

17.6 Traces with OpenTelemetry

A single click may touch a gateway, three services, a cache, and a database. A **trace** follows that request end-to-end as a tree of timed **spans**. **OpenTelemetry** (OTel) is the vendor-neutral standard: instrument once with the OTel SDK, send data to an **OTel Collector** running in the cluster, and export to Tempo, Jaeger, or a commercial backend.

otel-env.yaml

```yaml
env:
  - name: OTEL_SERVICE_NAME
    value: taskflow-api
  - name: OTEL_EXPORTER_OTLP_ENDPOINT
    value: http://otel-collector.observability:4317
  - name: OTEL_RESOURCE_ATTRIBUTES
    value: deployment.environment=production
```

17.7 Kubernetes events

Events are the cluster's own log: scheduling decisions, image pulls, probe failures, OOM kills. They expire after about an hour, so export them (kube-state-metrics, an event exporter) if you want history.

terminal

```
kubectl get events -n shop --sort-by=.lastTimestamp
kubectl get events -A --field-selector type=Warning
```

Chapter Recap

- Logs: stdout/stderr, shipped by a node agent to a central store.
- Metrics: Prometheus scrapes `/metrics`; Grafana visualises; Alertmanager notifies.
- ServiceMonitors and PrometheusRules configure monitoring declaratively.
- Traces: OpenTelemetry shows where time goes across services.



PART VI

Advanced Kubernetes

Packaging, extending, automating, and running in production

CHAPTER 18

18 Packaging: Helm and Kustomize



In this chapter you will learn

- why plain YAML stops scaling across environments;
- Kustomize: bases, overlays, patches, generators;
- Helm: charts, values, templates, releases, repositories;
- when to choose which (and how to use both).

18.1 The problem

Your app needs the same manifests in dev, staging, and production, but with different replica counts, image tags, resource sizes, and hostnames. Copying folders and editing by hand quickly leads to drift and mistakes. Two tools dominate:

	Kustomize	Helm
Idea	Patch plain YAML with overlays.	Render templates with values.
Templating	None — pure YAML.	Go templates + Sprig functions.
Built in	Yes: `kubectl apply -k`.	Separate CLI.
Distribution	Git folders.	Versioned charts in repos/OCI registries.
Release tracking	No.	Yes: install, upgrade, rollback, history.
Best for	Your own apps across environments.	Sharing apps; installing third-party software.

18.2 Kustomize

18.2.1 Layout

folder layout

```
k8s/
  base/
    deployment.yaml
    service.yaml
    kustomization.yaml
  overlays/
    dev/
      kustomization.yaml
    prod/
      kustomization.yaml
      replicas-patch.yaml
```

k8s/base/kustomization.yaml

```yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - deployment.yaml
  - service.yaml
labels:
  - pairs:
      app.kubernetes.io/part-of: shop
    includeSelectors: false
configMapGenerator:
  - name: app-config
    literals:
      - LOG_LEVEL=info
```

k8s/overlays/prod/kustomization.yaml

```yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: shop-prod
resources:
  - ../../base
images:
  - name: ghcr.io/example/api          # rewrite the image tag everywhere
    newTag: "1.4.0"
patches:
  - path: replicas-patch.yaml
  - target:
      kind: Deployment
      name: api
    patch: |-
      - op: replace
        path: /spec/template/spec/containers/0/resources/limits/memory
        value: 1Gi
configMapGenerator:
  - name: app-config
    behavior: merge
    literals:
      - LOG_LEVEL=warn
```

k8s/overlays/prod/replicas-patch.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
spec:
  replicas: 4
```

terminal

```
kubectl kustomize k8s/overlays/prod      # render to stdout and inspect
kubectl apply -k k8s/overlays/prod       # render and apply
kubectl diff  -k k8s/overlays/prod
```

Key Idea

`configMapGenerator` appends a content hash to the ConfigMap name (`app-config-7g2k9`) and rewrites references to it. Change a value and the Deployment's Pod template changes too — triggering a rolling update automatically.

18.3 Helm

Helm is the package manager for Kubernetes. Its vocabulary:

- a **chart** — a package of templated manifests;
- **values** — the inputs that customise a chart;
- a **release** — one installed instance of a chart in a cluster;
- a **repository** — where charts are published (HTTP or OCI registry).

18.3.1 Using other people's charts

terminal

```
helm repo add bitnami https://charts.bitnami.com/bitnami
helm search repo redis
helm show values bitnami/redis > redis-values.yaml     # see every knob

helm install cache bitnami/redis -n cache --create-namespace -f redis-values.yaml
helm list -A
helm upgrade cache bitnami/redis -n cache -f redis-values.yaml --set replica.replicaCount=2
helm history cache -n cache
helm rollback cache 1 -n cache
helm uninstall cache -n cache

# OCI registries work too
helm install eg oci://docker.io/envoyproxy/gateway-helm --version v1.5.0 -n envoy-gateway-system
    ↪ --create-namespace
```

18.3.2 Writing your own chart

terminal

```
helm create api      # scaffolds a working chart
```

chart layout

```
api/
  Chart.yaml            # name, version, appVersion
  values.yaml           # default values
  templates/
    _helpers.tpl        # reusable named templates
    configmap.yaml
    deployment.yaml
    service.yaml
    hpa.yaml
    NOTES.txt           # printed after install
```

api/values.yaml

```yaml
replicaCount: 2
image:
  repository: ghcr.io/example/api
  tag: ""                   # defaults to Chart.appVersion
  pullPolicy: IfNotPresent
service:
  port: 80
resources:
  requests: { cpu: 100m, memory: 128Mi }
  limits:   { memory: 256Mi }
autoscaling:
  enabled: false
  minReplicas: 2
  maxReplicas: 10
```

api/templates/deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "api.fullname" . }}
  labels:
    {{- include "api.labels" . | nindent 4 }}
spec:
  {{- if not .Values.autoscaling.enabled }}
  replicas: {{ .Values.replicaCount }}
  {{- end }}
  selector:
    matchLabels:
      {{- include "api.selectorLabels" . | nindent 6 }}
  template:
    metadata:
      labels:
        {{- include "api.selectorLabels" . | nindent 8 }}
      annotations:
```

```
        checksum/config: {{ include (print $.Template.BasePath "/configmap.yaml") . | sha256sum }}
    spec:
      containers:
        - name: api
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag | default .Chart.AppVersion
    ↪ }}"
          imagePullPolicy: {{ .Values.image.pullPolicy }}
          ports:
            - name: http
              containerPort: 8000
          resources:
            {{- toYaml .Values.resources | nindent 12 }}
```

terminal

```
helm lint ./api
helm template my-api ./api -f values-prod.yaml      # render locally, no cluster needed
helm install my-api ./api -n shop --create-namespace
helm upgrade --install my-api ./api -n shop -f values-prod.yaml --atomic --wait
helm package ./api && helm push api-0.1.0.tgz oci://ghcr.io/example/charts
```

Pro Tip

In CI/CD always use `helm upgrade -install -atomic -wait`: it installs or upgrades idempotently, waits until resources are ready, and rolls back automatically on failure.

18.4 Which one should I use?

A common, pragmatic split:

- **Helm** to install third-party software (Prometheus, cert-manager, Argo CD, databases).
- **Kustomize** (or a small Helm chart) for your own services.
- Kustomize can even inflate Helm charts (`helmCharts:` field), and Argo CD understands both natively.

Chapter Recap

- Kustomize: base + overlays + patches, template-free, built into kubectl.
- Helm: charts + values = releases, with upgrade/rollback history.
- Generators/checksums turn config changes into rolling updates.

CHAPTER 19

19 Extending Kubernetes: CRDs, Controllers, Operators



In this chapter you will learn

- how Kubernetes can learn brand-new resource types;
- CustomResourceDefinitions (CRDs) and custom resources;
- the controller pattern you can implement yourself;
- what Operators are and when to use them;
- admission webhooks and the wider extension landscape.

19.1 Kubernetes as a platform for platforms

Everything you have learned follows one pattern: *an object stores desired state, and a controller reconciles reality toward it*. Kubernetes lets you add your own objects and your own controllers using exactly the same machinery. This is why tools like cert-manager, Argo CD, Prometheus Operator, Crossplane, and KEDA feel "native": they are.

19.2 CustomResourceDefinitions

A **CRD** teaches the API server a new `kind`, complete with an OpenAPI schema for validation, `kubectl` support, RBAC, and watch events.

website-crd.yaml

```yaml
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: websites.demo.example.com      # <plural>.<group>
spec:
  group: demo.example.com
  scope: Namespaced
  names:
    kind: Website
    plural: websites
    singular: website
    shortNames: [ws]
  versions:
    - name: v1alpha1
      served: true
```

```
      storage: true
      schema:
        openAPIV3Schema:
          type: object
          properties:
            spec:
              type: object
              required: [gitRepo]
              properties:
                gitRepo:  { type: string }
                replicas: { type: integer, minimum: 1, maximum: 10, default: 2 }
            status:
              type: object
              properties:
                url: { type: string }
      subresources:
        status: {}
      additionalPrinterColumns:
        - name: Replicas
          type: integer
          jsonPath: .spec.replicas
        - name: URL
          type: string
          jsonPath: .status.url
```

my-site.yaml

```yaml
apiVersion: demo.example.com/v1alpha1
kind: Website
metadata:
  name: docs
spec:
  gitRepo: https://github.com/example/docs-site
  replicas: 3
```

terminal

```
kubectl apply -f website-crd.yaml
kubectl apply -f my-site.yaml
kubectl get websites          # or: kubectl get ws
```

On its own, a CRD is just storage: the `Website` is saved, but nothing happens. To make it *do* something you need a controller.

19.3 Writing a controller

A controller watches your custom resources (and the objects it creates) and reconciles. For the `Website` example, the reconcile function might:

1. read the `Website` object;
2. ensure a Deployment exists that clones `gitRepo` and serves it, with `replicas` copies;
3. ensure a Service and an HTTPRoute exist;

4. write the public URL into `status.url`;
5. set `ownerReferences` so deleting the Website garbage-collects everything.

reconcile.go (sketch, using controller-runtime)

```
func (r *WebsiteReconciler) Reconcile(ctx context.Context, req ctrl.Request) (ctrl.Result, error)
    ↪ {
    var site demov1alpha1.Website
    if err := r.Get(ctx, req.NamespacedName, &site); err != nil {
        return ctrl.Result{}, client.IgnoreNotFound(err)   // deleted: nothing to do
    }

    deploy := &appsv1.Deployment{ObjectMeta: metav1.ObjectMeta{Name: site.Name, Namespace: site.
    ↪ Namespace}}
    _, err := controllerutil.CreateOrUpdate(ctx, r.Client, deploy, func() error {
        deploy.Spec.Replicas = &site.Spec.Replicas
        // ... fill in the Pod template from site.Spec.GitRepo ...
        return controllerutil.SetControllerReference(&site, deploy, r.Scheme)
    })
    if err != nil {
        return ctrl.Result{}, err                          // retried with back-off
    }

    site.Status.URL = "https://" + site.Name + ".example.com"
    return ctrl.Result{}, r.Status().Update(ctx, &site)
}
```

Under the Hood

Reconcile functions must be **idempotent** and **level-triggered**: they look at the whole current state each time, not at "what changed". If a reconcile fails or the controller restarts, the next run simply converges again. Frameworks: **Kubebuilder** and **Operator SDK** (Go), **kopf** (Python), **Java Operator SDK**, **kube-rs** (Rust).

19.4 Operators

An **Operator** is a controller that encodes the operational knowledge of a human expert for a specific piece of software: how to install it, upgrade it, back it up, fail over, and scale it.

Operator	Manages
CloudNativePG	PostgreSQL clusters with replication, failover, backups to object storage.
Strimzi	Apache Kafka clusters, topics, and users.
Prometheus Operator	Prometheus, Alertmanager, ServiceMonitors, rules.
cert-manager	Certificates and issuers.
Crossplane	Cloud resources (databases, buckets, networks) as Kubernetes objects.

cnpg-cluster.yaml

```yaml
apiVersion: postgresql.cnpg.io/v1
kind: Cluster
metadata:
  name: taskflow-db
spec:
  instances: 3                # 1 primary + 2 replicas, automatic failover
  storage:
    size: 20Gi
  backup:
    barmanObjectStore:
      destinationPath: s3://my-backups/taskflow
      s3Credentials:
        accessKeyId:     { name: s3-creds, key: ACCESS_KEY_ID }
        secretAccessKey: { name: s3-creds, key: SECRET_ACCESS_KEY }
```

Key Idea

Three lines of YAML (`instances: 3`) replace pages of database runbooks. That is the promise of Operators — and why many teams run stateful systems on Kubernetes today.

19.5 Admission webhooks

Besides new types, you can intercept API requests:

- **Mutating** admission webhooks modify objects as they are created (inject sidecars, add default labels).
- **Validating** admission webhooks accept or reject them (enforce policy).

For many policies, the built-in **ValidatingAdmissionPolicy** and **MutatingAdmissionPolicy** (CEL expressions, no server to run) are now simpler than webhooks.

19.6 Other extension points

- **CNI**, **CSI**, **CRI** — network, storage, runtime plugins.
- **Device plugins** — GPUs and other hardware (with Dynamic Resource Allocation in newer releases).
- **Scheduler plugins** — custom scheduling logic.
- **API aggregation** — whole extra API servers (metrics-server uses this).
- **kubectl plugins** — any `kubectl-foo` binary on your PATH becomes `kubectl foo`; manage them with **krew**.

Chapter Recap

- CRDs add new resource types; controllers give them behaviour.
- Operators package expert operational knowledge as controllers.
- Admission control lets you mutate or validate every request.

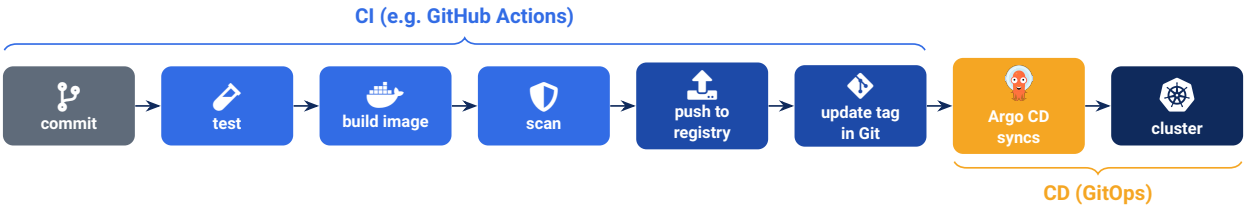
20 CHAPTER 20 CI/CD and GitOps



In this chapter you will learn

- the difference between Continuous Integration and Continuous Delivery;
- push-based vs. pull-based (GitOps) deployment;
- a CI pipeline that tests, builds, scans, and publishes images;
- GitOps with Argo CD: Applications, sync, self-heal, rollback;
- environment promotion and progressive delivery.

20.1 From commit to cluster



- **Continuous Integration (CI)**: every change is automatically tested, built into an image, scanned, and published.
- **Continuous Delivery (CD)**: every published version can be deployed to an environment automatically and safely.

20.2 Push vs. pull

	Push (CI runs `kubectl`/`helm`)	Pull / GitOps (agent in cluster)
Who deploys	The CI runner, with cluster credentials.	A controller inside the cluster.
Credentials	CI must hold a powerful kubeconfig.	Cluster only needs read access to Git.
Drift	Manual changes go unnoticed.	Detected and (optionally) reverted.
Audit / rollback	Re-run old pipeline.	`git log` / `git revert`.
Tools	GitHub Actions, GitLab CI, Jenkins.	Argo CD, Flux.

> **Key Idea**
>
> **GitOps**: the desired state of every environment lives in Git. An agent in the cluster continuously makes the cluster match Git. To deploy, you merge a pull request. To roll back, you revert it. Git becomes your audit log.

20.3 A CI pipeline with GitHub Actions

`.github/workflows/ci.yaml`

```yaml
name: ci
on:
  push:
    branches: [main]
  pull_request:

jobs:
  build:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12" }
      - run: pip install -r requirements.txt -r requirements-dev.txt && pytest -q

      - uses: docker/setup-buildx-action@v3
      - uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}
      - uses: docker/build-push-action@v6
        with:
          push: ${{ github.event_name == 'push' }}
          tags: ghcr.io/${{ github.repository }}:${{ github.sha }}
          cache-from: type=gha
          cache-to: type=gha,mode=max
```

The capstone (Chapter 23) extends this with scanning, manifest validation, and an automatic GitOps update.

20.4 GitOps with Argo CD

Argo CD is a CNCF-graduated GitOps controller with a web UI.

terminal

```
kubectl create namespace argocd
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/
    ↪ install.yaml

kubectl -n argocd get secret argocd-initial-admin-secret \
  -o jsonpath='{.data.password}' | base64 -d; echo
kubectl -n argocd port-forward svc/argocd-server 8443:443
# open https://localhost:8443  (user: admin)
```

20.4.1 The Application object

argocd-app.yaml

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: shop-prod
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/example/shop-deploy.git
    targetRevision: main
    path: k8s/overlays/prod          # Kustomize, Helm chart, or plain YAML
  destination:
    server: https://kubernetes.default.svc
    namespace: shop-prod
  syncPolicy:
    automated:
      prune: true                    # delete objects removed from Git
      selfHeal: true                 # revert manual changes in the cluster
    syncOptions:
      - CreateNamespace=true
    retry:
      limit: 5
      backoff: { duration: 10s, factor: 2, maxDuration: 3m }
```

Status	Meaning
Synced	The cluster matches Git.
OutOfSync	Git and the cluster differ; Argo CD will sync (if automated) or wait for you.
Healthy	Resources are running correctly (Deployments available, etc.).
Degraded	Something is failing (CrashLoop, failed rollout).

Pro Tip

Use the **app-of-apps** pattern or an **ApplicationSet** to manage many Applications (every service × every environment) from a single template, and keep your *application source* repo separate from your *deployment config* repo or folder.

20.5 Environment promotion

- **Dev**: CI updates the image tag in `overlays/dev` on every merge to `main`; Argo CD auto-syncs.
- **Staging**: a pull request copies the tag to `overlays/staging`; automated tests run against it.
- **Production**: a reviewed, approved pull request promotes the same immutable image (same digest) to `overlays/prod`.

Tools such as **Argo CD Image Updater**, **Kargo**, or **Flux** image automation can create these promotion commits for you.

20.6 Progressive delivery

Argo Rollouts replaces a Deployment with a `Rollout` that does canary or blue/green releases, shifting traffic in steps and checking metrics (e.g. Prometheus error rate) before each step. If metrics degrade, it aborts and rolls back automatically.

rollout-canary.yaml

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: api
spec:
  replicas: 5
  strategy:
    canary:
      steps:
        - setWeight: 10
        - pause: { duration: 5m }
        - analysis:
            templates:
              - templateName: error-rate-below-1pct
        - setWeight: 50
        - pause: { duration: 10m }
  selector:
    matchLabels: { app: api }
  template:
    metadata:
      labels: { app: api }
    spec:
      containers:
        - name: api
          image: ghcr.io/example/api:1.5.0
```

Chapter Recap

- CI: test, build, scan, publish. CD: deploy safely and repeatably.
- GitOps keeps environments in Git and lets an in-cluster agent pull changes.
- Argo CD Applications sync, prune, and self-heal.

- Promote the *same* image through environments via pull requests.



CHAPTER 21

Kubernetes in Production



In this chapter you will learn

- managed vs. self-managed clusters;
- high availability for the control plane and your apps;
- upgrades without drama;
- backup and disaster recovery with etcd snapshots and Velero;
- multi-cluster, service mesh, cost management, and a go-live checklist.

21.1 Managed or self-managed?

	Managed (EKS, GKE, AKS...)	Self-managed (kubeadm, k3s, RKE2, Talos)
Control plane	Run, patched, and backed up by the provider.	You run etcd, API servers, certificates.
Upgrades	A button or API call (nodes still need care).	Fully your responsibility.
Cost	Small control-plane fee.	Engineers' time.
When	Almost always, in the cloud.	On-premises, edge, air-gapped, special compliance.

Key Idea

Unless you have a strong reason, **use a managed Kubernetes service**. Your competitive advantage is your application, not running etcd.

21.2 High availability

21.2.1 Control plane

- 3 (or 5) control-plane nodes across availability zones;
- etcd with an odd number of members (quorum = majority);
- API servers behind a load balancer.

21.2.2 Your applications

- at least **2–3 replicas** for anything user-facing;
- **topology spread** across zones and nodes;
- **PodDisruptionBudgets** so drains never take everything down;
- **readiness probes** and graceful shutdown for zero-downtime deploys;
- **requests/limits** so noisy neighbours cannot starve you;
- node pools across multiple zones.

21.3 Upgrades

Kubernetes releases a minor version about every four months and supports each for roughly 14 months. Falling behind is painful; upgrade regularly.

1. **Read the release notes** and the *deprecated API* guide. Check your manifests and charts with tools like **pluto** or **kubent**.
2. Upgrade **one minor version at a time** (1.32 → 1.33 → 1.34).
3. Upgrade the **control plane first**, then nodes. Nodes may lag the control plane by a few minor versions, never lead it.
4. Upgrade nodes by **cordon & drain** or by creating a new node pool and moving workloads (blue/green nodes).
5. Upgrade add-ons (CNI, CSI, CoreDNS, Gateway controller) to compatible versions.
6. Practise on staging first.

21.4 Backup and disaster recovery

21.4.1 etcd snapshots (self-managed clusters)

```terminal
ETCDCTL_API=3 etcdctl snapshot save /backup/etcd-$(date +%F).db \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key

etcdutl snapshot status /backup/etcd-2026-10-10.db --write-out=table
```

21.4.2 Velero: backing up workloads and volumes

Velero backs up Kubernetes objects and persistent volume data to object storage, and restores them — into the same or a different cluster.

```terminal
velero backup create shop-daily --include-namespaces shop --snapshot-volumes
velero schedule create shop-nightly --schedule "0 3 * * *" --include-namespaces shop --ttl 720h
velero restore create --from-backup shop-daily
```

Watch Out

A backup you have never restored is a hope, not a backup. Schedule regular restore drills into a scratch cluster and measure how long they take (your real Recovery Time Objective).

Remember too that if *everything* (manifests, Helm values, Argo CD Applications) is in Git, rebuilding a cluster's configuration is "create cluster, install Argo CD, point at the repo". Only *data* needs separate backups.

21.5 Multi-cluster

Teams run several clusters for isolation (prod vs. non-prod), regions, latency, or blast-radius reduction. Common tools:

- **Argo CD ApplicationSets** / **Flux** to deploy to many clusters from one repo;
- **Cluster API** to create and upgrade clusters declaratively;
- global load balancing / DNS to route users to the nearest healthy cluster.

21.6 Service mesh

A **service mesh** (Istio, Linkerd, Cilium Service Mesh) adds a transparent network layer between services, giving you:

- **mutual TLS** between every service, with identities, automatically;
- fine-grained **traffic control**: retries, timeouts, circuit breaking, canaries;
- uniform **telemetry**: golden metrics and traces for every call.

Modern meshes offer *sidecar-less* modes (Istio Ambient, Cilium) that lower the overhead. Adopt a mesh when you actually need mTLS everywhere or advanced traffic control — it is a significant piece of infrastructure.

21.7 Cost management

- **Right-size requests** using real usage (VPA recommendations, Prometheus data). Over-requesting is the number-one source of waste.
- Use **autoscaling** at every level, and scale dev environments to zero at night.
- Run interruption-tolerant workloads on **spot/preemptible** nodes.

- Use **ARM nodes** where your images support them.
- Allocate costs per namespace/team with **OpenCost** or **Kubecost**.

21.8 Go-live checklist

- [x] Managed control plane, multi-zone nodes
- [x] Everything in Git; GitOps sync
- [x] Images pinned, scanned, signed
- [x] Probes, requests/limits, PDBs
- [x] HPA + cluster autoscaling
- [x] Topology spread across zones
- [x] TLS via cert-manager
- [x] NetworkPolicies, restricted PSS
- [x] Least-privilege RBAC, SSO
- [x] External secret manager
- [x] Metrics, logs, traces, SLO alerts
- [x] Runbooks for every alert
- [x] Backups + tested restores
- [x] Upgrade plan and cadence

Chapter Recap

- Prefer managed clusters; spend your time on your apps.
- HA = replicas + spread + PDBs + probes, across zones.
- Upgrade one minor version at a time, control plane first.
- Back up data (Velero, snapshots) and keep config in Git; test restores.



PART VII

Capstone Project

TaskFlow: build it, containerize it, ship it, scale it, watch it

CHAPTER 22

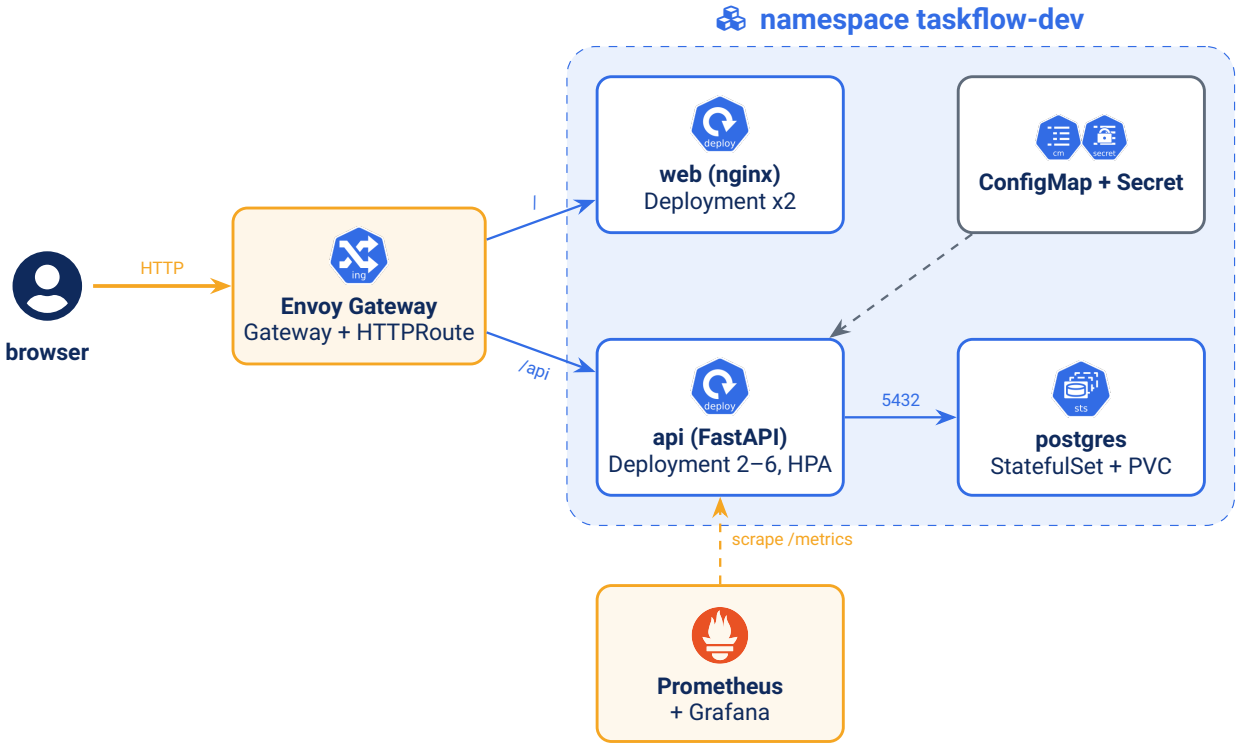
22 Capstone, Part 1: The TaskFlow Codebase

In this chapter you will learn

- what we are building and how its pieces fit together;
- the full repository layout;
- the complete source code of the API and the web frontend;
- production-quality Dockerfiles for both services;
- how to run the API locally before Kubernetes enters the picture.

22.1 What we are building

TaskFlow is a small to-do application. It is deliberately simple in *features* but realistic in *shape*: a static web frontend, a Python REST API, and a PostgreSQL database. That three-tier shape is the same one you will meet in most real companies, so everything you do here transfers directly.



Component	Technology	Responsibility
web	HTML + JavaScript on unprivileged nginx	Serves the single-page UI.
api	Python 3.12, FastAPI, psycopg 3	REST endpoints, health checks, Prometheus metrics.
postgres	PostgreSQL 16	Stores tasks on a persistent volume.
gateway	Envoy Gateway (Gateway API)	Routes `/` to web and `/api` to the API.

22.2 Repository layout

taskflow/

```
taskflow/
  api/                          # Python backend
    main.py
    requirements.txt
    requirements-dev.txt
    pytest.ini
    tests/
      test_api.py
    Dockerfile
    .dockerignore
  web/                          # static frontend
    index.html
    Dockerfile
  k8s/                          # Kubernetes manifests (Kustomize)
    base/
      kustomization.yaml
      postgres.yaml
      api.yaml
      web.yaml
      gateway.yaml
      autoscaling.yaml
      network-policies.yaml
    overlays/
      dev/
        kustomization.yaml
        namespace.yaml
      prod/
        kustomization.yaml
        namespace.yaml
        production-patch.yaml
        db-external-secret.yaml
  deploy/                       # cluster-level and platform config
    kind-cluster.yaml
    gatewayclass.yaml
    monitoring/
      servicemonitor.yaml
      alerts.yaml
```

```
    argocd/
      taskflow-dev.yaml
  charts/taskflow/              # optional Helm packaging (Chapter 23)
  scripts/
    lab-up.sh                   # build everything and deploy to kind
    lab-down.sh
  .github/workflows/ci.yaml     # CI pipeline
  docker-compose.yaml           # local API development
```

22.3 The API

The API is a single file. Read it once top-to-bottom; every Kubernetes-relevant choice is called out afterwards.

api/main.py

```python
"""TaskFlow API - the backend of the Kubernetes Handbook capstone project."""
import hashlib
import logging
import os
import time
from contextlib import asynccontextmanager

from fastapi import FastAPI, HTTPException, Response
from prometheus_fastapi_instrumentator import Instrumentator
from psycopg.rows import dict_row
from psycopg_pool import ConnectionPool
from pydantic import BaseModel, Field

# ---------------------------------------------------------------------------
# Configuration comes ONLY from environment variables (12-factor).
# In Kubernetes these are filled from a ConfigMap and a Secret.
# ---------------------------------------------------------------------------
DATABASE_URL = os.getenv("DATABASE_URL", "postgresql://taskflow:taskflow@localhost:5432/taskflow")
APP_VERSION = os.getenv("APP_VERSION", "dev")
POD_NAME = os.getenv("POD_NAME", "local")
LOG_LEVEL = os.getenv("LOG_LEVEL", "info").upper()

logging.basicConfig(level=LOG_LEVEL, format="%(asctime)s %(levelname)s %(name)s %(message)s")
log = logging.getLogger("taskflow")

# A small connection pool. open=False: we open it in the lifespan hook so the
# process starts even if the database is not reachable yet.
pool = ConnectionPool(
    DATABASE_URL, min_size=1, max_size=10, open=False,
    kwargs={"row_factory": dict_row},
)

SCHEMA = """
CREATE TABLE IF NOT EXISTS tasks (
    id         SERIAL PRIMARY KEY,
    title      TEXT        NOT NULL,
    done       BOOLEAN     NOT NULL DEFAULT FALSE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
)
```

```
"""
_schema_ready = False


def ensure_schema() -> None:
    """Create the table on first use (idempotent, safe with many replicas)."""
    global _schema_ready
    if not _schema_ready:
        with pool.connection() as conn:
            conn.execute(SCHEMA)
        _schema_ready = True


@asynccontextmanager
async def lifespan(app: FastAPI):
    pool.open(wait=False)
    log.info("taskflow-api starting version=%s pod=%s", APP_VERSION, POD_NAME)
    yield
    # Runs on SIGTERM: Kubernetes sends it before stopping the container.
    log.info("shutting down, closing database pool")
    pool.close()


app = FastAPI(title="TaskFlow API", version=APP_VERSION, lifespan=lifespan)

# Exposes Prometheus metrics at /metrics (http_requests_total, latency histograms...)
Instrumentator().instrument(app).expose(app, include_in_schema=False)


class TaskIn(BaseModel):
    title: str = Field(min_length=1, max_length=200)


class TaskPatch(BaseModel):
    done: bool


# ---------------------------------------------------------------------------
# Health endpoints used by Kubernetes probes
# ---------------------------------------------------------------------------
@app.get("/healthz", include_in_schema=False)
def healthz():
    """Liveness: is the process able to answer at all? No dependencies here."""
    return {"status": "ok"}


@app.get("/readyz", include_in_schema=False)
def readyz():
    """Readiness: can we serve real traffic? Checks the database."""
    try:
        ensure_schema()
        with pool.connection(timeout=2) as conn:
            conn.execute("SELECT 1")
    except Exception as exc:  # noqa: BLE001
        log.warning("not ready: %s", exc)
        raise HTTPException(status_code=503, detail="database unavailable")
    return {"status": "ready"}
```

```
# ---------------------------------------------------------------------------
# Public API
# ---------------------------------------------------------------------------
@app.get("/api/info")
def info():
    """Shows which Pod answered - handy for watching load balancing."""
    return {"service": "taskflow-api", "version": APP_VERSION, "pod": POD_NAME}


@app.get("/api/tasks")
def list_tasks():
    ensure_schema()
    with pool.connection() as conn:
        return conn.execute(
            "SELECT id, title, done, created_at FROM tasks ORDER BY id DESC LIMIT 100"
        ).fetchall()


@app.post("/api/tasks", status_code=201)
def create_task(task: TaskIn):
    ensure_schema()
    with pool.connection() as conn:
        return conn.execute(
            "INSERT INTO tasks (title) VALUES (%s) RETURNING id, title, done, created_at",
            (task.title,),
        ).fetchone()


@app.patch("/api/tasks/{task_id}")
def update_task(task_id: int, patch: TaskPatch):
    ensure_schema()
    with pool.connection() as conn:
        row = conn.execute(
            "UPDATE tasks SET done = %s WHERE id = %s RETURNING id, title, done, created_at",
            (patch.done, task_id),
        ).fetchone()
    if row is None:
        raise HTTPException(status_code=404, detail="task not found")
    return row


@app.delete("/api/tasks/{task_id}", status_code=204)
def delete_task(task_id: int):
    ensure_schema()
    with pool.connection() as conn:
        deleted = conn.execute("DELETE FROM tasks WHERE id = %s", (task_id,)).rowcount
    if deleted == 0:
        raise HTTPException(status_code=404, detail="task not found")
    return Response(status_code=204)


@app.get("/api/work")
def work(ms: int = 100):
    """Burns CPU for `ms` milliseconds. Used to demonstrate autoscaling."""
    deadline = time.perf_counter() + min(ms, 2000) / 1000
```

```
    digest = b"taskflow"
    while time.perf_counter() < deadline:
        digest = hashlib.sha256(digest).digest()
    return {"pod": POD_NAME, "worked_ms": ms, "digest": digest.hex()[:12]}
```

22.3.1 Why the code looks like this

Design choice	Kubernetes reason
Config from env vars only	The same image runs in every environment; ConfigMaps/Secrets inject the differences.
`/healthz` has no dependencies	Used by the *liveness* probe; a DB outage must not restart every Pod.
`/readyz` checks the DB	Used by the *readiness* probe; Pods without a DB leave the Service until it is back.
Pool opens lazily (`wait=False`)	The Pod starts even if Postgres is still booting; readiness gates traffic.
`CREATE TABLE IF NOT EXISTS`	Idempotent: safe when several replicas start at once.
Cleanup in `lifespan`	Graceful shutdown on `SIGTERM` during rollouts and scale-down.
`/metrics` endpoint	Scraped by Prometheus via a ServiceMonitor.
`/api/info` returns the Pod name	Lets you *see* load balancing across replicas.
`/api/work` burns CPU	Generates realistic load to trigger the HPA.
Logs to stdout	Collected by the node's log agent; `kubectl logs` just works.

22.3.2 Dependencies and tests

api/requirements.txt

```
fastapi>=0.115,<1.0
uvicorn[standard]>=0.30
psycopg[binary]>=3.2
psycopg-pool>=3.2
prometheus-fastapi-instrumentator>=7.0
```

api/requirements-dev.txt

```
pytest>=8.0
httpx>=0.27
```

api/pytest.ini

```
[pytest]
pythonpath = .
testpaths = tests
```

api/tests/test_api.py

```python
"""Fast unit tests that need no database - they run in CI on every commit."""
from fastapi.testclient import TestClient

from main import app

client = TestClient(app)


def test_liveness_is_independent_of_database():
    r = client.get("/healthz")
    assert r.status_code == 200
    assert r.json() == {"status": "ok"}


def test_info_reports_service_and_pod():
    body = client.get("/api/info").json()
    assert body["service"] == "taskflow-api"
    assert "pod" in body


def test_empty_title_is_rejected_before_touching_the_db():
    r = client.post("/api/tasks", json={"title": ""})
    assert r.status_code == 422


def test_metrics_are_exposed_for_prometheus():
    client.get("/healthz")
    r = client.get("/metrics")
    assert r.status_code == 200
    assert "http_requests_total" in r.text


def test_work_endpoint_caps_duration():
    body = client.get("/api/work", params={"ms": 1}).json()
    assert body["worked_ms"] == 1
```

22.4 The web frontend

A single HTML file with a little JavaScript. It calls the API with *relative* URLs (`/api/tasks`), so it works behind any hostname — the Gateway decides where `/api` goes.

web/index.html

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>TaskFlow</title>
  <style>
    body   { font-family: system-ui, sans-serif; background: #eaf1fd; margin: 0; }
    header { background: #326ce5; color: #fff; padding: 1rem 2rem; }
    main   { max-width: 640px; margin: 2rem auto; background: #fff; padding: 1.5rem;
```

```
            border-radius: 12px; box-shadow: 0 4px 16px rgba(15, 42, 92, .12); }
    form   { display: flex; gap: .5rem; }
    input  { flex: 1; padding: .6rem; border: 1px solid #8fb0f2; border-radius: 8px; }
    button { background: #326ce5; color: #fff; border: 0; padding: .6rem 1rem; border-radius: 8px;
    ↪  cursor: pointer; }
    li     { display: flex; justify-content: space-between; padding: .5rem 0; border-bottom: 1px
    ↪ solid #eaf1fd; }
    .done  { text-decoration: line-through; color: #5f6b7a; }
    small  { color: #5f6b7a; }
  </style>
</head>
<body>
  <header><h1>TaskFlow</h1><small id="served-by">connecting...</small></header>
  <main>
    <form id="new-task">
      <input id="title" placeholder="What needs doing?" required maxlength="200">
      <button type="submit">Add</button>
    </form>
    <ul id="tasks"></ul>
  </main>
  <script>
    const list = document.getElementById("tasks");

    async function api(path, options = {}) {
      const res = await fetch(path, { headers: { "Content-Type": "application/json" }, ...options
    ↪ });
      if (!res.ok && res.status !== 204) throw new Error(res.status + " " + res.statusText);
      return res.status === 204 ? null : res.json();
    }

    async function refresh() {
      const info = await api("/api/info");
      document.getElementById("served-by").textContent =
        "API " + info.version + " served by Pod " + info.pod;
      const tasks = await api("/api/tasks");
      list.innerHTML = "";
      for (const t of tasks) {
        const li = document.createElement("li");
        const label = document.createElement("span");
        label.textContent = t.title;
        if (t.done) label.className = "done";
        label.onclick = () => api("/api/tasks/" + t.id, { method: "PATCH", body: JSON.stringify({
    ↪ done: !t.done }) }).then(refresh);
        const del = document.createElement("button");
        del.textContent = "Delete";
        del.onclick = () => api("/api/tasks/" + t.id, { method: "DELETE" }).then(refresh);
        li.append(label, del);
        list.append(li);
      }
    }

    document.getElementById("new-task").onsubmit = async (e) => {
      e.preventDefault();
      const input = document.getElementById("title");
      await api("/api/tasks", { method: "POST", body: JSON.stringify({ title: input.value }) });
      input.value = "";
      refresh();
```

```
    };

    refresh().catch(err => document.getElementById("served-by").textContent = "API error: " + err.
    ↪ message);
  </script>
</body>
</html>
```

22.5 Container images

22.5.1 API image: multi-stage, non-root

api/Dockerfile

```
# syntax=docker/dockerfile:1

# ---------- Stage 1: install dependencies ----------
FROM python:3.12-slim AS builder
WORKDIR /build
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

# ---------- Stage 2: small runtime image ----------
FROM python:3.12-slim
# The version is a property of the build, so CI passes it in (e.g. the git SHA)
ARG APP_VERSION=dev
ENV APP_VERSION=${APP_VERSION} \
    PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1
WORKDIR /app
COPY --from=builder /install /usr/local
COPY main.py .

# Numeric UID so Kubernetes can verify runAsNonRoot
RUN useradd --uid 10001 --no-create-home --shell /usr/sbin/nologin app
USER 10001

EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000", "--timeout-graceful-shutdown",
    ↪ "20"]
```

api/.dockerignore

```
__pycache__/
*.pyc
.pytest_cache/
tests/
.venv/
```

22.5.2 Web image: unprivileged nginx

web/Dockerfile

```
# nginx-unprivileged runs as UID 101 and listens on 8080 (no root needed)
FROM nginxinc/nginx-unprivileged:1.27-alpine
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 8080
```

22.6 Run the API locally first

Before touching Kubernetes, prove the API works with Docker Compose. This is the fastest feedback loop while writing code.

docker-compose.yaml

```
services:
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: taskflow
      POSTGRES_PASSWORD: taskflow
      POSTGRES_DB: taskflow
    ports: ["5432:5432"]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U taskflow"]
      interval: 3s
  api:
    build: ./api
    environment:
      DATABASE_URL: postgresql://taskflow:taskflow@db:5432/taskflow
      APP_VERSION: compose
    ports: ["8000:8000"]
    depends_on:
      db: { condition: service_healthy }
```

terminal

```
docker compose up --build -d
curl localhost:8000/readyz                                    # {"status":"ready"}
curl -X POST localhost:8000/api/tasks -H 'Content-Type: application/json' -d '{"title":"Learn
    ↪ Kubernetes"}'
curl localhost:8000/api/tasks
# browse http://localhost:8000/docs for interactive API docs
docker compose down -v

# unit tests
cd api && pip install -r requirements.txt -r requirements-dev.txt && pytest -q
```

Chapter Recap

- TaskFlow = nginx frontend + FastAPI backend + PostgreSQL, routed by a Gateway.
- The code is “Kubernetes-ready”: env-var config, separate liveness and readiness end-

points, graceful shutdown, metrics, stdout logs.

- Images are small, multi-stage, and run as non-root numeric users.
- Next chapter: we do all the DevOps — cluster, manifests, scaling, security, monitoring, CI, and GitOps.

CHAPTER 23

23 Capstone, Part 2: DevOps for TaskFlow on Kubernetes

In this chapter you will learn

- create a multi-node, multi-"zone" cluster and install platform add-ons;
- write production-grade manifests: StatefulSet, Deployments, Services, Gateway, HPA, PDBs, NetworkPolicies, Pod Security;
- manage environments with Kustomize overlays (and optionally Helm);
- test rolling updates, rollbacks, self-healing, persistence, and autoscaling;
- add monitoring and alerts with Prometheus and Grafana;
- build a CI pipeline and deliver with GitOps through Argo CD.

23.1 The road map



Watch Out

Before you start, **replace** `your-github-user` **everywhere** in the repository with your GitHub user name in lower case (one search-and-replace in your editor). On Windows, run the commands in **Git Bash** or **WSL**: they use bash syntax.

23.2 Step 1 — Cluster and platform add-ons

23.2.1 A three-node cluster with two "zones"

We label the two workers with different zones so topology spreading has something to spread across.

`deploy/kind-cluster.yaml`

```
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
```

```
name: taskflow
nodes:
  - role: control-plane
  - role: worker
    labels:
      topology.kubernetes.io/zone: zone-a
  - role: worker
    labels:
      topology.kubernetes.io/zone: zone-b
```

terminal

```
kind create cluster --config deploy/kind-cluster.yaml
kubectl get nodes -L topology.kubernetes.io/zone
```

23.2.2 metrics-server (for `kubectl top` and the HPA)

terminal

```
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/
    ↪ components.yaml
kubectl -n kube-system patch deployment metrics-server --type=json \
  -p='[{"op":"add","path":"/spec/template/spec/containers/0/args/-","value":"--kubelet-insecure-
    ↪ tls"}]'
kubectl -n kube-system rollout status deployment/metrics-server
```

23.2.3 Envoy Gateway (Gateway API implementation)

The Helm chart installs the Gateway API CRDs and the Envoy Gateway controller. Then we declare a `GatewayClass` named `eg` that points to it.

terminal

```
helm install eg oci://docker.io/envoyproxy/gateway-helm --version v1.5.0 \
  -n envoy-gateway-system --create-namespace --wait
kubectl apply -f deploy/gatewayclass.yaml
kubectl get gatewayclass          # ACCEPTED should become True
```

deploy/gatewayclass.yaml

```
apiVersion: gateway.networking.k8s.io/v1
kind: GatewayClass
metadata:
  name: eg
spec:
  controllerName: gateway.envoyproxy.io/gatewayclass-controller
```

23.3 Step 2 — Build images and load them into kind

kind nodes cannot see your local Docker images, so we *load* them in. (Later, CI pushes to GHCR and the cluster pulls from there.)

terminal

```
docker build -t ghcr.io/your-github-user/taskflow-api:0.1.0 --build-arg APP_VERSION=0.1.0 api
docker build -t ghcr.io/your-github-user/taskflow-web:0.1.0 web
kind load docker-image --name taskflow \
  ghcr.io/your-github-user/taskflow-api:0.1.0 \
  ghcr.io/your-github-user/taskflow-web:0.1.0
```

23.4 Step 3 — The base manifests

Everything in `k8s/base` is environment-neutral: no namespace, no secrets, no environment-specific sizes. Overlays add those in Step 4.

23.4.1 PostgreSQL: StatefulSet + headless Service

k8s/base/postgres.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: postgres
  labels:
    app.kubernetes.io/name: postgres
spec:
  clusterIP: None                       # headless: stable DNS for postgres-0
  selector:
    app.kubernetes.io/name: postgres
  ports:
    - name: postgres
      port: 5432
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
  labels:
    app.kubernetes.io/name: postgres
    app.kubernetes.io/component: database
spec:
  serviceName: postgres
  replicas: 1
  selector:
    matchLabels:
      app.kubernetes.io/name: postgres
  template:
    metadata:
      labels:
        app.kubernetes.io/name: postgres
        app.kubernetes.io/component: database
    spec:
      securityContext:                  # satisfies the "restricted" Pod Security profile
        runAsNonRoot: true
        runAsUser: 70                   # the postgres user in the alpine image
        runAsGroup: 70
```

```
        fsGroup: 70
        seccompProfile:
          type: RuntimeDefault
      containers:
        - name: postgres
          image: postgres:16-alpine
          ports:
            - name: postgres
              containerPort: 5432
          env:
            - name: POSTGRES_DB
              value: taskflow
            - name: POSTGRES_USER
              valueFrom:
                secretKeyRef: { name: taskflow-db, key: username }
            - name: POSTGRES_PASSWORD
              valueFrom:
                secretKeyRef: { name: taskflow-db, key: password }
            - name: PGDATA
              value: /var/lib/postgresql/data/pgdata
          readinessProbe:
            exec:
              command: ["sh", "-c", 'pg_isready -U "$POSTGRES_USER" -d taskflow']
            periodSeconds: 5
          livenessProbe:
            exec:
              command: ["sh", "-c", 'pg_isready -U "$POSTGRES_USER" -d taskflow']
            initialDelaySeconds: 30
            periodSeconds: 10
          resources:
            requests: { cpu: 100m, memory: 256Mi }
            limits:   { memory: 512Mi }
          securityContext:
            allowPrivilegeEscalation: false
            capabilities:
              drop: ["ALL"]
          volumeMounts:
            - name: data
              mountPath: /var/lib/postgresql/data
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
        accessModes: ["ReadWriteOnce"]
        resources:
          requests:
            storage: 2Gi
```

23.4.2 The API: ServiceAccount, Deployment, Service

k8s/base/api.yaml

```yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: api
automountServiceAccountToken: false     # the API never calls Kubernetes
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
  labels:
    app.kubernetes.io/name: api
    app.kubernetes.io/component: backend
spec:
  # no "replicas": the HorizontalPodAutoscaler owns the count
  revisionHistoryLimit: 5
  selector:
    matchLabels:
      app.kubernetes.io/name: api
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0                 # zero-downtime rollouts
  template:
    metadata:
      labels:
        app.kubernetes.io/name: api
        app.kubernetes.io/component: backend
    spec:
      serviceAccountName: api
      terminationGracePeriodSeconds: 30
      securityContext:
        runAsNonRoot: true
        runAsUser: 10001
        runAsGroup: 10001
        seccompProfile:
          type: RuntimeDefault
      topologySpreadConstraints:
        - maxSkew: 1
          topologyKey: topology.kubernetes.io/zone
          whenUnsatisfiable: ScheduleAnyway
          labelSelector:
            matchLabels:
              app.kubernetes.io/name: api
      containers:
        - name: api
          image: ghcr.io/your-github-user/taskflow-api:0.1.0
          imagePullPolicy: IfNotPresent
          ports:
            - name: http
              containerPort: 8000
          envFrom:
```

```
            - configMapRef:
                name: taskflow-config
          env:
            - name: POD_NAME
              valueFrom:
                fieldRef: { fieldPath: metadata.name }
            - name: DB_USER
              valueFrom:
                secretKeyRef: { name: taskflow-db, key: username }
            - name: DB_PASSWORD
              valueFrom:
                secretKeyRef: { name: taskflow-db, key: password }
            - name: DATABASE_URL          # $(VAR) expands variables defined above
              value: postgresql://$(DB_USER):$(DB_PASSWORD)@postgres:5432/taskflow
          startupProbe:
            httpGet: { path: /healthz, port: http }
            periodSeconds: 2
            failureThreshold: 30
          livenessProbe:
            httpGet: { path: /healthz, port: http }
            periodSeconds: 10
            failureThreshold: 3
          readinessProbe:
            httpGet: { path: /readyz, port: http }
            periodSeconds: 5
            failureThreshold: 2
          lifecycle:
            preStop:
              exec:
                command: ["sleep", "5"]  # let endpoints update before SIGTERM
          resources:
            requests: { cpu: 100m, memory: 128Mi }
            limits:   { memory: 256Mi }
          securityContext:
            allowPrivilegeEscalation: false
            readOnlyRootFilesystem: true
            capabilities:
              drop: ["ALL"]
          volumeMounts:
            - name: tmp
              mountPath: /tmp
      volumes:
        - name: tmp
          emptyDir: {}
---
apiVersion: v1
kind: Service
metadata:
  name: api
  labels:
    app.kubernetes.io/name: api
spec:
  selector:
    app.kubernetes.io/name: api
  ports:
    - name: http
      port: 80
```

```
    targetPort: http
```

> **Watch Out**
>
> Because the password is placed into a URL, keep it URL-safe (letters, digits, `-` and `_`). Otherwise pass host, user, and password as separate variables and build the connection string in code.

23.4.3 The web frontend

k8s/base/web.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
  labels:
    app.kubernetes.io/name: web
    app.kubernetes.io/component: frontend
spec:
  replicas: 2
  selector:
    matchLabels:
      app.kubernetes.io/name: web
  template:
    metadata:
      labels:
        app.kubernetes.io/name: web
        app.kubernetes.io/component: frontend
    spec:
      automountServiceAccountToken: false
      securityContext:
        runAsNonRoot: true
        runAsUser: 101
        runAsGroup: 101
        seccompProfile:
          type: RuntimeDefault
      containers:
        - name: web
          image: ghcr.io/your-github-user/taskflow-web:0.1.0
          imagePullPolicy: IfNotPresent
          ports:
            - name: http
              containerPort: 8080
          readinessProbe:
            httpGet: { path: /, port: http }
            periodSeconds: 5
          livenessProbe:
            httpGet: { path: /, port: http }
            periodSeconds: 15
          resources:
            requests: { cpu: 20m, memory: 32Mi }
            limits:   { memory: 64Mi }
          securityContext:
            allowPrivilegeEscalation: false
            readOnlyRootFilesystem: true
```

```
            capabilities:
              drop: ["ALL"]
          volumeMounts:
            - name: tmp                    # nginx writes its pid and temp files here
              mountPath: /tmp
      volumes:
        - name: tmp
          emptyDir: {}
---
apiVersion: v1
kind: Service
metadata:
  name: web
  labels:
    app.kubernetes.io/name: web
spec:
  selector:
    app.kubernetes.io/name: web
  ports:
    - name: http
      port: 80
      targetPort: http
```

23.4.4 Gateway and routes

k8s/base/gateway.yaml

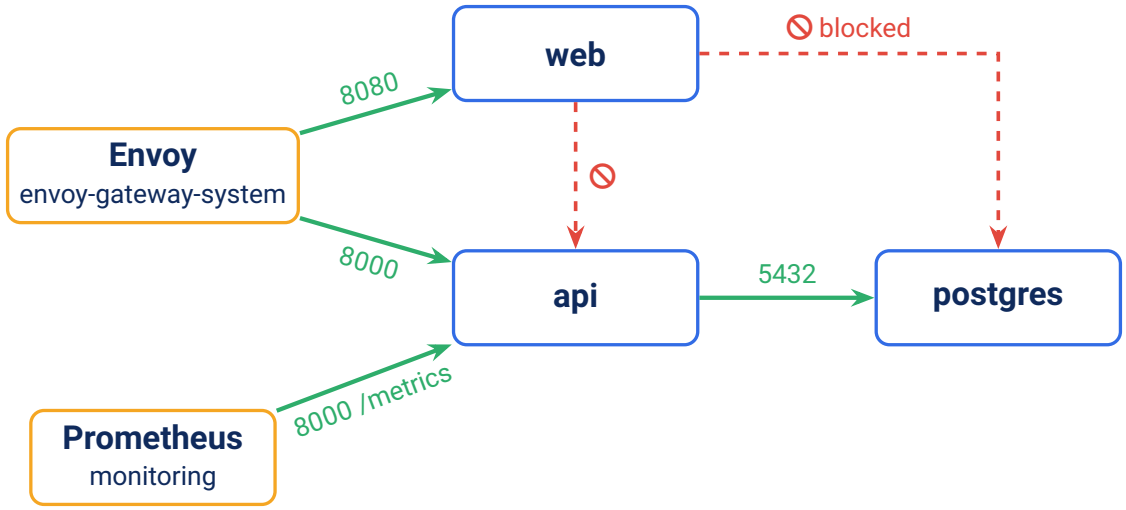
```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: taskflow-gateway
spec:
  gatewayClassName: eg
  listeners:
    - name: http
      protocol: HTTP
      port: 80
---
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: taskflow
spec:
  parentRefs:
    - name: taskflow-gateway
  rules:
    - matches:
        - path: { type: PathPrefix, value: /api }
      backendRefs:
        - name: api
          port: 80
    - matches:
        - path: { type: PathPrefix, value: / }
      backendRefs:
        - name: web
          port: 80
```

23.4.5 Autoscaling and disruption budgets

k8s/base/autoscaling.yaml

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: api
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: api
  minReplicas: 2
  maxReplicas: 6
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 120
---
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: api
spec:
  minAvailable: 1
  selector:
    matchLabels:
      app.kubernetes.io/name: api
---
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: web
spec:
  minAvailable: 1
  selector:
    matchLabels:
      app.kubernetes.io/name: web
```

23.4.6 Network policies: zero trust inside the namespace



k8s/base/network-policies.yaml

```yaml
# 1. Deny everything in and out of every Pod in this namespace
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
spec:
  podSelector: {}
  policyTypes: [Ingress, Egress]
---
# 2. Everyone may resolve DNS
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-dns
spec:
  podSelector: {}
  policyTypes: [Egress]
  egress:
    - to:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: kube-system
          podSelector:
            matchLabels:
              k8s-app: kube-dns
      ports:
        - { port: 53, protocol: UDP }
        - { port: 53, protocol: TCP }
---
# 3. The gateway may reach the web Pods
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: web-from-gateway
spec:
  podSelector:
```

```
    matchLabels:
      app.kubernetes.io/name: web
  policyTypes: [Ingress]
  ingress:
    - from:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: envoy-gateway-system
      ports:
        - { port: 8080, protocol: TCP }
---
# 4. The gateway and Prometheus may reach the API
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: api-ingress
spec:
  podSelector:
    matchLabels:
      app.kubernetes.io/name: api
  policyTypes: [Ingress]
  ingress:
    - from:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: envoy-gateway-system
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: monitoring
      ports:
        - { port: 8000, protocol: TCP }
---
# 5. The API may reach Postgres ...
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: api-egress-to-postgres
spec:
  podSelector:
    matchLabels:
      app.kubernetes.io/name: api
  policyTypes: [Egress]
  egress:
    - to:
        - podSelector:
            matchLabels:
              app.kubernetes.io/name: postgres
      ports:
        - { port: 5432, protocol: TCP }
---
# 6. ... and Postgres accepts only the API
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: postgres-from-api
spec:
  podSelector:
```

```
    matchLabels:
      app.kubernetes.io/name: postgres
  policyTypes: [Ingress]
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app.kubernetes.io/name: api
      ports:
        - { port: 5432, protocol: TCP }
```

> **Under the Hood**
>
> Recent kind releases enforce NetworkPolicies through their default CNI (kindnet). If you use an older kind version or another local cluster whose CNI ignores policies, install Calico or Cilium to see the drills in Step 6 behave as described.

23.4.7 The base kustomization

k8s/base/kustomization.yaml

```yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - postgres.yaml
  - api.yaml
  - web.yaml
  - gateway.yaml
  - autoscaling.yaml
  - network-policies.yaml
labels:
  - pairs:
      app.kubernetes.io/part-of: taskflow
    includeSelectors: false
configMapGenerator:
  - name: taskflow-config
    literals:
      - LOG_LEVEL=info
```

23.5 Step 4 — Environment overlays

23.5.1 Development

k8s/overlays/dev/namespace.yaml

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: taskflow-dev
  labels:
    pod-security.kubernetes.io/enforce: restricted
```

```
    pod-security.kubernetes.io/warn: restricted
```

k8s/overlays/dev/kustomization.yaml

```yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: taskflow-dev
resources:
  - namespace.yaml
  - ../../base
images:
  - name: ghcr.io/your-github-user/taskflow-api
    newTag: "0.1.0"
  - name: ghcr.io/your-github-user/taskflow-web
    newTag: "0.1.0"
configMapGenerator:
  - name: taskflow-config
    behavior: merge
    literals:
      - LOG_LEVEL=debug
secretGenerator:                        # acceptable ONLY for a local lab
  - name: taskflow-db
    literals:
      - username=taskflow
      - password=dev-only-password-change-me
```

23.5.2 Production

Production differs in four ways: its own namespace, pinned release tags, bigger sizes, and a real secret manager instead of a literal password.

k8s/overlays/prod/kustomization.yaml

```yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: taskflow-prod
resources:
  - namespace.yaml                      # same as dev, with name taskflow-prod
  - db-external-secret.yaml
  - ../../base
images:
  - name: ghcr.io/your-github-user/taskflow-api
    newTag: "1.0.0"
  - name: ghcr.io/your-github-user/taskflow-web
    newTag: "1.0.0"
patches:
  - path: production-patch.yaml
configMapGenerator:
  - name: taskflow-config
    behavior: merge
    literals:
      - LOG_LEVEL=warn
```

k8s/overlays/prod/production-patch.yaml

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: api
spec:
  minReplicas: 3
  maxReplicas: 20
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
spec:
  template:
    spec:
      containers:
        - name: api
          resources:
            requests: { cpu: 250m, memory: 256Mi }
            limits:   { memory: 512Mi }
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 3
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
spec:
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
        accessModes: ["ReadWriteOnce"]
        storageClassName: retain-ssd   # a Retain class, so data survives mistakes
        resources:
          requests:
            storage: 50Gi
```

k8s/overlays/prod/db-external-secret.yaml

```yaml
apiVersion: external-secrets.io/v1
kind: ExternalSecret
metadata:
  name: taskflow-db
spec:
  refreshInterval: 1h
  secretStoreRef:
    kind: ClusterSecretStore
    name: company-vault
  target:
```

```
  name: taskflow-db                 # creates the Secret the base expects
data:
  - secretKey: username
    remoteRef: { key: prod/taskflow/db, property: username }
  - secretKey: password
    remoteRef: { key: prod/taskflow/db, property: password }
```

> **Pro Tip**
>
> For a real production database, swap the StatefulSet for a **CloudNativePG** `Cluster` (Chapter 19) or a managed database service, and point `DATABASE_URL` at it. The rest of TaskFlow does not change at all.

Always inspect what an overlay renders before applying it:

terminal

```
kubectl kustomize k8s/overlays/dev | less
kubectl kustomize k8s/overlays/prod | grep -A2 "image:"
```

23.6 Step 5 — Deploy and verify

terminal

```
kubectl apply -k k8s/overlays/dev

kubectl -n taskflow-dev rollout status statefulset/postgres --timeout=180s
kubectl -n taskflow-dev rollout status deployment/api --timeout=180s
kubectl -n taskflow-dev rollout status deployment/web --timeout=180s

kubectl -n taskflow-dev get all,pvc,gateway,httproute,networkpolicy,hpa,pdb
```

Expected (abridged):

output

```
NAME                       READY   STATUS    RESTARTS   AGE
pod/api-6f7c9d8b5-4kq2z    1/1     Running   0          60s
pod/api-6f7c9d8b5-p8xwm    1/1     Running   0          45s
pod/postgres-0             1/1     Running   0          60s
pod/web-7d5b8c6f9-9tq7r    1/1     Running   0          60s
pod/web-7d5b8c6f9-h2m4c    1/1     Running   0          60s

NAME                                       REFERENCE        TARGETS       MINPODS   MAXPODS
    ↪ REPLICAS
horizontalpodautoscaler.autoscaling/api    Deployment/api   cpu: 3%/70%   2         6         2

NAME                                              CLASS   ADDRESS   PROGRAMMED
gateway.gateway.networking.k8s.io/taskflow-gateway   eg                True
```

23.6.1 Open the application

Envoy Gateway creates a Service for our Gateway in its own namespace. On a cloud cluster it would get a public IP; locally we port-forward to it:

terminal

```
export ENVOY_SERVICE=$(kubectl get svc -n envoy-gateway-system \
  --selector=gateway.envoyproxy.io/owning-gateway-namespace=taskflow-dev,gateway.envoyproxy.io/
    ↪ owning-gateway-name=taskflow-gateway \
  -o jsonpath='{.items[0].metadata.name}')
kubectl -n envoy-gateway-system port-forward service/${ENVOY_SERVICE} 8888:80 &

curl -s localhost:8888/api/info                # {"service":"taskflow-api","version":"0.1.0","pod
    ↪ ":"api-..."}
curl -s -X POST localhost:8888/api/tasks -H 'Content-Type: application/json' -d '{"title":"Ship
    ↪ TaskFlow"}'
curl -s localhost:8888/api/tasks
```

Now open `http://localhost:8888` in your browser. Refresh a few times and watch the "served by Pod" line change as the Service balances across replicas.

23.6.2 The same thing as a script

scripts/lab-up.sh

```
#!/usr/bin/env bash
# Build TaskFlow and deploy it, with all add-ons, to a fresh local kind cluster.
set -euo pipefail
TAG="${TAG:-0.1.0}"
IMG="ghcr.io/your-github-user"

kind create cluster --config deploy/kind-cluster.yaml

docker build -t "$IMG/taskflow-api:$TAG" --build-arg APP_VERSION="$TAG" api
docker build -t "$IMG/taskflow-web:$TAG" web
kind load docker-image --name taskflow "$IMG/taskflow-api:$TAG" "$IMG/taskflow-web:$TAG"

kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/
    ↪ components.yaml
kubectl -n kube-system patch deployment metrics-server --type=json \
  -p='[{"op":"add","path":"/spec/template/spec/containers/0/args/-","value":"--kubelet-insecure-
    ↪ tls"}]'

helm install eg oci://docker.io/envoyproxy/gateway-helm --version v1.5.0 \
  -n envoy-gateway-system --create-namespace --wait
kubectl apply -f deploy/gatewayclass.yaml

kubectl apply -k k8s/overlays/dev
for r in statefulset/postgres deployment/api deployment/web; do
  kubectl -n taskflow-dev rollout status "$r" --timeout=180s
done
echo "TaskFlow is running. Port-forward the gateway as shown in Step 5."
```

```
>_ scripts/lab-down.sh
#!/usr/bin/env bash
set -euo pipefail
kind delete cluster --name taskflow
```

23.7 Step 6 — Day-2 drills

This is where Kubernetes earns its keep. Run each drill and *watch* what happens; every one exercises a concept from earlier chapters.

23.7.1 Drill 1: zero-downtime rolling update

In one terminal, hammer the API and print only failures:

```
>_ terminal 1
while true; do
  curl -s -o /dev/null -w "%{http_code} " localhost:8888/api/info | grep -v 200 || true
  sleep 0.1
done
```

In another, build version 0.2.0 and roll it out declaratively:

```
>_ terminal 2
docker build -t ghcr.io/your-github-user/taskflow-api:0.2.0 --build-arg APP_VERSION=0.2.0 api
kind load docker-image --name taskflow ghcr.io/your-github-user/taskflow-api:0.2.0

cd k8s/overlays/dev && kustomize edit set image ghcr.io/your-github-user/taskflow-api:0.2.0 && cd
    ↪ -
#   (or edit newTag by hand)
kubectl apply -k k8s/overlays/dev
kubectl -n taskflow-dev rollout status deployment/api
curl -s localhost:8888/api/info                  # "version":"0.2.0"
```

Terminal 1 should stay silent: readiness probes, `maxUnavailable: 0`, the `preStop` sleep, and graceful shutdown work together so no request fails.

23.7.2 Drill 2: rollback

```
>_ terminal
kubectl -n taskflow-dev rollout history deployment/api
kubectl -n taskflow-dev rollout undo deployment/api
curl -s localhost:8888/api/info                  # back to 0.1.0
```

Key Idea

`rollout undo` is an emergency brake. Your Git overlay still says 0.2.0, so the next `apply` would roll forward again. In the GitOps setup of Step 9, you roll back by *reverting the commit* instead — keeping Git and the cluster in agreement.

23.7.3 Drill 3: self-healing

```
>_ terminal
kubectl -n taskflow-dev get pods -w &                       # watch in the background
kubectl -n taskflow-dev delete pod -l app.kubernetes.io/name=api --wait=false
# New api Pods appear within seconds; the HPA minimum of 2 is restored.
kill %1                                                     # stop the background watch
```

23.7.4 Drill 4: data survives Pod loss

```
>_ terminal
curl -s -X POST localhost:8888/api/tasks -H 'Content-Type: application/json' -d '{"title":"Survive
    ↪ a restart"}'
kubectl -n taskflow-dev delete pod postgres-0
kubectl -n taskflow-dev wait --for=condition=Ready pod/postgres-0 --timeout=120s
curl -s localhost:8888/api/tasks                  # the task is still there: the PVC was re-
    ↪ attached
```

While Postgres restarts, `/readyz` fails, so the API Pods are briefly taken out of rotation instead of returning errors — and they are *not* restarted, because liveness does not depend on the database.

23.7.5 Drill 5: node maintenance

```
>_ terminal
kubectl get pods -n taskflow-dev -o wide          # note which node each Pod is on
kubectl drain taskflow-worker --ignore-daemonsets --delete-emptydir-data
# PDBs ensure at least one api and one web Pod stay up while Pods move.
kubectl get pods -n taskflow-dev -o wide
kubectl uncordon taskflow-worker
```

Watch Out

With only one Postgres replica, draining its node causes a short database outage — a PDB cannot prevent that, it can only delay the drain. This is exactly why production databases use replicated Operators or managed services.

23.7.6 Drill 6: autoscaling under load

```
>_ terminal 1
kubectl -n taskflow-dev get hpa api -w
```

```
>_ terminal 2
# 8 parallel clients, each request burns 200 ms of CPU
for i in $(seq 8); do
  ( while true; do curl -s -o /dev/null "localhost:8888/api/work?ms=200"; done ) &
done
```

```
sleep 180
kill $(jobs -p)
```

Within a minute or two, CPU climbs far above 70 % of the 100m request and the HPA scales `api` towards 6 replicas. Stop the load and, after the 2-minute stabilisation window, it scales back to 2. Run `kubectl top pods -n taskflow-dev` to see the numbers yourself.

23.7.7 Drill 7: prove the network policies

```
>_ terminal
# The API may reach Postgres:
kubectl -n taskflow-dev exec deploy/api -- \
  python -c "import socket; socket.create_connection(('postgres', 5432), 3); print('api ->
    ↪ postgres: OPEN')"

# The web Pods may not reach the API or the database:
kubectl -n taskflow-dev exec deploy/web -- wget -T 3 -qO- http://api/api/info \
  || echo "web -> api: BLOCKED"
kubectl -n taskflow-dev exec deploy/web -- nc -z -w 3 postgres 5432 \
  || echo "web -> postgres: BLOCKED"
```

23.7.8 Drill 8: Pod Security in action

```
>_ terminal
kubectl -n taskflow-dev run rooty --image=nginx:1.27
# Error from server (Forbidden): pods "rooty" is forbidden: violates PodSecurity "restricted:
    ↪ latest":
#   allowPrivilegeEscalation != false, unrestricted capabilities, runAsNonRoot != true,
    ↪ seccompProfile ...
```

23.8 Step 7 — Monitoring and alerting

23.8.1 Install the monitoring stack

```
>_ terminal
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
helm install monitoring prometheus-community/kube-prometheus-stack \
  -n monitoring --create-namespace --wait
```

23.8.2 Scrape the API and define alerts

deploy/monitoring/servicemonitor.yaml

```yaml
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: taskflow-api
  namespace: taskflow-dev
  labels:
    release: monitoring              # matched by the chart's default selector
spec:
  selector:
    matchLabels:
      app.kubernetes.io/name: api
  endpoints:
    - port: http
      path: /metrics
      interval: 15s
```

deploy/monitoring/alerts.yaml

```yaml
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: taskflow-alerts
  namespace: taskflow-dev
  labels:
    release: monitoring
spec:
  groups:
    - name: taskflow.rules
      rules:
        - alert: TaskFlowAPIDown
          expr: absent(up{namespace="taskflow-dev", service="api"} == 1)
          for: 2m
          labels: { severity: critical }
          annotations:
            summary: "No healthy TaskFlow API targets for 2 minutes"
        - alert: TaskFlowHighErrorRate
          expr: |
            sum(rate(http_requests_total{namespace="taskflow-dev", status="5xx"}[5m]))
              / sum(rate(http_requests_total{namespace="taskflow-dev"}[5m])) > 0.05
          for: 5m
          labels: { severity: warning }
          annotations:
            summary: "More than 5% of TaskFlow API requests are failing"
        - alert: TaskFlowSlowRequests
          expr: |
            histogram_quantile(0.95,
              sum by (le) (rate(http_request_duration_seconds_bucket{namespace="taskflow-dev"}[5m
    ↪ ]))) > 0.5
          for: 10m
          labels: { severity: warning }
          annotations:
            summary: "p95 latency above 500 ms"
        - alert: TaskFlowPodRestarting
```

```
        expr: increase(kube_pod_container_status_restarts_total{namespace="taskflow-dev"}[15m])
    ↪ > 3
        labels: { severity: warning }
        annotations:
          summary: "{{ $labels.pod }} restarted more than 3 times in 15 minutes"
```

terminal

```
kubectl apply -f deploy/monitoring/

# Prometheus UI -> Status -> Targets: taskflow-api should be UP
kubectl -n monitoring port-forward svc/monitoring-kube-prometheus-prometheus 9090:9090 &

# Grafana (user admin)
kubectl -n monitoring get secret monitoring-grafana -o jsonpath='{.data.admin-password}' | base64
    ↪ -d; echo
kubectl -n monitoring port-forward svc/monitoring-grafana 3000:80 &
```

In Grafana, open **Dashboards › Kubernetes / Compute Resources / Namespace (Pods)** and select `taskflow-dev`. Re-run the load drill and watch CPU rise and new Pods appear. Then create a panel with:

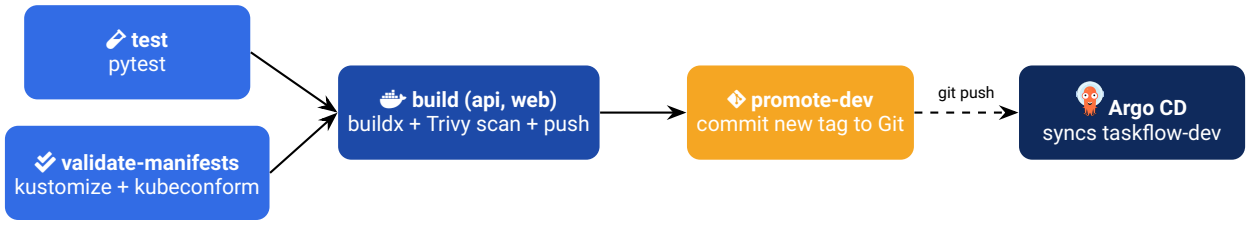
PromQL

```
sum by (pod) (rate(http_requests_total{namespace="taskflow-dev", handler="/api/work"}[1m]))
```

You now see requests per second *per Pod* — the HPA spreading load in real time.

23.9 Step 8 — Continuous Integration

The pipeline runs on every pull request and every push to `main`:



.github/workflows/ci.yaml

```
name: ci

on:
  pull_request:
  push:
    branches: [main]
    paths-ignore:
      - "k8s/overlays/**"          # tag bumps made by this pipeline do not re-trigger it

env:
  REGISTRY: ghcr.io
```

```yaml
jobs:
  test:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: api
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
          cache: pip
          cache-dependency-path: api/requirements*.txt
      - run: pip install -r requirements.txt -r requirements-dev.txt
      - run: pytest -q

  validate-manifests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Install kubeconform
        run: |
          curl -sSL https://github.com/yannh/kubeconform/releases/latest/download/kubeconform-
    ↪ linux-amd64.tar.gz | tar xz
          sudo mv kubeconform /usr/local/bin/
      - name: Render and validate every overlay
        run: |
          for overlay in k8s/overlays/*; do
            echo "::group::$overlay"
            kustomize build "$overlay" | kubeconform -strict -summary \
              -schema-location default \
              -schema-location 'https://raw.githubusercontent.com/datreeio/CRDs-catalog/main/{{.
    ↪ Group}}/{{.ResourceKind}}_{{.ResourceAPIVersion}}.json'
            echo "::endgroup::"
          done

  build:
    needs: [test, validate-manifests]
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write
    strategy:
      matrix:
        component: [api, web]
    steps:
      - uses: actions/checkout@v4
      - name: Compute image name (GHCR needs lower case)
        run: echo "IMAGE=${REGISTRY}/${GITHUB_REPOSITORY_OWNER,,}/taskflow-${{ matrix.component }}
    ↪ " >> "$GITHUB_ENV"
      - uses: docker/setup-buildx-action@v3
      - name: Build image
        uses: docker/build-push-action@v6
        with:
          context: ${{ matrix.component }}
          load: true                    # keep it locally so we can scan before pushing
```

```
          tags: ${{ env.IMAGE }}:${{ github.sha }}
          build-args: APP_VERSION=${{ github.sha }}
          cache-from: type=gha,scope=${{ matrix.component }}
          cache-to: type=gha,mode=max,scope=${{ matrix.component }}
      - name: Scan image for HIGH/CRITICAL vulnerabilities
        uses: aquasecurity/trivy-action@0.28.0
        with:
          image-ref: ${{ env.IMAGE }}:${{ github.sha }}
          severity: CRITICAL,HIGH
          ignore-unfixed: true
          exit-code: "1"
      - name: Log in to GHCR
        if: github.event_name == 'push'
        uses: docker/login-action@v3
        with:
          registry: ${{ env.REGISTRY }}
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}
      - name: Push image
        if: github.event_name == 'push'
        run: docker push "${IMAGE}:${GITHUB_SHA}"

  promote-dev:
    needs: build
    if: github.event_name == 'push'
    runs-on: ubuntu-latest
    permissions:
      contents: write
    steps:
      - uses: actions/checkout@v4
      - name: Point the dev overlay at the new images
        working-directory: k8s/overlays/dev
        run: |
          OWNER="${GITHUB_REPOSITORY_OWNER,,}"
          kustomize edit set image \
            "ghcr.io/your-github-user/taskflow-api=ghcr.io/${OWNER}/taskflow-api:${GITHUB_SHA}" \
            "ghcr.io/your-github-user/taskflow-web=ghcr.io/${OWNER}/taskflow-web:${GITHUB_SHA}"
      - name: Commit and push
        run: |
          git config user.name  "taskflow-ci[bot]"
          git config user.email "taskflow-ci@users.noreply.github.com"
          git commit -am "deploy(dev): taskflow ${GITHUB_SHA::7}"
          git push
```

Pro Tip

GHCR packages are private by default. Either make the two packages public (*Package settings* › *Change visibility*) or create an `imagePullSecret` in the namespace and reference it from the ServiceAccounts. Also set the SHA as the version: every image is traceable to the exact commit.

23.10 Step 9 — GitOps with Argo CD

23.10.1 Install Argo CD

terminal

```
kubectl create namespace argocd
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/
    ↪ install.yaml
kubectl -n argocd rollout status deployment/argocd-server

kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath='{.data.password}' | base64 -
    ↪ d; echo
kubectl -n argocd port-forward svc/argocd-server 8443:443 &
# https://localhost:8443  (user: admin)
```

23.10.2 Hand TaskFlow over to Argo CD

deploy/argocd/taskflow-dev.yaml

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: taskflow-dev
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/your-github-user/taskflow.git
    targetRevision: main
    path: k8s/overlays/dev
  destination:
    server: https://kubernetes.default.svc
    namespace: taskflow-dev
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=false          # our overlay already defines the Namespace
```

terminal

```
kubectl apply -f deploy/argocd/taskflow-dev.yaml
kubectl -n argocd get applications           # SYNC STATUS: Synced, HEALTH: Healthy
```

23.10.3 The GitOps loop, end to end

1. Change something visible in `web/index.html` (say, the heading) on a branch and open a pull request. CI runs *test* and *validate-manifests*, and builds and scans the images.
2. Merge it. CI pushes images tagged with the commit SHA and commits the new tags to `k8s/overlays/dev`.
3. Argo CD notices the commit (within about three minutes, or instantly with a webhook) and performs a rolling update. Refresh the browser: your change is live.

4. Something wrong? `git revert` the bot's commit. Argo CD rolls back.

> **Try It Yourself**
>
> Prove self-healing of *configuration*: run `kubectl -n taskflow-dev scale deployment web -replicas=0`. Within seconds Argo CD detects the drift from Git and restores two replicas. In GitOps, **Git always wins**.

23.10.4 Promoting to production

Production gets its own Application pointing at `k8s/overlays/prod`, usually on a separate cluster, *without* automated sync or with a manual approval gate. A release is a reviewed pull request that changes `newTag` in the prod overlay to an image SHA that has already been running happily in dev.

23.11 Optional — Packaging TaskFlow as a Helm chart

If you want to share TaskFlow, or let others install it with one command, package the same manifests as a chart.

charts/taskflow/Chart.yaml

```yaml
apiVersion: v2
name: taskflow
description: TaskFlow - the Kubernetes Handbook capstone application
type: application
version: 0.1.0          # chart version
appVersion: "0.1.0"     # default image tag
```

charts/taskflow/values.yaml

```yaml
image:
  registry: ghcr.io/your-github-user
  tag: ""               # defaults to .Chart.AppVersion
api:
  autoscaling: { minReplicas: 2, maxReplicas: 6, targetCPU: 70 }
  resources:
    requests: { cpu: 100m, memory: 128Mi }
    limits:   { memory: 256Mi }
web:
  replicas: 2
postgres:
  storage: 2Gi
  existingSecret: taskflow-db    # created out-of-band (ExternalSecret, Sealed Secret...)
gateway:
  className: eg
logLevel: info
```

charts/taskflow/templates/api-deployment.yaml (excerpt)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
```

```
  name: {{ .Release.Name }}-api
  labels:
    app.kubernetes.io/name: api
    app.kubernetes.io/instance: {{ .Release.Name }}
spec:
  selector:
    matchLabels:
      app.kubernetes.io/name: api
      app.kubernetes.io/instance: {{ .Release.Name }}
  template:
    metadata:
      labels:
        app.kubernetes.io/name: api
        app.kubernetes.io/instance: {{ .Release.Name }}
      annotations:
        checksum/config: {{ include (print $.Template.BasePath "/configmap.yaml") . | sha256sum }}
    spec:
      containers:
        - name: api
          image: "{{ .Values.image.registry }}/taskflow-api:{{ .Values.image.tag | default .Chart.
    ↪ AppVersion }}"
          resources:
            {{- toYaml .Values.api.resources | nindent 12 }}
          # ... probes, env and securityContext exactly as in k8s/base/api.yaml ...
```

>_ terminal

```
helm lint charts/taskflow
helm template demo charts/taskflow | less
helm upgrade --install demo charts/taskflow -n taskflow-helm --create-namespace --atomic --wait
```

23.12 What you have built

Concept (chapter)	Where it appears in TaskFlow
Containers (1)	Multi-stage, non-root images for api and web.
Pods, probes, shutdown (6)	Startup/liveness/readiness probes, `preStop`, graceful `SIGTERM`.
Labels, namespaces (7)	Recommended labels; one namespace per environment.
Deployments (8)	Zero-downtime rolling updates and rollbacks.
StatefulSets (9)	PostgreSQL with its own PVC.
Services, DNS (10)	`api`, `web`, headless `postgres`.
Gateway API, NetworkPolicy (11)	Envoy Gateway routes; default-deny zero-trust policies.
ConfigMaps, Secrets (12)	Generated config; dev secret; ExternalSecret in prod.
Storage (13)	`volumeClaimTemplates`; Retain class in prod.
Resources, scheduling (14)	Requests/limits, topology spread over zones, PDBs.
Autoscaling (15)	HPA on CPU, demonstrated under load.
Security (16)	Restricted Pod Security, non-root, read-only FS, no SA tokens.
Observability (17)	ServiceMonitor, PrometheusRule alerts, Grafana dashboards.
Helm, Kustomize (18)	Base + overlays; optional chart.
CI/CD, GitOps (20)	GitHub Actions + Trivy + Argo CD.

23.13 Where to go next

- Replace the StatefulSet with a 3-instance **CloudNativePG** cluster with backups.
- Add **cert-manager** and an HTTPS listener on the Gateway.
- Do a **canary release** with weighted `backendRefs` or Argo Rollouts.
- Sign images with **cosign** in CI and verify them with Kyverno.
- Add **OpenTelemetry** tracing to the API.
- Deploy the prod overlay to a managed cluster (EKS, GKE, or AKS).
- Prepare for the **CKAD**, **CKA**, and **CKS** certifications.

```terminal
# Clean up everything
kind delete cluster --name taskflow
```

Chapter Recap

- You built a three-tier app and ran it on a secure, observable, autoscaling Kubernetes setup.
- Kustomize overlays separate environments; GitOps keeps them in Git.
- CI tests, validates, scans, publishes, and promotes automatically.
- The drills showed self-healing, zero-downtime updates, rollbacks, persistence, scaling, and isolation — the reasons Kubernetes exists.



CHAPTER A

kubectl Cheat Sheet



A.1 Context and configuration

terminal

```
kubectl config get-contexts
kubectl config use-context <name>
kubectl config set-context --current --namespace=<ns>
kubectl cluster-info
kubectl version
kubectl api-resources
kubectl explain <kind>.<field> --recursive
```

A.2 Viewing resources

terminal

```
kubectl get pods|deploy|svc|ing|cm|secret|pvc|nodes [-n ns | -A] [-o wide|yaml|json|name]
kubectl get pods -l app=web --show-labels
kubectl get pods --field-selector status.phase=Running
kubectl get pods --sort-by=.metadata.creationTimestamp
kubectl get pods -w
kubectl describe <kind> <name>
kubectl get events --sort-by=.lastTimestamp
kubectl get all -n <ns>
kubectl top nodes | pods
```

A.3 Creating and changing

terminal

```
kubectl apply -f file.yaml | -f dir/ | -k overlay/
kubectl diff -f file.yaml
kubectl delete -f file.yaml
kubectl create deployment web --image=nginx:1.27 --replicas=3 --dry-run=client -o yaml
kubectl expose deployment web --port=80 --target-port=8080
kubectl create configmap cfg --from-literal=KEY=value --from-file=app.conf
kubectl create secret generic s --from-literal=password=xyz
kubectl edit deployment web
kubectl patch deployment web -p '{"spec":{"replicas":5}}'
kubectl label pod p env=dev --overwrite
```

```
kubectl annotate deployment web owner=team-a
kubectl scale deployment web --replicas=5
kubectl autoscale deployment web --cpu-percent=70 --min=2 --max=10
```

A.4 Rollouts

terminal

```
kubectl set image deployment/web web=nginx:1.28
kubectl rollout status deployment/web
kubectl rollout history deployment/web
kubectl rollout undo deployment/web [--to-revision=N]
kubectl rollout restart deployment/web
kubectl rollout pause|resume deployment/web
```

A.5 Debugging

terminal

```
kubectl logs <pod> [-c container] [-f] [--previous] [--since=10m] [--tail=100]
kubectl logs -l app=web --all-containers --prefix
kubectl exec -it <pod> [-c container] -- sh
kubectl port-forward pod/<pod> | svc/<svc> 8080:80
kubectl debug -it <pod> --image=busybox:1.36 --target=<container>
kubectl debug node/<node> -it --image=busybox:1.36
kubectl run tmp --rm -it --image=busybox:1.36 -- sh
kubectl cp <pod>:/path/file ./file
kubectl auth can-i <verb> <resource> [--as=user] [-n ns]
```

A.6 Nodes

terminal

```
kubectl get nodes -o wide
kubectl describe node <node>
kubectl cordon | uncordon <node>
kubectl drain <node> --ignore-daemonsets --delete-emptydir-data
kubectl taint nodes <node> key=value:NoSchedule[-]
kubectl label node <node> disktype=ssd
```

A.7 JSONPath and output tricks

```
>_ terminal
kubectl get pods -o jsonpath='{range .items[*]}{.metadata.name}{"\t"}{.status.phase}{"\n"}{end}'
kubectl get nodes -o custom-columns=NAME:.metadata.name,CPU:.status.capacity.cpu,MEM:.status.
    ↪ capacity.memory
kubectl get secret s -o jsonpath='{.data.password}' | base64 -d
kubectl get deploy -A -o jsonpath='{.items[*].spec.template.spec.containers[*].image}' | tr ' ' '\
    ↪ n' | sort -u
```

A.8 Short names

Short	Kind	Short	Kind
po	Pod	svc	Service
deploy	Deployment	ing	Ingress
rs	ReplicaSet	netpol	NetworkPolicy
sts	StatefulSet	cm	ConfigMap
ds	DaemonSet	pv / pvc	(Claim) PersistentVolume
cj	CronJob	sc	StorageClass
hpa	HorizontalPodAutoscaler	sa	ServiceAccount
pdb	PodDisruptionBudget	crd	CustomResourceDefinition
ns	Namespace	no	Node
ep	Endpoints	ev	Event

A.9 The resource icon set

The official Kubernetes icons used throughout this handbook:



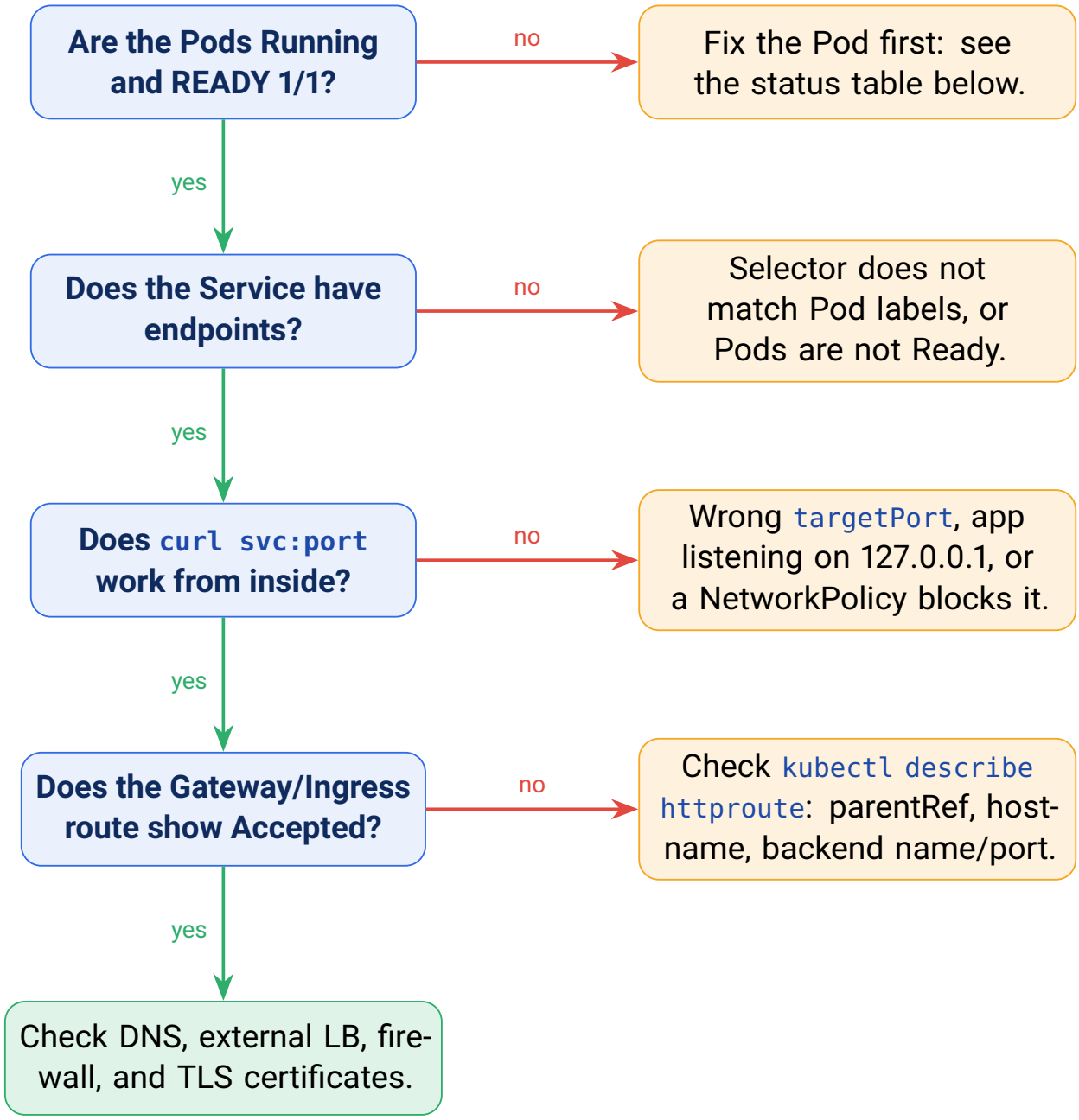


When something is wrong, resist the urge to guess. Work from the outside in, and let Kubernetes tell you what it knows. Nearly every problem is solved by some combination of `get`, `describe`, `logs`, and `events`.

B.1 The universal first five commands

```terminal
kubectl get pods -n <ns> -o wide                # 1. what state is everything in?
kubectl describe pod <pod> -n <ns>              # 2. read the Events at the bottom
kubectl logs <pod> -n <ns> [--previous]         # 3. what did the app say?
kubectl get events -n <ns> --sort-by=.lastTimestamp   # 4. what did the cluster do?
kubectl get endpointslices -n <ns> -l kubernetes.io/service-name=<svc>   # 5. is the Service wired
    ↪ up?
```

B.2 Decision tree: my app is not reachable



B.3 Pod status reference

Symptom	Likely cause	What to check / do
`Pending`	Not enough CPU/memory; nodeSelector/affinity/taint mismatch; PVC unbound.	`describe pod` events ("0/3 nodes are available…"); `get pvc`; lower requests or add nodes.
`ImagePullBackOff` `ErrImagePull`	Typo in image/tag; private registry; rate limits.	`describe pod`; verify the tag exists; add `imagePullSecrets`; for kind, `kind load`.
`CrashLoopBackOff`	App exits on start: bad config, missing env, failing dependency, wrong command.	`logs -previous`; check env/ConfigMap/Secret; run the image locally.
`OOMKilled`	Memory limit too low or a leak.	`describe pod` (Last State); raise the limit; profile memory.
`CreateContainerConf`	Missing ConfigMap/Secret or key.	`describe pod`; `get cm,secret`; check names after Kustomize hashing.
Running but 0/1 READY	Readiness probe failing.	`describe pod` probe messages; hit the endpoint with `exec`/`port-forward`.
Restarts keep rising	Liveness probe too strict, or real crashes.	Add/relax a `startupProbe`; check `logs -previous`.
`Terminating` forever	Finalizers, or an unreachable node.	`get pod -o yaml` (finalizers); as a last resort `delete -grace-period=0 -force`.
`Evicted`	Node under memory/disk pressure.	`describe node`; set requests; clean disk; check ephemeral storage.

B.4 Networking checks

terminal

```
# A throw-away debug Pod with networking tools
kubectl run netshoot --rm -it --image=nicolaka/netshoot -- bash
#   nslookup api.taskflow-dev.svc.cluster.local
#   curl -v http://api.taskflow-dev/api/info
#   nc -zv postgres.taskflow-dev 5432

kubectl get networkpolicy -n <ns>
kubectl -n kube-system logs -l k8s-app=kube-dns          # CoreDNS problems
kubectl get gateway,httproute -A                          # route status and conditions
```

Watch Out

If the namespace enforces the `restricted` Pod Security profile, a plain `kubectl run` debug

> Pod will be rejected. Debug from another namespace, use `kubectl debug` on an existing Pod, or pass a compliant `securityContext` with `-overrides`.

B.5 Node and cluster checks

```terminal
kubectl get nodes                                  # NotReady?
kubectl describe node <node>                       # Conditions: MemoryPressure, DiskPressure,
    ↪ PIDPressure
kubectl get --raw='/readyz?verbose'                # API server health checks
kubectl get pods -n kube-system                    # control-plane and add-on Pods healthy?
```

B.6 Habits that prevent incidents

- Always `kubectl diff` before `apply` in shared environments.
- Show the current context in your shell prompt.
- Never use `latest` tags; pin versions or digests.
- Set requests and limits on everything.
- Keep everything in Git; avoid `kubectl edit` in production.
- Write a runbook entry for every incident you solve.



CHAPTER C

Glossary and Learning Path



C.1 Glossary

Admission controller
Code in the API server that can modify or reject requests after authentication and authorisation (e.g. Pod Security, ValidatingAdmissionPolicy).

Annotation
Non-identifying key/value metadata used by tools and humans.

API server
The control-plane component that serves the Kubernetes API; the only component that talks to etcd.

CNI
Container Network Interface: the plugin standard for Pod networking.

ConfigMap
An object holding non-secret configuration data.

Container runtime
Software that runs containers on a node (containerd, CRI-O).

Context
A kubeconfig entry combining a cluster, a user, and a default namespace.

Controller
A loop that watches objects and reconciles actual state toward desired state.

CRD
CustomResourceDefinition: adds a new resource type to the API.

CronJob
Creates Jobs on a schedule.

CSI
Container Storage Interface: the plugin standard for storage.

DaemonSet
Runs one Pod per (selected) node.

Deployment
Manages ReplicaSets to run and update stateless Pods.

Desired state
What you declared in an object's `spec`.

EndpointSlice
The list of Pod IPs/ports currently backing a Service.

etcd
The consistent key–value store holding all cluster state.

Gateway API
The modern, role-oriented API for L4/L7 traffic routing (GatewayClass, Gateway, HTTPRoute…).

GitOps
Operating model in which Git holds desired state and an in-cluster agent continuously applies it.

Helm
The Kubernetes package manager; packages are called charts.

HPA
HorizontalPodAutoscaler: scales the number of replicas on metrics.

Ingress
The original API for HTTP routing into the cluster.

Init container
A container that runs to completion before the app containers start.

Job
Runs Pods until a number of them complete successfully.

kubeconfig
The file `kubectl` uses to find clusters and credentials.

kubelet
The node agent that runs Pods and reports their status.

kube-proxy
Programs node networking so Service virtual IPs reach Pods.

Kustomize
Template-free customisation of YAML through bases and overlays.

Label / selector
Identifying key/value pairs, and queries over them.

LimitRange
Per-namespace defaults and bounds for container resources.

Liveness probe
Health check whose failure restarts the container.

Namespace
A virtual partition of a cluster for names, RBAC, and quotas.

NetworkPolicy
Allow-list rules for Pod network traffic.

Node
A worker machine (VM or physical) in the cluster.

Operator
A controller that encodes operational expertise for specific software.

PDB
PodDisruptionBudget: limits voluntary disruptions of an app.

Pod
The smallest deployable unit: one or more containers sharing network and storage.

PV / PVC
PersistentVolume (actual storage) / PersistentVolumeClaim (a request for it).

QoS class
Guaranteed, Burstable, or BestEffort — derived from requests and limits.

RBAC
Role-Based Access Control: Roles/ClusterRoles bound to subjects.

Readiness probe
Health check whose failure removes the Pod from Service endpoints.

ReplicaSet
Keeps a fixed number of identical Pods running.

Request / limit
Guaranteed / maximum amount of CPU or memory for a container.

ResourceQuota
A per-namespace cap on total resource consumption.

Scheduler
Assigns unscheduled Pods to nodes.

Secret
An object holding sensitive data (base64-encoded, not encrypted by default).

Service
A stable virtual IP and DNS name that load-balances to selected Pods.

ServiceAccount
An identity for processes running in Pods.

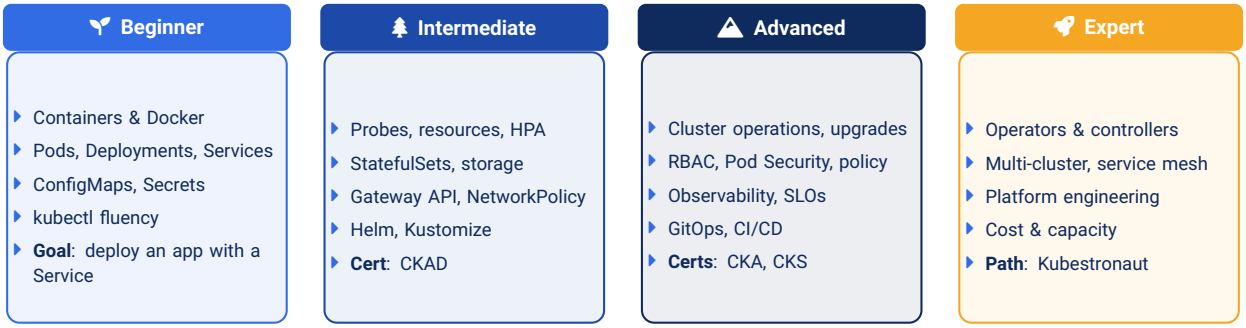
Sidecar
A helper container that runs alongside the main container for its whole life.

StatefulSet
Runs Pods with stable identities and per-Pod storage.

StorageClass
Describes a type of storage and enables dynamic provisioning.

Taint / toleration
Node-side repulsion and the Pod-side permission to ignore it.

C.2 Your learning path



C.3 Certifications

Exam	Focus	Good preparation
KCNA	Cloud-native fundamentals (multiple choice).	Parts I–III of this book.
CKAD	Building and running applications (hands-on).	Parts II–IV, and the capstone.
CKA	Administering clusters (hands-on).	Parts I, V, VI; kubeadm practice.
CKS	Securing clusters and workloads (hands-on, requires CKA).	Chapter 16 in depth, Falco, supply chain.

C.4 Where to keep learning

- **Official documentation**: `https://kubernetes.io/docs/` — the tasks and tutorials sections especially.
- **Kubernetes the Hard Way** (Kelsey Hightower): build a cluster by hand to understand every component.
- **Gateway API docs**: `https://gateway-api.sigs.k8s.io/`.
- **CNCF Landscape**: `https://landscape.cncf.io/` — the map of the ecosystem.
- **KubeCon talks** on YouTube — real-world experience from people running Kubernetes at scale.
- **Play**: break your kind cluster on purpose, then fix it. That is how experts are made.



Happy sailing, helmsman.

You now know how to steer.